Proxmox Storage DRS

Balancing disk I/O across shared storages on Proxmox VE 9.2

Bernd Zeimetz bernd@bzed.de

2026-10-13

Rendered from `IMPLEMENTATION_PLAN.md`, SHA-256 `4a02808ed86cd55db5ed2d510e37515dc1713046c9fb8460fc1459275a135c2d` (`sha256sum --check docs/IMPLEMENTATION_PLAN.pdf.sha256` in the repository must succeed).

Copyright © 2026 Bernd Zeimetz. Licensed under the GNU Affero General Public License, version 3 or later.

Contents

1. Scope . . . 5
   - Goal . . . 5
   - Non-goals . . . 5
   - Relationship to the PVE 9.2 Dynamic Load Balancer . . . 5
   - Requirement interpretation: “minimal number of migrations” . . . 6
   - Requirement interpretation: “spread data evenly” . . . 6
   - Operating assumptions . . . 6
2. Architecture . . . 6
   - Module layout . . . 7
   - 2.1 Implementation, deployment and operations . . . 8
   - 2.2 Packaging and continuous integration . . . 9
   - 2.3 Logging . . . 9
     - The two audiences, and the one rule that separates them . . . 10
     - Levels . . . 10
     - Format . . . 11
     - Where the handler is attached . . . 11
     - An error is one fact, logged once . . . 11
     - The event catalogue . . . 11
     - What this does *not* add . . . 13
     - Verification . . . 14
     - The timer . . . 14
   - 2.4 Monitoring status file . . . 14
3. Data acquisition . . . 15
   - 3.1 Where the per-disk data comes from . . . 15
   - 3.2 Two paths deliberately not taken . . . 16
   - 3.3 Metric name mapping and verification . . . 16
   - 3.4 PromQL . . . 17
   - 3.5 PVE API . . . 19
   - 3.6 Which disks can actually be moved online . . . 22
   - 3.7 Disks with snapshots are excluded, loudly . . . 25
   - 3.8 Disks with an unapplied pending change are excluded . . . 25
4. The load model . . . 26
5. The optimization problem . . . 28
   - 5.1 Sets and data . . . 28
     - 5.1.1 Computing U^{sext} . . . 28
   - 5.2 Variables . . . 29
   - 5.3 Constraints . . . 29
     - 5.3.1 Configuring the free-space requirement: `free_space` . . . 32
     - 5.3.2 Enforcing a disk format on a storage: `enforce_format` . . . 36
     - 5.3.3 The per-VM snapshot footprint, and splitting large VMs . . . 38
   - 5.4 Objective . . . 40
   - 5.5 Solver backends . . . 42
6. Gating — deciding whether to act at all . . . 43
7. Migration cost and the payback rule . . . 45
   - 7.1 Cost . . . 45
   - 7.2 Benefit . . . 46
   - 7.3 Acceptance . . . 48
8. Migration ordering . . . 51

8.1 The transient invariant . . . 51
8.2 Scheduling algorithm . . . 52
8.3 Deadlock and staging . . . 53
9. Execution . . . 54
9.1 Modes . . . 54
9.2 Performing one move . . . 54
9.3 Locks, and why a completed move is not a finished move . . . 56
9.4 Failure handling . . . 58
9.5 Output . . . 58
10. Forecasting . . . 61
10.1 What is forecast, and what history it needs . . . 61
10.2 Implementation warnings . . . 62
11. Configuration . . . 62
11.1 Validation rules . . . 64
11.2 `state.json` . . . 65
11.3 Global command-line options . . . 65
11.4 Storage name patterns in `groups[].storages[]` . . . 66
12. Implementation phases . . . 68
12.1 Phase 14 in detail . . . 74
14a — remove the §7.3 saturation guard (one commit) — **done** . . . 74
14b — Holt-Winters-driven ℓ_d (one commit) — **done** . . . 75
13. Failure modes and safety . . . 76
14. Worked example . . . 79
14.1 Input . . . 79
14.2 Initial state . . . 80
14.3 Solution . . . 80
14.4 Ordering . . . 81
14.5 Payback . . . 82
14.6 Companion fixture: when the reserve and the balance objective disagree . . . 82
14.7 Companion fixture: affinity repair under the payback rule . . . 83
14.8 Companion fixture: the free-space repair mandate . . . 84
14.9 Companion fixture: splitting a large VM . . . 86
15. Requirements traceability . . . 88
15.1 Config knob → formula . . . 88
16. Diagnostic bundles: `collect-testdata` and `--replay` . . . 89
16.1 The bundle . . . 90
16.2 Capture: the superset, not the configured path . . . 90
16.3 Anonymization . . . 92
The governing rule: allowlist, never denylist . . . 92
The pseudonym function, and the salt . . . 92
Per-kind mapping . . . 93
The configuration in the bundle . . . 95
What is deliberately preserved . . . 95
Threat model, stated honestly . . . 95
16.4 The command . . . 96
16.5 Replay: the `--replay` global option . . . 96
16.6 The corpus: `tests/corpus/` . . . 97
What the suite asserts . . . 97
The variant matrix . . . 98

Size, and bundles too big to commit 98
16.7 Modules, phases and the honest status 99

A specification for a VMware Storage DRS replacement targeting **Proxmox VE 9.2**. It is written to be implementable without further research: every external fact it relies on is stated inline, and section 14 is a worked numeric example that doubles as a test fixture.

---

1. Scope

Goal

Given configurable **groups** of shared storages, continuously equalize **disk I/O load** across the storages within each group by live-migrating individual VM disks, subject to:

- a disk may only move between storages in **its own group**;
- every storage must always retain free space for snapshots — by default **2× the largest per-VM footprint on it**, where a VM's footprint on a storage is the sum of *all* its disks there, because a snapshot of a VM always snapshots every disk of that VM at once (§5.3.3) — and this must hold *during* migrations, not merely before and after;
- a VM whose disks add up to more than a configured size (default 2 TiB) should be **split** over several storages so that no single storage has to hold the snapshot reserve for all of it — a preference, weighted above the affinity preference below (§5.3.3);
- every storage must retain its **configured free space** — an absolute byte count or a percentage of its capacity, per storage or per LUN family (§5.3.1) — and when a new volume has eaten it, the engine migrates disks away until it is free again: the requirement is repaired, never traded (§5.3), and the larger of it and the snapshot reserve always wins;
- the number of migrations must be **minimal**;
- a VM's disks should stay **together** on one storage unless space, I/O or the split rule above forces otherwise — a preference weighted by the VM's own I/O (§5.4) and backed by the cost model: reuniting a VM counts as benefit in the payback test, and disks too small to cost anything move for free (§7.1–§7.3);
- data is spread evenly across a group's storages as a **second priority to I/O** — no single storage should hold a disproportionate share of the group's bytes, so the failure of any one storage costs a bounded share of the data;
- a migration's own I/O cost must not exceed the benefit it delivers, judged over a horizon that reflects how long the placement will actually last — a move's cost is paid in days of degraded I/O, its benefit accrues for as long as the VM keeps running there (default horizon: one year, configurable).

Non-goals

Compute/CPU/memory DRS, VM-to-node placement, HA, backup scheduling, storage provisioning, thin-pool overcommit management, and any modification of guest configuration beyond disk location.

Relationship to the PVE 9.2 Dynamic Load Balancer

PVE 9.2 ships a built-in Dynamic Load Balancer — a dynamic mode for the Cluster Resource Scheduler that continuously live-migrates **HA-managed guests between nodes** to even out node CPU/memory utilization. It is **complementary, not overlapping**: it balances guests across *hypervisors* and has no notion of storage or of the FC LUNs behind it. Nothing in PVE balances disk I/O across storages, which is the gap this tool fills.

There is, however, a real interaction: the built-in balancer may live-migrate a VM to another node *while a Storage DRS plan is executing*, invalidating the `{node}` in a queued `move_disk` call. The pre-move re-validation in §9.2 exists precisely to catch this — it re-reads the VM's current node before every move and re-plans on mismatch. Implementers must not cache the node across moves.

Requirement interpretation: "minimal number of migrations"

The requirement that migrations be minimal is implemented as a **tunable preference** (the β term in §5.4), not as a strict lexicographic minimum. A strict minimum would refuse a second cheap move that halves the remaining imbalance, which is not what is wanted. β sets the exchange rate between "one more migration" and "this much less imbalance"; §14.3 demonstrates β selecting a two-move plan over a three-move plan on the same input (shown there at `δ = 0`; at the default δ the capacity term reaches the same two-move plan for its own reason).

Requirement interpretation: "spread data evenly"

Even spread of **data** across a group's storages is a risk-reduction requirement — the failure of one storage costs the group whatever was on it — and it is subordinate to the I/O requirement: the load model of §4, in-flight I/O time by default with the ops/bytes terms available where the operator wants them, remains the primary quantity. Like the migration count, even spread is implemented as a **tunable preference** (the δ term in §5.4) rather than a strict "I/O first, bytes second" lexicographic order: two assignments' load objectives are essentially never exactly equal, so a strict order would reduce the capacity term to a tie-breaker that never acts. The term is priced in **absolute TiB** of misplaced data, in the same unit as the cost of the moves that fix it, so a given move is worth the same on a small group as on a large one. The default weight keeps I/O the first priority on an imbalanced group, gives δ the deciding vote among plans the load objective is nearly indifferent between, and lets it act on its own — behind §6's capacity gate — on a group whose I/O is already balanced but whose data is concentrated on few storages.

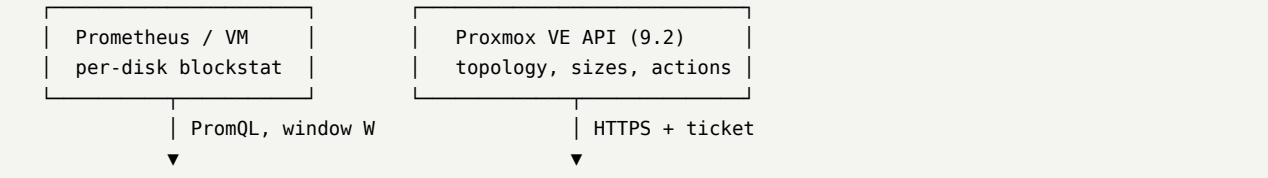
Operating assumptions

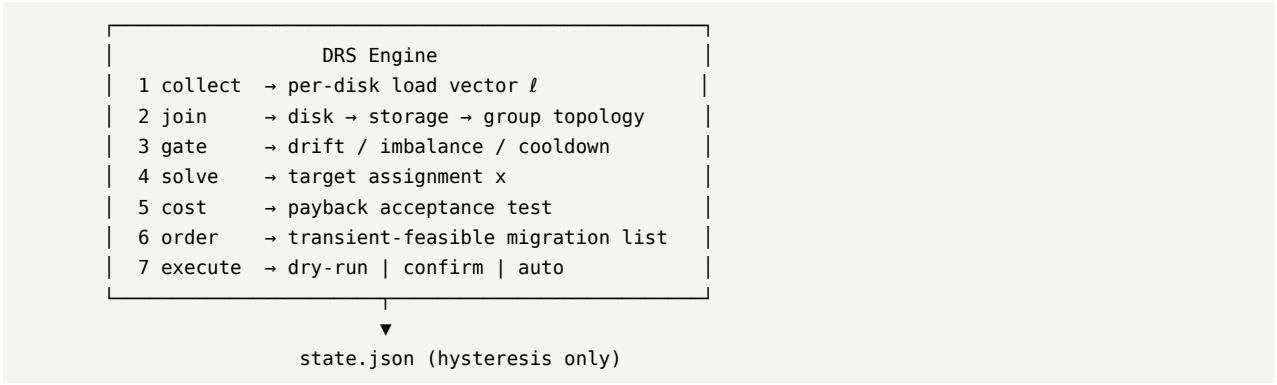
- PVE 9.2, shared LVM (typically over FC) or any shared storage supporting `move_disk`.
- Only **running** VMs are considered. A stopped VM generates no I/O to balance, and its disks are **never moved, whatever capacity or reserve violation they cause**: they count as foreign bytes (§5.1.1) and the planner works around them. Replication and backup products such as Veeam track a VM's volumes by location, and moving a replicated VM's disks breaks the replica. The `exclude.stopped` key exists only so a config that spells it out still loads; `false` is refused at load time, and the pre-move check (§9.2) refuses a VM that has stopped since planning.
- Disks are thick-provisioned by default, so migration cost is proportional to *provisioned* size.
- **A VM is never stopped, suspended or reconfigured.** The only write the engine issues is `move_disk`. This is what rules out the offline path for snapshotted disks (§3.7), which are movable only with the guest down — a maintenance-window decision for a human.
- Every bus is in scope and every one of them moves online: `ide`, `sata`, `scsi`, `virtio`, plus `efidisk0`, `tpmstate0` and `unused` volumes (§3.6). A VM's disks are not all `scsi*`, and anything not enumerated is capacity the model cannot see (§3.5).
- Storage-side cleanup is part of a migration, not an afterthought: with LVM `saferemove` the source volume is zeroed at a throttled rate after the mirror completes, and neither its space nor the VM is available until that finishes (§7.1, §9.3).

---

2. Architecture

The engine is **stateless with respect to metrics** — all history lives in Prometheus. It keeps one small local state file purely for hysteresis.





Two data sources are genuinely required, and neither substitutes for the other:

- **Prometheus** knows the load of `vmid=101, device=scsi0` but has **no idea which storage that disk lives on**.
- **The PVE API** knows the topology, sizes and capacities, and is the only way to *act*.

The join between them is the disk identity `(vmid, device)`, which both sides expose.

Module layout

Module	Responsibility
`config.py`	Load/validate YAML, defaults, unit parsing (`24h`, `200MiB`), `/…/` storage patterns (§11.4)
`metrics.py`	Prometheus client, PromQL construction, `verify-metrics`
`pve.py`	API client: auth, topology, storage status, `move_disk`, task polling
`topology.py`	Build the disk/storage/group model, eligibility rules, storage-pattern expansion (§11.4)
`loadmodel.py`	Reduce raw series to the scalar load vector ℓ
`forecast.py`	Holt-Winters forecast of a disk's p95 over the next window, and its backtest gate (§10, §12.1)
`optimize.py`	MILP formulation (CBC via `pulp`)
`heuristic.py`	Dependency-free greedy + local search fallback
`payback.py`	Migration cost model and acceptance test
`schedule.py`	Ordering under the transient reserve invariant
`execute.py`	Three execution modes, task supervision
`anonymize.py`	Keyed pseudonyms and field allowlists for diagnostic bundles (§16.3). Pure
`collect.py`	`collect-testdata`: capture, anonymize and write a diagnostic bundle (§16)
`replay.py`	Read a bundle back and serve it through the `metrics.py`/`pve.py` interfaces (§16.5)
`cli.py`	`plan`, `apply`, `verify-metrics`, `verify-storages`, `show-load`, `explain`, `collect-testdata`

The installed executable is **`pve-storage-drs`** — one `[project.scripts]` entry point onto `cli.py`. Every command in this document is written as `pve-storage-drs <subcommand>`; the manpage is `pve-storage-drs(1)`, and the Debian source and binary package are `pve-storage-drs` too. The Python package keeps its distribution name `proxmox-storage-drs` and its import name `proxmox_storage_drs`. The word **storage** is load-bearing: PVE 9.2's own Dynamic Load Balancer moves guests between nodes (§1), and a shorter name would suggest this tool replaces it.

2.1 Implementation, deployment and operations

Language: Python 3.11+. The decision is driven by the solver and forecasting libraries — `pulp` driving CBC, and `statsmodels`, have no usable equivalent in Go or Rust without substantial reimplementation, and PVE hosts already ship Python, so operators can read and patch the tool. Single-binary deployment is not a requirement here; if it ever becomes one, the dependency-free heuristic path (§5.5) is the portable subset worth porting.

Dependencies come from Debian. The deployment target runs Proxmox VE 9.x, which is built on Debian trixie, so a module that trixie packages is a module the operator already trusts, already patches through their normal update path, and already has on a host with no outbound network. Every dependency below is therefore chosen for being in trixie, and the tool is packaged as a `.deb` (§2.2). A module that is *not* in Debian is only acceptable if it is pure Python and small enough to vendor into our own source package; anything else must be optional, with a code path that works without it.

Package	Purpose	In Debian trixie
`requests`	Prometheus HTTP, and the transport `proxmoxer`'s https backend uses	`python3-requests` 2.32
`proxmoxer`	PVE API client (§3.5)	`python3-proxmoxer` 2.2
`ruamel.yaml`	Config (round-trips comments)	`python3-ruamel.yaml` 0.18
`jsonschema`	Config validation (§11.1)	`python3-jsonschema` 4.19
`pulp`	MILP via CBC — **the default, packaged solver path**	`python3-pulp` 2.7 + `coinor-cbc` 2.10 (both `Depends`)
`statsmodels`	Holt-Winters, optional	`python3-statsmodels` 0.14
`pytest`, `pytest-cov`, `pytest-xdist`	Tests; groups are independent so they parallelize	`python3-pytest*`

There is deliberately no `ortools` row. A CP-SAT backend built on it was specified here once, and removed (REVIEW.md AL-02): Debian does not carry `ortools`, it is not a candidate for vendoring (a large C++ extension, not a pure-Python module), and it would not be installed by hand on PVE hosts — so on every deployment `solver.backend: auto` resolved to CBC regardless, and a second model builder existed only for a backend production could never run. The MILP is solved by **CBC through `python3-pulp`**, with the dependency-free heuristic below that.

Make `statsmodels` an **optional extra** in `pyproject.toml`; `pulp` stays one too, for a plain `pip install` outside Debian, but `debian/control` treats it differently: `pulp` and `coinor-cbc` are `Depends`, not `Recommends`, so `apt install pve-storage-drs` always gets a real MILP solver by default — CBC-through-`pulp` is the primary solver on the deployment target, not a bonus for whoever remembers to add it. This is a packaging default, not a claim that the *code* needs a solver to run: the heuristic of §5.5 remains a genuine, exercised fallback — used whenever `pulp`/CBC genuinely are not importable or executable (a non-Debian install with no solver extra, or a broken one), whenever a solve fails or times out, or whenever `solver.backend: heuristic` is configured explicitly — and the code path that reaches it is still tested (`test_heuristic.py`, and `test_optimize.py`'s own "solver unavailable" branches), just no longer by installing the Debian package without its `Depends`, since that configuration no longer exists. The autopkgtest in §2.2 still imports every module of the installed package with only the binary package's `Depends` present — now including `pulp`/`coinor-cbc` — so it verifies the tool's real, default solver path rather than proving heuristic-only operation; an optional dependency (`statsmodels`) imported at module level still fails it.

Licence and contribution rules. The project is **AGPL-3.0-or-later**, copyright Bernd Zeimetz bernd@bzed.de; every source file carries the two-line SPDX header. `AGENTS.md` and `.agents/` hold the working agreement that any implementer — human or model — is expected to follow: black + isort + flake8 + mypy with configurations written so the formatter and the linter cannot disagree, an enforced 85% coverage

floor, feature branches merged only when `make check` is green, and the list of domain invariants (dry-run default, reserve never traded, no auto-delete) that a refactor must not quietly remove. A dependency whose licence is incompatible with AGPL-3.0-or-later cannot be added.

Deployment. Runs on a management host — not necessarily a PVE node — needing outbound access to the PVE API (tcp/8006) and to Prometheus. It holds credentials and must be treated accordingly; prefer an API token over username/password for unattended operation.

Cadence. Invoked by a systemd timer (or cron) every 15–30 min. Note the separation of concerns: the *invocation* cadence is independent of `execution.time_windows`, which constrain only when **moves execute**. The engine may plan at any time and simply decline to act outside the window. The drift and imbalance gates (§6) make frequent invocation cheap — most runs exit at a gate having issued only read queries.

Logging. Structured lines to **stderr**, never stdout, so that `--json`'s machine-readable report (section 9.5) can be captured from stdout alone with nothing interleaved. What is logged, at which level, in which format, and the audit trail an `auto` run owes a human afterwards are all specified in **section 2.3**.

2.2 Packaging and continuous integration

The tool is delivered as a Debian package, `pve-storage-drs`, built from `debian/` in this repository (source format `3.0 (native)`, since upstream and packaging are the same tree). It installs the executable, the manpage, the example configuration and the generated documentation. Two rules follow from that and are not negotiable:

- **`debian/control` is the single source of truth for dependencies.** Build-Depends and Depends are updated in the same commit as the code or documentation that needs them. CI installs the build dependencies *from* `debian/control` with `mk-build-deps`, so a stale declaration fails the build rather than working by accident on a developer's machine.
- **The autopkgtest asks the only question the build cannot.** The build chroot has the Build-Depends installed and therefore cannot notice a missing runtime dependency. `debian/tests` installs the built package on a system that has only its `Depends`, then runs `pve-storage-drs --version`, `pve-storage-drs --help`, and an import of every module in the package.

Two pipelines, deliberately different:

	Runs	Covers
GitHub Actions	`debian:trixie` containers	lint, types, tests and coverage with Debian's packaged tooling; the document build against trixie's older pandoc; `dpkg-buildpackage`, lintian, and install-then-run
GitLab CI (Salsa)	Debian's Salsa CI pipeline	sbuild in an unshare chroot with **no network**, then lintian, piuparts, reprotest and autopkgtest

The Salsa build having no network is the load-bearing part: it is what proves the package builds from trixie alone. If a Python module ever has to be fetched during a build, the answer is to vendor it into the source package; enabling `--enable-network` for sbuild is the fallback, and it is a deliberate, visible change to `debian/.gitlab-ci.yml`, not a default.

2.3 Logging

Status: implemented (phase 11). The event catalogue below is the built one; the three level corrections and the retraction at the end of this section have all landed.

Why this section exists. The first implementation got the plumbing right and the policy wrong. Streams are correct (report on stdout, log on stderr — verified), `--mode` overrides are logged exactly as section 11.3 requires, and the JSON formatter is sound. But a plain `pve-storage-drs explain` on a healthy cluster prints this to stderr before a single line of its own report:

```
{"event": "config_loaded", "level": "INFO", ...}
{"event": "config_warning", "level": "WARNING", ... "migration.payback_horizon (…s) is below 30 days ..."}
{"event": "config_warning", "level": "WARNING", ... "migration.payback_horizon (…s) is below 30 days ..."}
{"event": "storage_pattern_expanded", "level": "INFO", ...}
{"event": "optimize_backend_unavailable", "level": "WARNING", ... "solver.backend=cpsat requested but
ortools is not importable"}
```

Five machine-readable lines, on every run, for a run in which nothing whatsoever went wrong. Worse, the requirement that actually matters — "in `auto` mode this log is the only record a human will see" — is **not implemented at all**: `gates.py`, `loadmodel.py`, `schedule.py` and `payback.py` contain no logging calls of any kind, and `execute.py` never logs the UPID of a `move_disk` it issued. A timer running `apply --mode auto` today records that it started and nothing about what it did to the cluster. The policy below replaces the "Logging." paragraph of section 2.1.

The two audiences, and the one rule that separates them

A person at a terminal and a journal read three weeks later want opposite things. The person wants **silence unless something is wrong** — their report is the output, the log is not. The journal wants **every decision that changed the cluster**, in a form `journalctl` can filter and a script can parse. Both are served by the same records at different levels and in different formats; no record exists for only one of them.

The rule that follows, and the one to apply when adding any new log call: **a log record is a fact about this run's decisions or a problem with this run. A fact about static configuration belongs in a report, not in a log that repeats it every 15 minutes forever.**

Levels

Level	Contents	Seen by default
`ERROR`	The run failed and is exiting non-zero	yes
`WARNING`	Something degraded but the run continued: a fallback, a skipped move, a lock waited out, an orphaned volume, corrupt state	yes
`INFO`	The audit trail — every decision that changed, or was allowed to change, the cluster	only where mandated below, or with `-v`
`DEBUG`	Per-query, per-candidate, per-poll detail. Third-party library logs	`-vv` only

The ladder is `--quiet` (`ERROR`) < default (`WARNING`) < `-v` (`INFO`) < `-vv` (`DEBUG`), with one mandatory exception:

A run that can change the cluster logs its audit trail whether or not anyone asked for it. When the effective mode is `confirm` or `auto`, `apply` raises its own floor to `INFO`. This is not a convenience: section 2.1's "in `auto` mode this log is the only record a human will see" cannot be satisfied by an option the operator has to remember to pass, and an unattended timer that silently migrated 400 GiB is not acceptable output regardless of how the unit file was written. `--quiet` still wins — an operator may insist on silence — but the manual must state plainly that `--quiet` on an `auto` timer discards the only record of what was moved and why. `--quiet` is the *only* escape: an explicit `--log-level warning/error` on a `confirm`/`auto` run is raised back up to the floor rather than silently discarding the audit trail too (as built, REVIEW.md X-06 — this was, for a time, a second, undocumented way to lose it).

Read-only commands (`plan`, `explain`, `show-load`, `verify-metrics`, `verify-storages`, `collect-testdata`) and `apply --mode dry-run` keep the `WARNING` default: they change nothing, so there is nothing to audit, and their report already says everything.

Format

`--log-format {auto,text,json}`, default `auto`: **`text` when stderr is a TTY, `json` otherwise.** A person gets `WARNING: <message>`; journald, a pipe and a redirect all get the JSON object that is already implemented. This is the whole fix for "JSON in my terminal", it needs no flag in the common case, and it falls out correctly for a systemd unit without the unit having to know anything. Both explicit values override the sniffing, in both directions, for the operator whose terminal wants JSON or whose pipeline wants text.

`--log-level {error,warning,info,debug}` sets the level explicitly and wins over `-v`/`--quiet`. Automation states a level; it should not have to count vs to get one. The one exception is the mandatory floor above: a level below it on a `confirm`/`auto` run is raised back up, not honoured.

As built, revised after 0.1.9: `auto` no longer sniffs the stream. JSON as the non-TTY default put one JSON object per line into `journalctl -u pve-storage-drs`, which a person reads far more often than `jq`. The default is now `text` everywhere; `json` is opt-in with `--log-format json`, and `auto` stays accepted as a legacy alias for `text`. The reason to keep JSON at all is that it is the only record written as each move starts and finishes – the end-of-run `--json` report never appears for a run that dies mid-move. The paragraph above describes the original design.

Where the handler is attached

As built (REVIEW.md X-06): the single handler is attached to the **root** logger, not to `proxmox_storage_drs` — the opposite of what an earlier draft of this section specified, and a deliberate choice, not a drift: `handler-on-package` plus `propagate = False` was tried first and hides every record from pytest's `caplog` and from anything embedding this package as a library, since neither talks to a handler nested under `proxmox_storage_drs` that never propagates up. `root.handlers = [handler]` sidesteps that. The separation this section actually needs — `-v`/`-vv` raising *this tool's* verbosity without also turning on `urllib3`/`proxmoxer`/`statsmodels` debug output — comes from the *levels* instead of from which logger owns the handler: `proxmox_storage_drs`'s own logger carries the run's level, root (which every third-party logger inherits from, having no handler or level of its own) stays clamped to `WARNING` except under `-vv`. A record from either still reaches the one handler by propagating up to root, so nothing is emitted twice and nothing is silently dropped. The behavioural requirements below (third-party loggers quiet until `-vv`, no duplicate emission) hold either way; this is the mechanism that delivers them.

Every module keeps `logging.getLogger(__name__)` with one exception: **`cli.py` must use the explicit name `proxmox_storage_drs.cli`**, because `__name__` there is `__main__` whenever the module is executed rather than imported, and a `journalctl` filter keyed on a logger name that changes with how the process was started is not a filter. (Observed in the field: the same event appearing as both `"logger": "__main__"` and `"logger": "proxmox_storage_drs.cli"`.)

An error is one fact, logged once

A failing command currently emits the `command_failed` record *and* a human `pve-storage-drs: <msg>` line — the same failure twice, in two formats, on the same stream. In `text` format the log record *is* the human line and the separate print is dropped; in `json` format the human line is dropped and the record carries it. Exit codes are unaffected either way.

The event catalogue

Event names are an interface — `journalctl ... | jq 'select(.event=="move_started")'` is a supported way to use this tool — so they are specified here rather than left to whoever writes the call. **Every log record carries an `event`** – enforced by a test that parses `src/` with `ast`, so a call site cannot silently add an unnameable member.

Event	Level	When	State
`run_started`	INFO	Once per invocation: command, effective mode, config path + sha256, version	built (replaced `config_loaded`)
`run_summary`	INFO	Once per invocation: groups visited, moves issued/succeeded/failed, bytes moved, wall time, exit code	built
`gate_decision`	INFO	Per group: `act`, the computed drift/imbalance fractions and the thresholds they were compared against	built — section 2.1 required it; nothing implemented it
`load_digest`	INFO	Per group: total load, per-storage `u_s`, disk count, how many disks were coverage-rejected	built — section 2.1 required it; nothing implemented it
`plan_selected`	INFO	Per acting group: backend, solver status, move count, the six objective terms, before/after spread	built — section 2.1 required it; nothing implemented it
`payback_verdict`	INFO	Per acting group: benefit, cost, ratio, the configured minimum, and the accept/reject outcome	built — section 2.1 required it; nothing implemented it
`move_started`	INFO	Immediately after `move_disk` returns, carrying **the UPID**, disk key, source, target, bytes	built — section 2.1 required it; nothing implemented it
`move_finished`	INFO	Per move: UPID, outcome (`moved/failed/replan_needed/draining`), duration	built
`replan`	INFO	Section 9.2's re-plan loop fired: which group, which attempt, why	built
`deadlock`	WARNING	Section 8's scheduler could not order a move set	built
`mode_override`	INFO / WARNING	`--mode` differs from the config; WARNING when it escalates (section 11.3)	as built, correct
`config_warning`	INFO	Configuration advisories (e.g. a `payback_horizon` under 30 days)	built: INFO
`storage_pattern_expanded`	DEBUG	A `/regex/` storage id matched a set	built: DEBUG
`optimize_backend_unavailable`	DEBUG under `auto`, WARNING when that backend was explicitly configured	An optional solver dependency is not importable	built: DEBUG under `auto`, WARNING otherwise
`solver_fallback`	WARNING	A configured backend produced no plan and the heuristic took over	as built, correct
`forecast_backtest_failed`	WARNING	`holt_winters` did not beat the quantile baseline for a group (or there is too little history to check); the group ran on `quantile`	built (phase 14b)
`forecast_history_unavailable`	WARNING	The `holt_winters` history query failed (`MetricsError`, or a `--replay` bundle captured over less than `2W`); the group ran on `quantile`	built (phase 14b)

Event	Level	When	State
`forecast_used`	INFO	`holt_winters` drove a group's loads: both backtest errors and how many disks were scaled or kept	built (phase 14b)
`forecast_fit_failed`, `forecast_fit_skipped`	DEBUG	One disk's fit failed, or `statsmodels` is missing	built (phase 14b)
`vm_locked`, `orphaned_volumes`, `orphan_check_failed`	WARNING	Section 9.4's hazards	as built, correct
`state_corrupt`, `state_read_failed`	WARNING	Section 11.2's state file is unusable	as built, correct
`inflight_found`, `inflight_reconciled`, `inflight_check_failed`, `cluster_task_scan_failed`	WARNING (INFO for `inflight_reconciled`)	Section 13's crash recovery	as built, correct
`apply_lock_held`	INFO	Another instance holds the lock; exit 0 quietly (section 11.2)	as built, correct
`salt_rotated`	WARNING	`--new-salt` discarded an existing mapping (section 16.3)	as built, correct
`command_failed`	ERROR	The run is exiting non-zero	built: printed once, per the format in use
`status_file_write_failed`	ERROR	`monitoring.status_file` could not be written (section 2.4); the run and its exit code are unaffected	built

Three level corrections deserve their reasons stated, since each one is a judgement that could otherwise be quietly reverted:

- **`optimize_backend_unavailable` under `auto` is not a warning.** `solver.backend: auto` *means* "use the best solver installed here"; probing for an optional solver and not finding it (when this was written, CP-SAT; today CBC on a plain `pip install` without the `solver` extra) is that option working, not degrading. It then logged at WARNING once per group per run, forever, on every cluster that did not install the optional dependency — and its message said `solver.backend=cpsat requested`, which was false: `auto` requested it, not the operator. `cli._solve_group()` already suppresses its *own* fallback warning under `auto`; the inner probe must learn the same distinction, and must say `not available` rather than `requested but ...` when nobody requested it.
- **`config_warning` is an advisory about a file, not an event in a run.** "`payback_horizon` is below 30 days" is equally true on every run until someone edits the config; repeating it at WARNING every 15 minutes trains operators to ignore warnings. It drops to INFO (so the audit trail still records what configuration was in force) and `verify-storages` — the command whose entire job is auditing storage configuration — reports it where an operator will act on it.
- **`storage_pattern_expanded` is a detail of topology building**, emitted once per group per topology build (twice per `collect-testdata` run). Which storages a pattern matched is already in every report that lists storages. DEBUG.

What this does *not* add

Section 2.1 previously promised "plus an optional file sink". No such sink, and no `logging` section in the configuration schema, was ever implemented. **The promise is retracted rather than built**: `StandardOutput=`/`StandardError=` in a unit file, a shell redirect, or `systemd-cat` already put this stream anywhere an operator wants it, and a second, in-process copy of that mechanism would be one more

thing to test, rotate and get wrong. An operator who wants a file gets it from the shell. *(Flagged explicitly because it is a removal from the specification, not an omission.)*

Verification

The tests that existed before phase 11 covered the formatter's mechanics and nothing about policy, which is why the policy drifted. These now exist, in `tests/unit/test_logging_setup.py`, `test_cli.py`, `test_execute.py` and `test_optimize.py`:

- A clean read-only run against a `--replay` bundle (section 16) emits **nothing on stderr** at the default level, and its report on stdout is byte-identical to the same run with `--quiet`.
- `apply --mode auto` emits `gate_decision`, `plan_selected`, `payback_verdict`, `move_started` (with a UPID), `move_finished` and `run_summary` **without** `-v`, and `apply --mode dry-run` does not.
- `--quiet` suppresses the mandatory `INFO` floor; `--log-level` overrides both `-v` and `--quiet` for the ladder itself, but a level below the floor on a `confirm`/`auto` run is raised back up to it rather than suppressing the audit trail (X-06 of REVIEW.md section 29).
- `--log-format text` on a non-TTY emits no JSON; `--log-format json` on a TTY emits only JSON.
- Third-party loggers are not raised by `-v` (only by `-vv`).
- `--json` on stdout parses as a single JSON document with every log level and format combination — the stream separation property, asserted rather than assumed.
- Every `logger.*()` call in `src/` passes an `event` in `extra=` (a repository-wide grep test, in the spirit of `tests/unit/test_documentation.py`), so the catalogue above cannot silently gain an unnamed member.

The timer

Section 2.1 assumes a systemd timer and `--quiet`'s help text names one, but `debian/` ships no unit. When it lands it must set `StandardError=journal`, rely on `auto` mode's mandatory `INFO` floor rather than passing `-v`, and must **not** pass `--quiet` — for the reason given above.

2.4 Monitoring status file

A timer-driven tool that fails quietly is worse than one that fails loudly, and the journal is not a monitoring system. `monitoring.status_file` (default `null`: write nothing) names a file that `apply` rewrites at the end of **every** run that produced a result, in the format of the `check_statusfile` plugin (`monitoring-plugins-contrib`, `/usr/lib/nagios/plugins/check_statusfile`) — so Nagios, Icinga, or anything that runs a Nagios plugin, can watch it with no code of ours on the monitoring side.

The format is the plugin's, read from its source rather than assumed:

- **Line 1** is exactly `OK`, `WARNING`, `CRITICAL` or `UNKNOWN` (case-sensitive; anything else is reported `UNKNOWN`).
- **Every later line** is the service output, printed verbatim; a file with nothing after line 1 is reported `UNKNOWN` ("Found no output"). Nagios takes perfdata from the first output line, so the summary (file line 2) ends with `| label=value ...`; the lines after it are detail.
- **Freshness is the file's modification time**, not its content: older than the plugin's `--age` (default 26 h) is reported `WARNING`. Hence *every* run rewrites it — including runs that moved nothing and dry runs — so that a stale file means "the timer stopped" and never "it was a quiet day". A run that could not read its configuration writes nothing and the file goes stale, which is the right signal too.

Levels. `CRITICAL` when the run exits non-zero: a failed move, or a PVE API or metrics error the run could not plan around (§9.2, "Errors are not mismatches"), or an unexpected crash (the handler re-raises, but only after the file says `CRITICAL` — a bug must not leave the previous run's `OK` in place). `WARNING` when the run exits `0` but left something for a human: it gave up after `execution.max_replans_per_run` re-plans (external churn — try again later); a failed move's orphaned volume was reported (never deleted, §9.4); a source storage had not released a volume after a move (`draining`, §9.3); or a group still breaches

its snapshot reserve or free-space requirement once this run's plan has run (the tool never trades the reserve for balance, so it will not fix that by itself). Otherwise `OK`. `UNKNOWN` is never written by the tool. The summary line carries the first problem; detail lines list the rest, each on one line (newlines in messages are flattened, or the summary could be pushed off line 2), each line cut at 300 characters (monitoring systems truncate plugin output, and a PVE or Prometheus error can carry a whole request URL — the full text is in the run log), and capped at 20 lines with an "and N more" line.

Mechanics. Written to a temporary file in the same directory and renamed into place, so the plugin never reads half a report; mode `0644`, because the monitoring user is not the user `apply` runs as; the parent directory is created if missing. The content is counts, group and storage names and volume ids — no credentials. Only `apply` writes it (`plan` and friends are interactive and would overwrite the timer's last result), not `--replay`, and not a run that exited quietly because another instance held the lock (it produced no result; the file keeps the last real one). A failure to write it is logged (`status_file_write_failed`, ERROR) and changes neither the run nor its exit code — and a file that stops updating turns `WARNING` by age on its own.

Verified against the real plugin, not a paraphrase of it: `tests/unit/test_statusfile.py` and the CLI tests run `check_statusfile` over what the tool writes whenever the plugin is installed (`OK` → exit 0, `WARNING` → 1, `CRITICAL` → 2, missing → 3, stale → 1) and over the two shapes the tool must never write (wrong-case level word, nothing after line 1).

---

3. Data acquisition

3.1 Where the per-disk data comes from

This is the single most important implementation fact, so it is stated precisely.

`pvestatd` calls `PVE::QemuServer::vmstatus(undef, 1)` — note the `full=1` flag. With `full` set, `vmstatus` issues QMP `query-blockstats` and stores the result as:

```
$res->{$vmid}->{blockstat}->{$drive_id} = $blockstat->{stats};
```

where `$drive_id` is the QMP device name with the `drive-` prefix stripped — i.e. **`scsi0`, `virtio0`, `ide2`**, matching the VM config keys exactly. The whole QMP stats hash is kept without filtering, which includes:

```
rd_bytes  wr_bytes  rd_operations  wr_operations
rd_total_time_ns  wr_total_time_ns
flush_operations  flush_total_time_ns
rd_merged  wr_merged  failed_rd_operations  failed_wr_operations
invalid_rd_operations  invalid_wr_operations  account_invalid  account_failed  idle_time_ns
```

`PVE/Status/InfluxDB.pm` then flattens the nested hash so that the **first** nesting level becomes the InfluxDB *measurement* (`blockstat`), the **second** becomes an *instance tag* (`instance=scsi0`), and the leaves become *fields*. Combined with the base tags from `update_qemu_status` (`object=qemu`, `vmid=…`, `nodename=…`, `host=<vm name>`), a series arrives in Prometheus roughly as:

```
blockstat_rd_operations{vmid="101", instance="scsi0", nodename="pve01", host="db-01"}
```

Everything needed for true per-disk IOPS, throughput **and latency** is therefore already present in an existing PVE → InfluxDB → Telegraf → Prometheus pipeline. **No new exporter or collector is required.**

Present at the source is not the same as present at the query endpoint. The InfluxDB protocol `InfluxDB.pm` emits carries string fields and fixes a field's type at its first write; Prometheus/OpenMetrics has no string sample type. A transport between the two therefore *can* drop one of the six counters, for one series, permanently and silently — Telegraf with a `prometheus_remote_write`/`prometheus_client`

output and no deliberate processor configuration being the common case. §3.3's step 2 exists because of this, and it is the reason every metric name here is configuration rather than a constant.

A backend that ingests **the InfluxDB protocol exactly as PVE exports it** and serves PromQL itself removes the bridge entirely — no Telegraf, no output plugin, nothing in the path that can drop a field on type grounds. **gigapipe on ClickHouse** is that backend here: it is what this project is dogfooded against and what both `tests/corpus/` bundles were captured from. gigapipe also ingests OpenTelemetry, but that path is untested by this project and is not a route around §3.2 — PVE's OTel metric server bakes the drive id into the metric name regardless of what consumes it. `docs/manual/05-metrics-pipeline.md` is the operator-facing version of all of this.

Verification status of the above. The `vmstatus(undef, 1)` call, the `blockstat->{$drive_id}` assignment, the `s/drive-//r` prefix stripping and the `InfluxDB.pm` nesting behaviour were read from the `pve-manager` and `qemu-server` sources, not inferred from documentation. Independently, the operator of the target cluster confirmed empirically that their existing Prometheus already carries `rd_operations`, `wr_operations`, `rd_bytes`, `wr_bytes`, `rd_total_time_ns` and `wr_total_time_ns` per disk. The *existence* of the data is therefore settled.

What remains genuinely unconfirmed is the **Telegraf-side naming** — the measurement/field join character, and whether the `instance` tag survived the collision described in §3.3. That varies per deployment and is exactly what `pve-storage-drs verify-metrics` exists to pin down. Treat §3.4's metric names as defaults to be confirmed, not as constants.

3.2 Two paths deliberately not taken

Do not use RRD. `GET /nodes/{node}/qemu/{vmid}/rrddata` exposes only VM-aggregate `diskread`/`diskwrite` in bytes/s — no per-disk breakdown and no operation counts. `GET /nodes/{node}/storage/{storage}/rrddata` exposes only `used` and `total`, with no I/O metrics at all. Prometheus supersedes RRD entirely here.

Do not switch to the OpenTelemetry metric server (new in PVE 9.1), despite it being the more modern-looking option. Its `_convert_node_metrics_recursive` handles a nested hash by appending the key to the metric **name**:

```
if (ref($value) eq 'HASH') {
    push @metrics, $class->_convert_node_metrics_recursive(
        $value, $ctime, "${metric_prefix}_${key}", $attributes,   # <-- name, not attribute
    );
}
```

so `blockstat.scsi0.rd_operations` becomes the metric **`proxmox_vm_blockstat_scsi0_rd_operations_total`**. The device is baked into the metric name rather than carried as a label, which makes PromQL aggregation across devices impossible without `label_replace` gymnastics and produces unbounded metric-name cardinality. The InfluxDB path puts the device in a **label** and is strictly superior for this use case. (The plugin also only supports OTLP/JSON encoding, not protobuf.)

3.3 Metric name mapping and verification

Telegraf naming is deployment-specific, so **every metric and label name is configuration**, not a constant. Implement `pve-storage-drs verify-metrics` as a first-class command that must be run before anything else. It shall:

1. query `/api/v1/label/__name__/values` and confirm each configured metric name exists;
2. run one instant query per metric and print a sample series with all labels. Reusing these same six results at no extra Prometheus cost, cross-check the full `(vmid, device)` set each metric reports against the other five's, and warn naming the metric and the disk(s) whenever one metric's set is a

strict subset of another's — the one common, real-world way this happens: InfluxDB's line protocol fixes a field's type from its first write, and Telegraf's Prometheus-compatible output silently drops a field the moment it sees a non-numeric value for it (Prometheus/OpenMetrics has no string sample type), independently of the other five fields for the same disk. Step 5 below (checking only `read_ops`) cannot catch this alone if a *different* field is the one silently dropped;

3. confirm the configured `vmid`, `device` and `node` labels are present and non-empty;
4. warn loudly if the device label is literally `instance` — **PVE's `instance` tag collides with Prometheus's own scrape-target `instance` label**, and many Telegraf configurations rename or overwrite it. The implementer must confirm the real label name here before proceeding;
5. report per-disk sample coverage over the configured window, so gaps are visible up front;
6. measure the **observed sample spacing** (the median over series of a short window divided by that window's `count_over_time()` – never a `query_range`, whose points are one per *step*, so it would only ever echo the step it was asked for) and compare it against `metrics.pvestatd_push_interval`. That interval is a PVE-side setting the tool cannot read from the API, so it is declared in config; this step is what stops a stale declaration from silently invalidating the `rate_window ≥ 4 × interval` rule of §11.1. Error if the two disagree by more than 20%, and error if `rate_window` is below four times the *observed* spacing regardless of what config claims.

3.4 PromQL

Let `R` be `metrics.rate_window` (default `5m`) and `W` be `window.lookback` (default `24h`). Three raw quantities per disk, all keyed by `(vmid, device)`:

Read and write are fetched **separately** so that `read_factor`/`write_factor` (§4) can be applied engine-side — six queries per group, not three:

```
# in-flight I/O, read and write (divide by 1e9 to get s/s)
sum by (vmid, device) (rate(blockstat_rd_total_time_ns[5m])) / 1e9
sum by (vmid, device) (rate(blockstat_wr_total_time_ns[5m])) / 1e9

# operations per second
sum by (vmid, device) (rate(blockstat_rd_operations[5m]))
sum by (vmid, device) (rate(blockstat_wr_operations[5m]))

# bytes per second
sum by (vmid, device) (rate(blockstat_rd_bytes[5m]))
sum by (vmid, device) (rate(blockstat_wr_bytes[5m]))
```

Reduce each to one scalar over the decision window with a robust quantile rather than a mean, so a single spike neither triggers nor suppresses a migration:

```
quantile_over_time(0.95, ( <expression above> )[24h:5m])
```

Notes for the implementer:

- `rate()` already handles the counter resets that occur when a VM reboots or migrates between nodes.
- The `sum by` collapses the `nodename`/`host` labels, which is what we want: a VM that live-migrated between nodes during the window must remain **one** disk, not two.
- Use the **range** (`query_range`) form as well when the forecaster needs a series rather than a scalar; the same expression works with a `step`.
- Reject a disk whose sample coverage over `W` is below `window.min_coverage` and fall back to its last known load from `state.json`, flagging it in the plan output. Never treat missing data as zero load — that would silently invite migrations *onto* a busy storage.

- **Scope every query to this cluster's own nodes.** None of the expressions above restrict which series they match beyond the metric name itself, which is fine when one Prometheus serves exactly one PVE cluster but silently wrong the moment it serves more than one (or anything else emitting a same-named metric): `vmid` is only unique *within* a cluster, so an unscoped query would sum a same-numbered vmid from somewhere else into this one's load without any error or warning. Add a matcher inside the vector selector, before `rate()`, in this order:

  1. `metrics.extra_selector`, verbatim, when the operator set one — needed when Telegraf's tagging doesn't carry PVE's own node name verbatim, the operator wants to scope by a cluster-naming tag their own deployment happens to carry, or the restriction needed is something else entirely.
  2. `<metrics.labels.node>=~"pve01|pve02|..."` otherwise, built from the cluster's own node list (`GET /nodes`, §3.5) every run rather than hand-maintained, so a node added to the cluster is covered with no config edit. This is the default tier.

  A run-time symptom of a scoping mismatch (whichever tier produced it) — a resolved selector that matches literally nothing across all six raw queries — is named explicitly, as "the resolved query filter matched no series at all", rather than being reported as an idle group (`loadmodel.compute_group_load`'s `no_series_matched`, REVIEW.md W-07).

  `pve-storage-drs verify-metrics` applies only tier 1, never tier 2: it is deliberately independent of the PVE API, so it has no node list to build tier 2 from.

  An earlier revision of this section had a middle tier here, matching a `cluster`-naming label (`metrics.labels.cluster`, defaulting to the literal `"cluster"`) against this cluster's own name (`GET /cluster/status`, needing `Sys.Audit` at `/`). It was removed: the premise that this project's deployments carry such a label "as standard practice" was an operator's own mistaken assumption, not a real convention or a Prometheus finding — there never was such a tag, and the tier existed only on that premise. Use `metrics.extra_selector` for a cluster-naming (or any other) label a deployment actually has.

As built (0.1.3): a gigapipe `step >= range` workaround. Two live-confirmed deployments of a recently updated gigapipe backend silently return **zero series** for any `query_range`/subquery call on a range-vector function (`rate()` included) whenever the query's own `step` is `>=` the function's own range-vector duration — binary-searched to the exact second, independent of window length or absolute step size. This project's own defaults set `metrics.step == metrics.rate_window` (both `5m`) — idiomatic back-to-back tiling, and exactly the failing boundary — so a fresh install with untouched defaults can hit this on an affected backend with no misconfiguration at all. `metrics.py`'s `safe_range_step_seconds(step, rate_window)` computes the largest whole-second step strictly below `rate_window` that divides `step` as evenly as a whole-second value can (150s at the 5m/5m default), and every `query_range`/subquery call this module issues — coverage, the forecaster's raw range series, the quantile-over-time reduction's subquery — uses that step instead, unconditionally (not only when the affected backend is detected: the symptom, zero series, is indistinguishable from genuinely absent data). `decimate_to_configured_step()` then recovers the originally configured grid by keeping every Nth point, so a range series' retained points are byte-identical to what a plain `metrics.step` query would have returned on an unaffected backend — the workaround is a true no-op there, except for `quantile_over_time`'s own inner evaluation, which cannot decimate (an instant query returns one scalar): it evaluates its subquery on the denser, safe-step grid on *every* backend, a small but real shift in the reduced statistic (REVIEW.md Z-04). `docs/internals/30-metrics.md` has the full mechanism; `collect-testdata --estimate` and `support.max_series_points` account for the doubled (or more, at a wider `metrics.step`/`rate_window` ratio) point count this stores (REVIEW.md Z-05).

3.5 PVE API

`pve.py` is built on `proxmoxer`, not a hand-rolled ticket/CSRF client. `proxmoxer` implements the ticket exchange and API-token auth below itself, behind a `ProxmoxAPI` object whose attribute chaining (`proxmox.nodes(node).qemu(vmid).config.get()`) maps directly onto the endpoint table below. The decisive reason is its **backend abstraction**: the same `ProxmoxAPI` interface is available over plain HTTPS (`backend="https"`, the default and the only one this project uses today), or over SSH — either `openssh` (shells out to the system's own `ssh` + `pvesh`) or `ssh_paramiko` (an in-process SSH client). A deployment that cannot or will not open tcp/8006 to the management host can switch to an SSH-based backend as a **configuration change in one factory function** (`pve.build_client`), with no change to `PveClient`'s methods or to anything that calls them — exactly the shape a hand-rolled HTTPS-only client would not have offered. `python3-proxmoxer` is packaged for Debian trixie (§2.1).

Authentication, as `proxmoxer`'s `https` backend performs it (username/password as originally specified, API token preferred and what this project actually configures by default):

```
POST /api2/json/access/ticket        {username, password}
  → data.ticket                      → Cookie: PVEAuthCookie=<ticket>
  → data.CSRFPreventionToken         → header CSRFPreventionToken on every write
```

Tickets are valid ~2h; `proxmoxer` refreshes them itself on its own internal interval when using password auth. API tokens (`Authorization: PVEAPIToken=USER@REALM!TOKENID=SECRET`) need no CSRF header, no refresh, and are preferred for unattended `auto` mode; `proxmox.auth.token_id`'s `user@realm!tokenname` format is split into `proxmoxer`'s separate `user`/`token_name` arguments at the one place `pve.py` constructs the client.

Read path:

Endpoint	Used for
`GET /cluster/resources?type=vm`	VM inventory: vmid, node, status, name, tags
`GET /cluster/resources?type=storage`	Storage inventory, `shared` flag, used/total per node
`GET /storage`	Storage definitions: type, `content`, `shared`, `nodes` restriction, **and** per-storage `saferemove` / `saferemove_throughput` — see §7.1 and §9.3
`GET /nodes`	Every node in the cluster, by name — §3.4's PromQL node-scoping filter, independent of which nodes currently host a VM or shared storage
`GET /nodes/{node}/qemu/{vmid}/config`	**disk → storage mapping and size**
`GET /nodes/{node}/storage/{storage}/status`	authoritative `total`/`used`/`avail`
`GET /nodes/{node}/storage/{storage}/content`	per-volume real allocated sizes, owner vmid
`GET /nodes/{node}/qemu/{vmid}/snapshot`	detect existing snapshot/volume chains (§3.7)
`GET /nodes/{node}/qemu/{vmid}/pending`	detect an unapplied pending config change per disk (§3.8)
`GET /nodes/{node}/qemu/{vmid}/status/current`	`lock` state immediately before a move (§9.3)

Use `GET /storage` (the list form), never `GET /storage/{storage}`, for a storage's own config including `saferemove`. Verified empirically against a live PVE 9.2.11 cluster: an API token granted only `Datastore.Audit` can list every storage's full config via `GET /storage`, but the same token gets `403 Forbidden (Datastore.Allocate)` calling `GET /storage/{storage}` for the exact same storage — the single-item form apparently backs an edit-UI code path gated by the ability to change the config, not merely read it. The list form returns the identical per-storage object for every storage in one call, so nothing is lost by preferring it; it is also one call instead of `|storages|` calls.

`GET /nodes/{node}/storage/{storage}/content` itself needs `Datastore.Allocate`, not merely `Datastore.Audit`, contrary to what an "Audit-only, read-planning tool" design would hope. Also

verified empirically on the same live cluster: with only `Datastore.Audit` granted, the call succeeds (HTTP 200) but returns an empty list on every storage, node and content-type filter tried, with no error at any level — a false negative that looks exactly like "this storage has no content" unless you already know to be suspicious of it. Granting `Datastore.Allocate` (bundled with `Datastore.Audit` in a custom role, scoped per-storage — see below) on the same storages made `/content` immediately start returning real data, retested and confirmed. So the tool's true minimum privilege per storage in scope is `Datastore.Audit` **and** `Datastore.Allocate`, not Audit alone; `move_disk` needs nothing beyond that on the storage side (the VM side needs `VM.Config.Disk` and `VM.Migrate` or equivalent, out of scope for this note). Document this precisely in the operator manual rather than repeating the more comfortable but wrong "Audit is enough" claim — an operator who grants only Audit will see the tool silently treat every storage as empty of tracked volumes, which corrupts the U^{sext} accounting of §5.1.1 without ever raising an error.

Two more things this exposed about Proxmox's ACL model, worth knowing before configuring a token: first, **API tokens have their own ACL entries, separate from their owning user** — with privilege separation enabled (the default), a token's effective permission is the *intersection* of the user's permissions and whatever is granted to the token specifically, so granting a role to the user alone has no effect on the token, and granting it to the token alone has no effect either unless the user has it too. Both need the grant. Second, ACL entries at `/storage` (the parent path, with `propagate: 1`) were not sufficient in practice to make `Datastore.Allocate` retroactively work on already-listed sub-paths in one observed sequence during testing — granting the role explicitly at each `/storage/{id}` resolved it; whether the parent-path grant would have converged given more time was not conclusively isolated, so the operator-facing guidance below grants per-storage explicitly rather than relying on propagation from `/storage`.

Parsing a disk from the VM config: a key matching

```
^(?:ide[0-3]|sata[0-5]|scsi(?:[0-9]|[12][0-9]|30)|virtio(?:[0-9]|1[0-5])|efidisk0|tpmstate0|unused\d+)$
```

whose value looks like `san-a:vm-101-disk-0,size=512G,iothread=1`. The storage id is the part before the first `:`; the size comes from the `size=` parameter, cross-checked against `/storage/{storage}/content`, which is authoritative for what is actually allocated. **Every bus counts** — a VM's boot disk on `ide0` or `sata0` occupies and loads a storage exactly as a `scsi0` does, and a tool that enumerates only `scsi*` will silently mis-account capacity and produce plans that cannot reach balance. §3.6 covers which of these can actually be moved.

Skip only entries whose value contains `media=cdrom`. That covers ISO mounts, empty drives (`none,media=cdrom`) and cloud-init drives (`san-a:vm-101-cloudinit,media=cdrom`). A cloud-init volume does occupy real bytes on the storage, so it is not in `D` but it **is** counted in U^{sext} (§5.1.1) like any other volume DRS does not manage.

Disk format. Detect the source format from the volume returned by `/storage/{storage}/content` (`format: raw|qcow2|…`), falling back to the storage type's default (raw for LVM, qcow2 for directory storages, raw for ZFS zvols). (C2) fixes $x_{d,s} = 0$ when s cannot store that format. `move_disk` without an explicit `format` preserves the source format, which is what we want by default: **format conversion stays disabled unless the operator opts in.** The opt-in is per target storage, `groups[].storages[].enforce_format` (§5.3.2): only then is `format=` passed, and only for a move whose source format differs from it — converting raw→qcow2 on shared LVM is what enables volume-chain snapshots, but it is a deliberate storage-policy change, not something a balancer should do silently.

`pve-storage-drs verify-storages`. A companion to `verify-metrics` (§3.3), run once per storage before relying on any plan. For every storage in every group it reports `type`, `shared`, `content`, `saferemove`, `saferemove_throughput`, total/used, the largest disk currently on it, the largest per-VM footprint currently on it with the VM it belongs to (Z_s, §5.3.3 — the quantity the snapshot reserve is computed

from, which is not the largest disk once a VM has two disks there), and the **resolved** free-space requirement `soft_s/hard_s` with the level each came from (§5.3.1), and the resolved `enforce_format` with the entry it came from plus the number and bytes of disks currently on the storage in another format (§5.3.2 — non-conforming, informational, never a violation); then it derives the implied wipe time `z_max / |saferemove_throughput|` (§7.1 on that sign) and warns when that exceeds `migration.max_single_move_duration` or `gates.cooldown_per_storage` (§9.3). It also prints the expansion of every `/…/` storage pattern (§11.4) — the entry and the storages it matched — and lists cluster storages matched by no group, so an over-broad or dead pattern is visible before any plan relies on it. The resolved requirement is there for the same reason the pattern expansion is: a percentage resolves against *each* LUN's own capacity, a pattern entry's `free_space` lands on every storage it matched — two ways for one line of config to mean a different number per storage, and this is the one command where that derivation is visible before a plan depends on it. Since `soft_s` now drives §6's override, §7.3's exemption and §9.5's shortfall lines, an operator who cannot predict it cannot predict the balancer. This is the command that turns "why has this balancer been stuck for two days" into a line of output before the first move.

Read-path cost and concurrency. The topology read is `O(number of VMs)`: PVE has no batch config endpoint, so `/qemu/{vmid}/config` must be fetched per VM. For a few hundred VMs, serial fetching dominates run time. Specify:

- a bounded thread pool (8–16 workers, configurable) for the per-VM config fetches, with a short per-request timeout and bounded retries on 5xx;
- a **per-run topology cache** — one snapshot at the start of the run, reused by the load model, solver and scheduler;
- the pre-move re-validation in §9.2 **must bypass this cache** for the specific VM and target storage it is about to touch, and only for those; everything else may be reused;
- `/cluster/resources` is a single call and gives the VM inventory, node placement and coarse storage usage, so fetch it first and use it to decide which per-VM configs are needed at all — in practice this means only a **stopped** VM can be skipped without fetching its config, since that is the only exclusion `cluster/resources`'s own fields (`status`) can decide. A VM excluded by `exclude.vmids`/tags still needs its config fetched: (C2) *pins* such a disk into `D` rather than dropping it (§5.1.1's note on why), which needs its size. A disk turning out to be "ungrouped" (§3.6) is only discoverable *after* fetching the config that reveals which storage it is actually on, so it costs a fetch too — the saving from this optimization is real but smaller than a naive reading suggests.

Expected call count per run: `4 + 3·|VMs considered| + 2·|storages| + |extra content pairs|` — four cluster-wide calls (VM inventory, storage inventory, storage definitions, and `GET /nodes` for §3.4's default node-scoping filter, skipped only when `metrics.extra_selector` is set), three per considered VM (config, which also carries `lock` per §9.3's pseudocode so no separate `/status/current` call is needed at planning time; `/snapshot`, per §3.7; and `/pending`, per §3.8), and two per storage (`status`, `content`). `|extra content pairs|` is the amplification the managed-disk own-VM-node size fetch adds (below, "The actual mechanism…"): one additional `content` call per distinct `(node, storage)` pair a managed disk's VM runs on, beyond the per-storage active-node pick already counted above — deduplicated by `_needed_content_node_pairs()`, so it is bounded by distinct nodes hosting managed disks, not by VM count, but is otherwise unbounded above (worst case `|nodes| · |storages|`) and grows with how spread out VMs are across nodes, not with cluster size alone.

Write path:

```
POST /api2/json/nodes/{node}/qemu/{vmid}/move_disk
     disk=scsi0  storage=san-c  delete=1  bwlimit=<KiB/s>  [format=qcow2]
  → UPID string
GET  /api2/json/nodes/{node}/tasks/{upid}/status   → status=running|stopped, exitstatus=OK|…
```

`bwlimit` is in **KiB/s**. `delete=1` removes the source volume after a successful mirror; without it the old volume is left behind as an unreferenced volume and the reserve math will silently drift.

3.6 Which disks can actually be moved online

Enumerating every bus (§3.5) is necessary but not sufficient: the set of disks that *exist* is larger than the set that can be relocated while the VM is running, and the difference is load-bearing for both the capacity model and the affinity objective.

Config key	In `D` (movable)?	Why
`ide0-3`, `sata0-5`, `scsi0-30`, `virtio0-15`	**yes**	ordinary QEMU block devices; `move_disk` mirrors them online
`efidisk0`	**yes**	movable online — verified on PVE 9.2. Tiny (a few MiB), so its migration cost is effectively zero
`tpmstate0`	**yes**	movable online — verified on PVE 9.2. Tiny. PVE may not use the `drive-mirror` path for it, since `swtpm` rather than QEMU owns the state; see the note below
`unused0-N`	**yes**, with $\ell_d = 0$	a real allocated volume detached from the VM. It occupies bytes and counts against the reserve, but generates no I/O
anything with `media=cdrom`	no — not in `D`, counted in U^{sext}	ISO mounts, empty drives, cloud-init volumes

`efidisk0` and `tpmstate0` move online. This was confirmed empirically on a live PVE 9.2 cluster by the operator. Older Proxmox forum threads (PVE 6.x/7.x era) report online moves of these two failing, and that history is worth knowing only so nobody re-derives an obsolete restriction from a search result: it does not apply to 9.2. They are ordinary members of `D`.

Two practical notes. First, they are **small** — an EFI var store is a few MiB, TPM state likewise — and below `migration.tiny_disk_bytes` (§5.4) that smallness is priced in: their β and γ charges are zero and §7's payback test does not apply to them, so κ may drag a 528 KiB `efidisk0` across the group for free. An earlier revision of this section charged such a move a full β on the grounds that it *is* a task, with task overhead and a lock window, and called the §7 cost "negligible" rather than zero; dogfooding overturned that judgement (found live, not by review): a four-move plan whose entire value was reuniting VMs — two of the moves 528 KiB EFI disks — was rejected wholesale by a payback arithmetic in which the affinity those moves bought was structurally worth zero (§7.2), so the tool refused the one action the operator most wanted. A tiny move occupies its VM's lock for seconds and transfers a rounding error of the mirror budget; pricing it as a migration priced affinity repair out of the tool. Free is not the same as unconstrained, though: a small disk moves only *with* its VM — to a storage where a larger disk of the same VM in the same group ends the plan — and never on its own ((C8), §5.3). Second, PVE may not use the `drive-mirror` path for `tpmstate0`, because `swtpm` rather than QEMU holds that state. Do not depend on drive-mirror semantics for it. The transient invariant of §8.1 — the volume occupies **both** storages until the move completes — is the conservative assumption and stays correct under either mechanism, so no part of the model needs to know which one PVE picked. Be clear about which way that conservatism cuts: if `swtpm` does *not* use drive-mirror, the both-storages assumption over-reserves during the move. Over-reserving can never cause a reserve breach, so it is safe; the only cost is that a `tpmstate0` move onto a nearly-full storage may fail the transient check when it would physically have fitted. Treat that exactly like any other infeasible move — **defer it, never force it** — and let `pve-storage-drs explain` say that the transient check was the blocker. Given that TPM state is a few megabytes, a storage tight enough for this to bind is a storage with a much larger problem.

Metrics coverage differs by device type. `efidisk0` is a QEMU drive and should appear in `blockstat`; `tpmstate0` and `unused{N}` are not QEMU block devices, so no series will exist for them and $\ell_d = 0$ under the `min_coverage` rule of §3.4. That is correct rather than a gap — they generate no guest I/O worth balancing. Have `pve-storage-drs verify-metrics` report which config keys resolved to a series and which did not, and classify these as **expected-absent** rather than as errors, so a genuine coverage problem on a `scsi0` still stands out. Treat the exact per-device-type coverage as a thing to observe on your cluster, not to assume from this table.

Pinned disks are modelled, not ignored. Disks pinned by (C2) — snapshot-blocked (§3.7), config-excluded, or locked this run — enter the MILP as ordinary disks with $x_{d,\sigma_0(d)} = 1$ fixed. This is deliberately *not* the same as excluding them: their bytes must still count toward $\Sigma_d\ z_d \cdot x_\{d,s\}$ and toward Z_s in the reserve constraint (C5), and their load toward L_s. Treating them as foreign volumes instead would work for capacity but would lose the fact that they belong to a VM whose other disks we are placing.

Pinned disks count toward the affinity term by default. κ (§5.4) counts a VM's spread over storages, and a disk that cannot move this run still *occupies* a storage — the VM is genuinely spread whether or not that particular volume is reachable. So the $y_\{v,s\}$ linking of (C3) ranges over all of D; set `objective.affinity_counts_pinned_disks: false` to range over D^{mov} instead.

An earlier revision defaulted the other way, reasoning that an immovable disk would otherwise leave a VM permanently "fragmented" and let κ "veto good placements to chase a co-location that cannot be achieved this run". Dogfooding overturned that (found by replaying a real bundle, not by review), and it is worth being precise about why, because the reasoning is seductive and wrong in two separate places.

The co-location usually can be achieved. A pinned disk's storage is a **constant**, so κ's only marginal effect is a preference for putting the VM's movable disks where it already has one. That is reachable this run — by moving the movable disk, which is the very thing the solver is choosing. Excluding pinned disks does not make an unreachable goal reachable; it hides a reachable one. On the bundle that found this, VM 717219 had two pinned disks together on one storage and a single movable `efidisk0` on another; the plan sent the `efidisk0` to a *third* storage, because with the pinned pair excluded the VM's counted footprint was one disk and every target scored identically.

Worse, the exclusion can invert the sign. Take a VM with a pinned disk on `a`.

- **One movable disk, on `b`.** Under D^{mov} the debt is 0 wherever that disk goes — one counted disk, one storage — so moving it to `a` to reunite the VM scores exactly the same as leaving it or sending it to a third storage: repair is *unrewarded*. Under all of D the debt is 1 until it joins `a`, then 0.
- **Two movable disks, both on `b`.** Under D^{mov} the debt is 0; moving one of them to `a` to rejoin the pinned disk raises it to 1. The tool *charges* κ for taking a step towards reassembling the VM. Under all of D that step is neutral ($1 \to 1$), and moving both is a gain ($1 \to 0$).

The original worry does have a real residue: κ now charges a VM for a split it cannot fully undo, so a VM with pinned disks on two different storages carries a permanent debt floor. That floor is a *constant* — it shifts the objective's absolute value but not its argmin, and it cancels in §7.2's $A_before - A_after$ — so it changes no decision. And where κ does pull a movable disk toward a bad target, it remains what §5.4 calls it: a soft preference that (C5)'s free space or a strong imbalance can legitimately override.

Unused disks move only to repair the reserve, and that is correct. They carry $\ell_d = 0$ — no series exists for a volume QEMU has not opened — so relocating one yields zero imbalance benefit while incurring the full $\gamma \cdot z_d$ byte penalty and the full payback cost of §7. The solver will therefore leave them alone until a capacity constraint forces the issue, which is exactly the desired policy. Set `exclude.include_unused_disks: false` to pin them instead; they then count via U^{ext}.

An `unusedN` volume can be absent from `GET /storage/{s}/content` — not PVE's normal behaviour, but not rare enough to ignore either. Found on a live PVE 9.2.11 cluster on Ceph RBD storage: a VM's `unused0` entry named a volume (`VM:vm-104-disk-2`) that PVE's own config still tracked, but that volume did not appear anywhere in that storage's content listing, while the same VM's two active (`scsiN`) disks did. The operator confirmed the cause: the underlying volume had been removed directly on the storage backend, outside Proxmox, leaving `unused0` a dangling reference in the VM's config. This is not something PVE catches on its own — an `unusedN` entry is not validated at VM start the way an attached disk is, so the VM boots normally with the stale reference still sitting in its config. `topology.py` treats a content-listing gap the same regardless of cause (§3.5): it falls back to the VM config's own `size=` for that disk and logs a warning naming it as unauthoritative, which is the conservative direction to be wrong in here — believing a since-deleted volume still occupies its former space costs nothing but a temporarily pessimistic reserve calculation, while the reverse (silently dropping it) could let a real volume's bytes go uncounted. An operator seeing this warning should treat it as a prompt to check for, and if confirmed gone, clean up the dangling reference (e.g. `qm unlink <vmid> unusedN`) rather than something the tool should silently paper over.

A content item can also be present but missing its own `size` — a second, narrower variant of the same gap. Found on a live cluster (a `verify-storages` crash, `KeyError: 'size'`, not a warning): unlike the `unusedN` case above, the volid *is* in `GET /storage/{s}/content`, just without a `size` key on that entry at all. `_resolve_disk_size_and_format()` originally assumed any matched entry had one; it now falls back the same way an absent entry does (VM config `size=`, a warning naming which of the two gaps applies), still trusting the entry's own `format` since that field is independent of `size`. The equivalent sum over *foreign* (unreferenced) volumes in `_build_storages()` had the identical unguarded `item["size"]` and no VM config to fall back to for one — that one skips the volume from the reserve calculation instead (an undercount, the opposite conservative direction from the case above, for lack of any better number) and warns just as loudly.

`approximate-size` sits between those two, and is preferred over the VM config. A `size`-less content item can still carry `approximate-size` — PVE's own field for storage plugins where an exact size is expensive to determine — and both call sites above check for it before falling further back: still a live number from the storage plugin itself, not a static one recorded at disk-attach time and never revisited, so it is the better of the two imperfect answers. Only when *neither* `size` nor `approximate-size` is present does either function fall all the way back to the VM config (managed disk) or skip the volume (foreign one).

The actual mechanism, per the operator who hit this live: `approximate-size` is a PVE 9.2+ feature for qcow2 volumes on shared LVM storage specifically. PVE 9.2 added snapshot support on ordinary (non-thin) LVM by formatting the LV itself as a qcow2 image rather than using it raw. Reading a qcow2 image's own logical size means opening it, which needs its LV *active* — cheap on the node already running the owning VM (PVE keeps that LV active there), expensive or outright impossible on a shared LVM storage's other nodes, which may have no reason to have activated it at all and can collide with whichever node already has. `approximate-size` is PVE's way of answering the content listing anyway, from LVM's own metadata, without activating anything. This means `_pick_active_node()`'s one pick per *storage* (used for every other section 3.5 call) is the wrong node for *this* value specifically: `size` on a `GET /storage/{s}/content` response is only reliable from the one node guaranteed to have the LV active, which is the node the volume's own VM is running on, not an arbitrary node that merely reports the storage as available. `_needed_content_node_pairs()`/`build_topology()` now fetch a managed disk's content listing from *its own VM's node* specifically, not `_pick_active_node()`'s pick, precisely so `size` (not `approximate-size`) is what gets used whenever it can be. The tier above still matters: a foreign (unreferenced) volume has no owning VM to pick a node from, and a managed disk's own node can itself

report `approximate-size` if PVE has not (yet) activated the LV there either.

Evacuating a storage completely is therefore possible online — every disk type in the table above except CD-ROM-media entries can be relocated with the guest running, and CD-ROM entries hold no storage-owned data except cloud-init volumes, which are regenerable. Where a full evacuation is *not* achievable it is because of a §3.7 snapshot or an explicit exclusion, never because of a device type. `pve-storage-drs explain` must name the actual blocker per VM — *"106: cannot fully consolidate, 2 snapshots on scsi0"* — rather than emitting a plan that quietly leaves a stray volume behind.

3.7 Disks with snapshots are excluded, loudly

`move_disk` is issued with `delete=1` throughout (§3.5), and PVE refuses that combination on a volume that has snapshots — *"you can't move a disk with snapshots and delete the source"*. Even where a move is accepted, PVE does not carry the snapshots across: they are left behind or lost. Under PVE 9 volume-chain snapshots the situation is worse, because a snapshot is a *separate full-size volume* on the source storage, so a "moved" disk would leave most of its bytes behind and the reserve arithmetic would silently drift.

There is no safe automatic remedy. The two real options both belong to the operator: delete the snapshots (a data-retention decision the balancer must not make), or move the disk offline with the VM shut down (out of scope — this tool never stops a VM, §1). So the policy is **skip, and complain**:

1. **Detect per volume, not per VM.** `GET /nodes/{node}/qemu/{vmid}/snapshot` is authoritative for VM-level snapshots and pins *every* disk of that VM. Independently, cross-check `/storage/{s}/content` for volumes owned by the VM that its current config does not reference: under volume-chain storages those are chain members, and elsewhere they are orphans. Either way the disk is unsafe to move. Do not rely on volume-name patterns — they are storage-specific.
2. **Pin, do not drop.** An excluded disk stays in the model with $x_{d,\sigma_0(d)} = 1$ so its bytes and load remain accounted for in (C4)/(C5)/(C6); it is not demoted to a foreign volume.
3. **Complain, every run, at WARN.** List each affected VM with its pinned bytes and pinned load, and the group total of both. A one-line "3 VMs skipped" is not enough: the operator needs to know *which* snapshots to clear to unblock balancing. For an unreferenced volume the reason names it (`storage:volume`, the first two and a count of the rest): it appears nowhere else the operator looks — not in the VM config, not among the plan's disks — and it is usually the source copy a move without "delete source" left behind. The tool never deletes it (§9.4); the operator checks it and removes it by hand.
4. **Say when the goal has become unreachable.** If pinned load exceeds `report.warn_pinned_load_fraction` of a group's total (default 0.25), the residual imbalance may be structural rather than a planning failure. Report the best achievable spread *given the pins* alongside the actual one, so a stubborn 40% spread is attributable to snapshots rather than looking like a broken solver.

`exclude.skip_vms_with_snapshots` stays as a knob but its `false` setting does not make such moves work — it merely stops pre-filtering them, and PVE will reject them at the API. Keep it `true`; the pre-flight check of §9.3 runs regardless.

3.8 Disks with an unapplied pending change are excluded

Confirmed live against a real cluster (`config/drs.yaml`'s dev cluster, VM 102): `GET /nodes/{node}/qemu/{vmid}/config` — what §3.5's `vm_config()` calls, with no `current` parameter, which is every call this project ever makes — returns a key's **pending** value once one exists, not the value actually in effect. A disk edited through the PVE UI while the guest is running (a resize, an option change) shows up there as if the edit had already taken effect, even though PVE has queued it for the VM's next reboot. `GET /nodes/{node}/qemu/{vmid}/pending` (`vm_pending()`) is the only endpoint that exposes both: each entry carries `key`, the still-in-effect `value`, and — only when that key has an unapplied change — a `pending` field (an edit) or a truthy `delete` field (queued for removal).

Why this matters for a storage migration, specifically. `move_disk` acts on the *current* volume, not on the pending edit — moving `scsi0` when its pending change is, say, a queued resize to a larger size relocates the disk at its *current* size, then leaves that same pending resize entry sitting in the config afterwards, now describing a change relative to a volume that has since moved storage. PVE does not reconcile or drop a key's `pending` entry as a side effect of `move_disk` — there is no code path that would, since the two are unrelated operations from PVE's own point of view. The operator's next reboot then applies a resize computed against a value that predates the migration, on whichever storage the disk ended up on. This is not a hypothetical: it is what "the pending disk entry is not being updated when the disk is migrated" (an operator report against the dev cluster) actually is once traced to the API level, and PVE has no fix for it because nothing in its own model treats `move_disk` and a pending edit as interacting at all. The engine's only safe option is to keep the two from ever overlapping in the first place.

So, exactly like §3.7's snapshot handling:

1. **Detect per disk device**, from `vm_pending()`, not from `vm_config()`'s own merged view — the pending value is exactly what `vm_config()` cannot be trusted to distinguish from the real one.
2. **Pin, do not drop.** A disk with an unapplied pending change stays in `D` with $x_{d,\sigma_0(d)} = 1$ fixed (C2), the same as a snapshot-blocked or locked disk (§3.6's "pinned disks are modelled, not ignored").
3. **Re-check immediately before issuing the move** (§9.2's step 6, below), not only at planning time: an operator can queue a pending change in the PVE UI in the gap between `plan` and `apply`/`auto` issuing the move, exactly as a lock (§9.3) or a new snapshot (§3.7) can.
4. **No knob disables this.** Unlike `exclude.skip_vms_with_snapshots`, there is no configuration escape hatch — a pending change is not a policy preference to override, it is PVE leaving a config key referring to state that would become wrong the moment the disk moves.

(C2)'s eligibility list and §9.2's re-check list both name this condition explicitly.

---

4. The load model

For each disk `d`, reduce the three raw quantities to a single scalar ℓ_d, expressed in **average in-flight I/O requests**. The three terms have wildly different magnitudes, so they are normalized against each other before weighting and the blend is then rescaled back onto the in-flight-I/O scale; the units matter downstream and are spelled out below.

Read and write are combined **before** normalization, using the configurable asymmetry factors ρ = `load_weights.read_factor` and ω = `load_weights.write_factor` (both 1.0 by default). Reads and writes are therefore fetched as **separate series** and combined engine-side in `loadmodel.py`, not summed inside PromQL — this keeps the factors in tested code rather than in deployed query strings, and makes them unit-testable in isolation:

```
raw_t(d) = ρ·rd_time_d + ω·wr_time_d        (in-flight I/O, s/s)
raw_o(d) = ρ·rd_ops_d  + ω·wr_ops_d         (ops/s)
raw_b(d) = ρ·rd_bytes_d + ω·wr_bytes_d      (bytes/s)
```

Each term is then normalized by its group total, because the three have wildly different magnitudes (in-flight I/O ≈ 0–10, ops/s ≈ 0–50 000, bytes/s ≈ 0–10^9) and unnormalized weights would be meaningless. Normalization makes the three terms **commensurable**; it must not be the last step, because on its own it also destroys the physical meaning of the result. So blend in normalized space, then rescale back onto the in-flight-I/O scale:

$$i_d = \frac{raw_t(d)}{\Sigma_{e \in g}\, raw_t(e)}, \quad o_d = \frac{raw_o(d)}{\Sigma_{e \in g}\, raw_o(e)}, \quad b_d = \frac{raw_b(d)}{\Sigma_{e \in g}\, raw_b(e)}$$

$$\ell_d = T_g \cdot \frac{w_t \cdot i_d + w_o \cdot o_d + w_b \cdot b_d}{w_t + w_o + w_b} \qquad \text{with } T_g = \Sigma_{e \in g}\ \text{raw}_t(e)$$

with `w_t = 1, w_o = w_b = 0` by default — **I/O time is the primary quantity**.

Units of ℓ, and why the rescale is not cosmetic. `T_g` is the group's total average in-flight I/O, so $\Sigma_{d \in g}\ \ell_d = T_g$ exactly and ℓ_d is measured in **average in-flight I/O requests** — the same physical unit as `raw_t`. Under the default weights the rescale is an exact identity, $\ell_d = \text{raw}_t(d)$. Everything downstream depends on this absolute scale:

- §7 compares a plan's benefit against a mirror's cost, where $\omega = 1.0$ means *one* sequential reader or writer in flight. That comparison is only valid if ℓ is on the same absolute scale. Were ℓ left normalized to sum to 1, $\omega = 1.0$ would silently be `T_g` times too large — on a busy group with $T_g \approx 20$ every migration would look twenty times more expensive than it is and the payback test of §7.3 would reject almost everything.
- §5.3's big-M bound needs an absolute load scale.
- The worked example in §14 uses raw loads ($\Sigma\ell = 7.4$, not 1.0) and is self-consistent only under this definition.

Only *ratios* between disks matter to the balance objective, so the rescale changes no optimal assignment; it changes every quantity that is compared against a physical constant. Guard the divisions: if a group's total for a term is 0 that term contributes 0 for every disk rather than producing NaN, and if `T_g = 0` the entire group is idle — skip it, there is nothing to balance.

The objective weights of §5.4 are therefore calibrated in units of in-flight I/O per storage, which is what makes $\alpha = 1.0$ against $\beta = 0.25$ a meaningful default pair: a migration must buy at least a 0.25-request reduction in summed deviation to be worth making.

Why I/O time is the right default. `rate(rd_total_time_ns + wr_total_time_ns) / 1e9` is, by Little's law, the **average number of I/O requests in flight** for that disk. It is dimensionless, it is additive across disks on the same storage, and it is directly comparable between storages of different size and speed. Crucially it *self-weights*: a 1 MiB write that takes 8 ms contributes eight times what a 4 KiB read taking 1 ms does, without anyone having to guess a read/write weight. Pure IOPS treats those two operations as equal and so systematically under-counts large sequential load; pure throughput does the reverse and under-counts small random load. `ops` and `bytes` remain available as additional weighted terms for operators who want to express a policy the array's own timings do not capture.

Because I/O time already embodies the real cost asymmetry between reads and writes, leaving `read_factor = write_factor = 1.0` is correct for the default configuration. The factors exist for the `ops`/`bytes` terms, where the asymmetry is *not* otherwise represented — a write to a RAID-6 array costs far more than a read of the same size, and neither an operation count nor a byte count knows that. Applying them to the I/O-time term as well is supported but double-counts, so do it only deliberately.

One caveat to document: I/O time is a *feedback* signal — it rises when the array is slow, including when the slowness is caused by some other tenant. Combined with the drift gate and cooldowns this is stable in practice, but an operator seeing oscillation should shift weight toward `ops`/`bytes`, which measure only what the guest requested.

Storage load and utilization, where `c_s` is the configured capability weight:

$$L_s = \Sigma_d\ \ell_d \cdot x_{d,s} \qquad u_s = L_s / c_s$$

`u_s` — not `L_s` — is what gets equalized, so a storage with half the spindles can be given `capability_weight: 0.5` and will correctly be assigned half the load.

---

5. The optimization problem

Solved **independently per group** — groups share no variables and no constraints, so a cluster with four groups is four small problems, not one large one.

5.1 Sets and data

Symbol	Meaning
`g`	a storage group
`S`	storages in the group
`D`	movable disks currently in the group
`V`	VMs owning at least one disk in `D`
`v(d)`	the VM owning disk `d`
$\sigma_0(d)$	the storage disk `d` is on **now**
`z_d`	provisioned size of disk `d` (bytes)
ℓ`_d`	load of disk `d` (section 4)
`C_s`	total capacity of storage `s`
U^{ext}	bytes on `s` consumed by volumes DRS does not manage (§5.1.1)
`c_s`	capability weight of `s`
`f_s`	snapshot reserve factor for `s` (default 2.0)
`F_{v,s}`	footprint of VM `v` on `s`: `Σ_{d∈D : v(d)=v} z_{d,s}·x_{d,s}`, the bytes a snapshot of `v` doubles on `s` (§5.3.3)
`T`	split threshold, `snapshot_reserve.split_vm_footprint` (default 2 TiB; `null` disables, §5.3.3)
`T_v`	per-VM footprint cap, `max(T, max_{d∈D : v(d)=v} z_d)` — a single disk larger than `T` cannot be split (§5.3.3)
`V^split`	the VMs whose total size in the group exceeds `T`: `{ v : Σ_{d∈D : v(d)=v} z_d > T }` (§5.3.3)
`soft_s`	configured free-space requirement for `s`, in bytes — the plan endpoint (§5.3.1)
`hard_s`	transient free-space floor for `s`, in bytes, ≤ `soft_s` (§5.3.1, §8.1)
`fmt_d`	current format of disk `d` (`raw`, `qcow2`, ...; §3.5)
`φ(d,s)`	format `d` would have on `s`: `enforce_format_s` for a move onto an enforcing storage, else `fmt_d` (§5.3.2)
`z_{d,s}`	bytes `d` occupies on `s`; equals `z_d` unless the move converts it (§5.3.2)

Provisioned size, never allocated size — on every storage type. `z_d` is what the volume was *provisioned* at, and `Σ_d z_d·x_{d,s} + U^ext` is what a storage is counted as holding, including on thin-provisioned storage (Ceph RBD, LVM-thin, ZFS), where the pool may report far less allocated. This tool never counts on over-provisioning: a plan that fits only while the disks stay thin is one a growing guest can turn into a full pool, and nothing here can bound that growth. So (C4)/(C5), the free-space requirement (§5.3.1), §8.1's transient predicate and the cost model all read the same provisioned sums, and `used` from `GET .../status` is a display figure, never a model input — including at execution time, where §9.2 step 2 re-reads the provisioned figure live from the content listing. The consequence is deliberate: on a thin pool a `free_space.soft` can show a shortfall the pool's own numbers do not.

5.1.1 Computing U^{ext}

```
U^ext = Σ { size(vol) : vol ∈ GET /nodes/{node}/storage/{s}/content ,
                        identity(vol) ∉ D }
```

where `identity(vol)` is the `(vmid, device)` pair the volume belongs to, resolved by matching the volume id against the owning VM's config, and `D` here means *every* disk (C2)'s eligibility pass tracks for this group, pinned or not — `Σ_{s∈S} x_{d,s} = 1` holds for `d ∈ D` regardless of whether a specific `x` is fixed. Everything that is **not** a member of `D` counts as foreign: templates, ISOs and backups, disks of stopped or ungrouped VMs (§3.6/(C2): never fetched, never entered into any `D` at all), disks belonging to another group that shares the storage, unreferenced orphans, and **orphaned target volumes left by a previously failed move** (§9.3). Counting those orphans is intentional — they really do occupy the LUN, and letting them inflate U^{ext} is what makes their cost visible rather than silently eroding the reserve.

Config-excluded disks (`exclude.vmids`/`exclude.disks`/tags/`no-drs`) are *not* on this list. Section 5.3 (C2) pins them into `D` rather than dropping them, precisely so their bytes still count toward `Σ_d z_d·x_{d,s}` in (C4)/(C5) and their fragmentation toward `κ` — see "Pinned disks are modelled, not ignored" in §3.6. Treating a config-excluded disk as foreign instead would double the inconsistency: it would still occupy the reserve calculation correctly by accident (foreign bytes are also subtracted from capacity) but would silently break the affinity accounting for the rest of that VM's disks, which is exactly the failure mode §3.6's rule exists to prevent. An earlier draft of this section listed "excluded" alongside "stopped" and "ungrouped" here, which was a direct contradiction of (C2) rather than a second valid path — fixed in the same commit that first implemented this join in `topology.py`.

Set $U^{ext} = 0$ only if `snapshot_reserve.count_foreign_volumes` is false, which is not recommended.

5.2 Variables

Variable	Domain	Meaning
`x_{d,s}`	`{0,1}`	disk `d` is placed on storage `s`
`y_{v,s}`	`{0,1}`	VM `v` has at least one disk on `s`
`Z_s`	`≥ 0`	largest per-VM footprint on `s`, `max_v F_{v,s}` (§5.3.3)
`o_v`	`≥ 0`	split excess of `v ∈ V^split`: how far its largest footprint on any storage exceeds `T_v` (§5.3.3 (C9))
`e_s`	`≥ 0`	absolute deviation of `u_s` from target (L1 objective)
`d_s`	`≥ 0`	absolute deviation, in TiB, of the bytes on `s` from its fill target $\bar{b} \cdot C_s$ (capacity-spread objective, §5.3 (C7))
`t`	`≥ 0`	maximum utilization (min–max objective)
`r_s`	`≥ 0`	reserve violation slack

5.3 Constraints

(C1) Assignment. Every disk lands on exactly one storage in its group:

```
Σ_{s∈S} x_{d,s} = 1                                   ∀ d ∈ D
```

(C2) Eligibility. Fix `x_{d,s} = 0` wherever placement is impossible or forbidden. This is where all the domain rules live, and doing it as variable fixing rather than as constraints keeps the model small:

- `s` does not have `images` in its `content` list;
- `s` is not shared, or is restricted to nodes that cannot see the VM;
- `s` cannot hold the disk's format **as it would arrive there**, `φ(d,s)` — the source format, or the storage's `enforce_format` when it sets one (§5.3.2);
- `d` or `v(d)` is excluded by config (`exclude.vmids`, `exclude.disks`, tags, `no-drs`) — also pin;
- $\sigma_0(d)$ belongs to **no** configured group — such a disk is unmanaged: it is not in any `D`, it is pinned

where it is, and its bytes count toward U^{sext} of its storage. Report it in `show-load` as "ungrouped, not managed" so an unintended omission from `groups` is visible rather than silent;
- `d`, or any disk of `v(d)`, has a snapshot or an unreferenced companion volume on its storage (§3.7) — pin $x_{d,\sigma_0(d)} = 1$. `move_disk delete=1` cannot move such a volume and would not carry the snapshots if it could;
- `d` has an unapplied pending config change on PVE (§3.8, `GET .../pending`) — also pin. PVE does not reconcile a disk's `pending` entry when that disk is moved, so migrating one leaves the entry referring to state that predates the move;
- `d` is within its per-disk cooldown — also pin to current;
- `v(d)` is currently `locked` (§9.3) — pin for this run. A lock is transient, so this is a *planning-time* pin only and carries no cooldown; the next run re-evaluates it;
- `d` is small (`z_d < migration.tiny_disk_bytes`) and `v(d)` has no larger disk in this group — pin, reported as "small disk with no larger disk of its VM in this group". (C8) below would hold it in place anyway, having nothing to follow; the pin is what makes the reason visible.

(C3) VM affinity linking. Couple `y` to `x` in both directions so the objective term is exact. Let `D^mov ⊆ D` be the disks that are not pinned by (C2):

```
x_{d,s}  ≤  y_{v(d),s}                              ∀ d ∈ D^mov, s ∈ S
y_{v,s}  ≤  Σ_{d ∈ D^mov : v(d)=v} x_{d,s}          ∀ v ∈ V, s ∈ S
```

`D^mov` is the range only when `objective.affinity_counts_pinned_disks: false` is set explicitly. **By default the linking ranges over all of `D`** (`D^mov` replaced by `D` in both constraints above), so that a disk which cannot move this run — snapshot-blocked, config-excluded or locked — still anchors its VM: $y_{v,\sigma_0(p)} = 1$ is fixed for every pinned `p`, and κ then rewards bringing the VM's movable disks to that storage instead of being blind to where they go. §3.6 carries the full argument, including why the opposite default silently penalized affinity repair.

(C4) Largest-footprint linearization. $Z_s = \max_v F_{v,s}$ — the largest *per-VM footprint* on `s`, the sum of every disk one VM has there (§5.3.3 says why it is the sum and not the largest single disk) — is not linear, but because the reserve constraint pushes Z_s *down* while this pushes it *up*, a one-sided bound is exact at the optimum:

```
Z_s  ≥  Σ_{d∈D : v(d)=v} z_{d,s} · x_{d,s}          ∀ v ∈ V_all, s ∈ S
```

`V_all` is every VM owning a disk in `D`, pinned-only VMs included — a pinned disk's bytes are snapshotted with its VM exactly like a movable one's. For a VM with one disk in the group the row is the old per-disk bound $Z_s \geq z_d \cdot x_{d,s}$; the model gets *smaller*, not larger, since there is one row per VM and storage rather than one per disk and storage. (This constraint was the per-disk $Z_s \geq z_d \cdot x_{d,s}$ until the per-VM footprint replaced it, §5.3.3.)

(C5) Capacity, snapshot reserve and free space. The core safety constraint. The reserve is the **larger** of the snapshot term and the configured free-space floor, so introduce $R_s \geq 0$:

```
R_s  ≥  f_s · Z_s
R_s  ≥  soft_s                                          (constant, §5.3.1)

Σ_d z_d·x_{d,s}  +  U^sext  +  R_s   ≤   C_s  +  r_s          ∀ s ∈ S
```

$r_s \geq 0$ is the shortfall, and it is an **integer number of MiB**: the constraint then makes it the byte-exact shortfall rounded *up* to the next whole MiB, and every other place a shortfall is computed (`reserve.compute_reserve_status()`, which `show-load`, the heuristic, the gate and §7.3's repair exemption all read) rounds the same way. Up, not to nearest, so that one byte short is still short: $r_s > 0$ stays exactly the byte-exact violation predicate, and only the amount is coarsened, by under a MiB, in the safe

direction. The granularity is what lets the lexicographic solve below bound stage 2 *exactly* by stage 1's result, and it is the `ε_r = 1 MiB` the big-M alternative names as the smallest shortfall worth refusing to trade. **As built:** a continuous `r_s` with a stage-2 tolerance of `1e-6` MiB (about a byte) against CBC's roughly three-decimal readback made stage 2 infeasible on every group whose minimum shortfall was not a round number of MiB, silently sending it to the heuristic; found on a real bundle and fixed by this integer definition.

Two one-sided bounds are exact for `R_s = max(f_s·Z_s, soft_s)` because (C5) pushes `R_s` *down* while both bounds push it *up*. The `f_s · Z_s` term is the "always keep 2× the largest VM footprint free" rule — enough room to snapshot the VM with the most bytes on `s`, all of its disks there at once (§5.3.3) — and (C4) is what makes it expressible in a linear model at all. `soft_s` is the storage's **configured free-space requirement** (§5.3.1): the number of bytes that must be free on `s` when the plan has fully run, whether the operator asked for it as an absolute byte count or as a percentage of the storage's capacity. It replaces the old global `min_free_bytes` floor (removed, no compatibility shim) — a storage whose largest VM footprint is small needed one (with `f=2` and a 10 GiB largest footprint, the snapshot term alone would reserve only 20 GiB on a 20 TiB LUN), but so does a storage that must keep headroom for reasons the snapshot rule cannot see: a thin-provisioning safety margin, a quota for volumes this tool does not manage, or the operator's plain policy that a LUN is not allowed to run full. **If the snapshot reserve is bigger than the configured free space, the snapshot reserve wins** — the `max()` is the whole integration, and neither term can erode the other.

`soft_s` is a **requirement, not a preference.** A storage that ends the plan below its configured free space is in violation, exactly as if it had breached the snapshot reserve, and the engine must migrate disks away until the requirement is met — the Storage-DRS cluster function: when an admin places a new VM on an overfull storage, the balancer takes care of the situation and moves things so the configured free space is free again. Three properties make that mandate real rather than declared:

1. **The lexicographic stage 1 minimizes it.** `Σ_s r_s` covers both terms of the `max()` — there is one slack per storage, not one per reason — so a free-space shortfall is minimized ahead of balance at the same weight, in the same stage, as a snapshot shortfall. The repair is *winning by construction*: no achievable balance gain can pay for a byte of it (§5.3's two solve paths).
2. **It bypasses the gates.** §6's reserve override fires for a free-space violation exactly as for a snapshot violation — safety is not subject to hysteresis, and neither is a mandate.
3. **It is exempt from payback.** §7.3's aggregate test does not apply to a plan that repairs: the repair is the requirement, and a cost/benefit test would let a large disk's mirror cost veto the very rule the operator configured. The exemption is **plan-level** — decided on the plan's outcome, the `Σ r_s` of the scheduled assignment it ends at (§7.3: the R-02 state the ordered moves actually reach, not the solver's target) against the current assignment's — plan-level in the same way the built `payback.py`'s `has_reserve_override` short-circuits the aggregate test for the whole plan, but on a deliberately **narrower trigger** than that flag's "some move's source was violating at scheduling time". §7.3 states the difference and proves the new trigger is a strict subset of the built one; do not read "plan-level" as "already built". Which moves *carried* the repair is reported per move by the **revert test**: a move is marked `repair: true` iff holding that one disk on its current storage would strictly raise the plan's final `Σ r_s`, evaluated by re-scoring `Σ r_s` on the final assignment with that one `x_{d,σ₀(d)}` held (no re-solve — the plan is fixed; the test asks what the *plan's own* slack would be without the move). The revert test is what covers the indirect repair: a move that empties the destination another repair needs is marked, even though its own source was never in violation (§14.8 works exactly this case).

A group can be **unable** to satisfy `soft_s` — every storage full, or every candidate disk pinned by (C2). That is what `r_s > 0` means, and it is reported exactly as a snapshot shortfall is: loudly, with the byte

amount, as an unfixable shortfall (§9.5). The mandate is to *migrate until the configured free space is free*, not to guarantee that it can be — and when it cannot, the tool says so rather than silently planning around the violation.

5.3.1 Configuring the free-space requirement: `free_space`

`soft_s` and `hard_s` come from the `free_space` config block, resolved per storage at run start — after `/…/` pattern expansion (§11.4), before the model is built, so the solver, the scheduler and every report see plain byte constants.

```
free_space:
  soft: 0                 # bytes, byte-unit string, or "N%" — the plan-endpoint requirement
  hard: null              # same grammar; global null = soft (no transient dip); per-storage
                          # null = inherit the global value
```

The grammar. A value is either

- an **absolute size**: an integer byte count (`1073741824`) or a byte-unit string (`"1 GiB"`, `"512MiB"`) parsed by the same unit parser as `migration.bwlimit_bytes_per_sec`; or
- a **percentage**: a string ending in `%` (`"10%"`), resolved as `round(C_s · N/100)` against *that storage's own capacity*, with `0 ≤ N < 100` — `100%` is rejected by the grammar itself, at config-parse time, rather than accepted here and left to §11.1's `soft_s < C_s` rule, which is an inventory-time check and a different error surface. A percentage is a per-storage number even when it comes from a global or pattern-level setting: one `"10%"` applied across a group of a 20 TiB and a 2 TiB LUN demands 2 TiB and 200 GiB respectively, which is the point — "a tenth of the LUN free" is one policy, not two configs.

Where it can be set, most specific wins:

Level	Key	Applies to
per storage	`groups[].storages[].free_space.soft` / `.hard`	that storage (a literal entry, or every storage a `/…/` pattern matches — §11.4)
global	`free_space.soft` / `free_space.hard`	every storage in every group with no per-storage setting

A pattern entry's `free_space` applies to every storage it matches, exactly as its `capability_weight` does — one entry reserves a whole LUN family, and a literal entry overrides it for the one exception. `null` at the global level means *no free-space requirement from this knob* (the snapshot reserve may still impose one); `null` at the per-storage level means *inherit the global value*, the same inheritance `reserve_factor` already has.

`snapshot_reserve.min_free_bytes` — a global scalar with no per-storage form and no percentages — was the built predecessor of `free_space.soft`. It has been **removed** outright (no fold, no warning; the schema is closed, so a config that still sets it fails validation). `soft_s` is exactly what the inheritance above produced, converted from a percentage where needed, and validated as written: `hard_s ≤ soft_s` and `soft_s < C_s` are §11.1 startup errors, checked against that resolved pair. `hard: null` means `hard_s = soft_s`.

Soft is the plan endpoint; hard is the floor at every instant. `soft_s` is the requirement the finished plan must satisfy — (C5) enforces it, the lexicographic stage repairs it, and §6's override bypasses the gates for it. `hard_s` is the floor the **transient** states may not cross: while moves are in flight a storage may sit below its soft requirement — a mirror target is fully allocated before its source releases anything (§8.1) — and `hard_s` is how far down it may dip. The scheduler's feasibility predicate (§8.1) checks `hard_s` for every intermediate state; the solver's (C5) checks `soft_s` for the endpoint. Defaults: `soft: 0` (no

requirement beyond the snapshot reserve — the pre-§5.3.1 behaviour, so the *model* is unchanged for a config that sets nothing: the same `max(f_s·Z_s, 0)` floor, the same slack, the same §7.3 override). `hard: null` at the **global** level means `hard_s = soft_s` for every storage that does not override it, i.e. *no dip below soft at all* — the conservative reading, and the one that keeps §8.1's invariant exactly as strong as it is today whenever an operator has not thought about transients. Per-storage `null` means *inherit the global value* (below), so the two `null`s never collide: the global `hard: null` is a value ("no dip"), the per-storage `hard: null` is an absence ("use the global").

Two validation rules (§11.1): `hard_s ≤ soft_s` is a **hard error** — a floor above the requirement would make every plan infeasible for a storage that already satisfies its soft requirement; and `soft_s < C_s` — a requirement no disk could ever leave room for is a config typo, not a policy. A percentage of 100 or more never reaches that second rule: the grammar above rejects it at parse time.

An unsatisfiable requirement thrashes, and that is a deliberate cost. A `soft` the group cannot reach (say `"30%"` on a cluster that is 85% full) bypasses the drift and imbalance gates on *every* run (§6), and a plan that reduces the shortfall — the best a saturated group can do — is payback-exempt (§7.3). Both anti-thrash mechanisms are disabled for exactly the case that recurs forever, leaving only `cooldown_per_disk` between successive repair attempts. That is accepted on purpose: the alternative is hysteresis on a safety property, which §6 already forbids for the snapshot reserve. The mitigations are the reporting ones — the unfixable-shortfall line of §9.5 names the byte amount every run, so an operator staring at a permanent WARN fixes the config or the cluster — and the fact that a plan that moves nothing costs only read queries. Do not "fix" this with a back-off timer without revisiting §6's no-hysteresis-on-safety rule first.

`r_s` is a **repair slack**, not a licence to overfill. A storage can already be violating the reserve when the engine first runs (see the worked example in §14), and a hard `≤ C_s` would make the model infeasible and the tool useless exactly when it is most needed. Two ways to keep it effectively hard:

1. **Lexicographic, preferred and provably correct.** Solve in two stages: minimize `Σ_s r_s` alone; then fix `Σ_s r_s` to that minimum as a constraint (`Σ_s r_s ≤ min`, exact: both sides are whole MiB, (C5) above) and minimize the §5.4 objective. CBC supports this by re-solving. The reserve is then never traded against balance at any weight, and `Σ r_s > 0` provably means *physically impossible*, not merely *unattractive*.
2. **Single-stage big-M**, simpler but requiring calibration: keep `P · Σ_s r_s` in the objective with `P` large enough that no achievable gain from the other terms can pay for a violation worth caring about. `P` must be **computed at model-build time, not taken from config as a fixed number**, because the bound depends on the group's absolute load `T_g` (§4):

```
U_obj  =  2·α·T_g  +  β·|D|  +  γ·Σ_d z_d  +  κ·w_max·|V|·(|S|−1)
       +  δ·2·(Σ_d z_d + Σ_s U^sext)
       +  μ·Σ_{v∈V^split} (Σ_{d : v(d)=v} z_d − T_v)
                                                              (upper bound on the
                                                              non-reserve objective;
                                                              Σ_s d_s ≤ 2·(Σ z_d + Σ U^sext)
                                                              by (C7),
                                                              w_max = max_v w_v, since the κ
                                                              term is w_v-weighted (§5.4),
                                                              o_v ≤ Σ z_d − T_v by (C9))
P_min  =  U_obj / ε_r          with  ε_r = the smallest reserve shortfall we refuse to trade
```

`Σ_s e_s ≤ 2·T_g` because every `u_s` lies in `[0, T_g/c_s]` and the deviations from `u*` sum to at most twice the total; `Σ_s d_s ≤ 2·(Σ_d z_d + Σ_s U^sext)` for the same reason, since `Σ_s |B_s − b̄·C_s| ≤ Σ_s B_s + b̄·Σ_s C_s` and both sums are the group's total bytes. Take `ε_r = 1 MiB`

expressed in the same size unit as `z_d` (2^{-20} TiB if sizes are TiB), which says: never accept even a one-mebibyte reserve shortfall in exchange for balance. Config `objective.reserve_violation_penalty` is then a *floor*, not the value used: the model uses `P = max(configured, P_min)` and logs a warning when it had to raise it. That warning must be *checkable*, not just an announcement: log `P_configured`, `P_min`, `P_used`, and the seven inputs the bound came from — `T_g`, `|D|`, `Σ_d z_d`, `Σ_s U^ext`, `w_max`, `|V|·(|S|−1)` and the summed split headroom `Σ_{v∈V^split} (Σ z_d − T_v)` (§5.3.3) — plus the `ε_r` granularity, and repeat them in `pve-storage-drs explain`. `P_min` moves with `T_g`, so the same config file legitimately yields different effective penalties on a quiet group and a busy one, and on the same group at different times of day. An operator who sets `reserve_violation_penalty: 5000` and sees the engine using 2.3×10^7 needs to be able to reconstruct that number rather than take it on faith.

Worked against §14 (`T_g = 7.4`, `|D| = 6`, `Σz = 6.5 TiB`, `|V| = 5`, `|S| = 3`, `δ = 0.5`, `w_max = w₁₀₁ = 2.703` (§14.3), sizes in TiB, and the split rule off as that fixture sets it, so the μ term is 0, no foreign volumes): `U_obj = 14.8 + 1.5 + 0.325 + 13.51 + 6.5 = 36.64`, so `P_min = 36.64 · 2²⁰ ≈ 3.84×10⁷` (the figure `fc-tier1.expected.json` records as `big_m_p_min_computed`; an earlier revision of this bound used a flat κ weight of 1 and read `24.6`, 2.58×10^7 — REVIEW.md AO-05 — and the relative-fill revision of (C7) bounded the δ term by `δ·2·|S| = 3.0` and read `33.14`, 3.47×10^7). The configured default `P = 1000` is **four orders of magnitude too small** to be provably dominant at mebibyte granularity — it is dominant for violations above roughly 37 GiB and silently tradeable below that. This is precisely why option 1 is the default and this option needs the computed `P`.

Use the lexicographic solve (option 1) by default. It needs no calibration, its correctness does not depend on `T_g`, and both backends support it by re-solving. Option 2 exists for a backend that cannot re-solve cheaply.

As built (REVIEW.md V-01): option 2's `P_min` computation and `max(configured, P_min)` floor above were never implemented — there is no build-time computation of a provably-dominant `P` anywhere in `src/`. Both MILP backends (`optimize.py`) implement option 1 only, and never consult `objective.reserve_violation_penalty` at all. The heuristic backend (`heuristic.py`) is the key's one live consumer: it multiplies the *configured* value straight into its objective as `reserve_penalty_term = objective.reserve_violation_penalty * reserve_shortfall_tib`, with no floor and no warning if it is too small to be provably dominant for a given group. Until option 2 is implemented, read this section's `P = max(configured, P_min)` machinery as the target design for the single-stage path, not current behaviour — see `docs/manual/10-configuration.md`'s `objective.reserve_violation_penalty` entry for what the key does today.

`tests/fixtures/reserve-tradeoff.yaml` (§14.6) is the fixture for this rule: a two-storage group in which the two options provably disagree, with the exact `P` at which big-M flips recorded alongside the computed `P_min`. Test both paths against it. The §14 fixture cannot do this job — there, every reserve-violating assignment is also worse on balance, so both options agree at any `P`.

Report any residual `r_s > 0` prominently as an unfixable shortfall, with the byte amount (§9.5 gives the line; as built, it also names the pinned disks on the short storages, since those are what an operator can act on).

(C6) Spread. With `u* = (Σ_d ℓ_d) / (Σ_s c_s)` — a **constant**, since total group load is invariant under reassignment — the L1 form is fully linear:

```
u_s = (Σ_d ℓ_d·x_{d,s}) / c_s
u_s − u*  ≤  e_s        and        u* − u_s  ≤  e_s     ∀ s ∈ S
```

The min–max alternative is `t ≥ u_s ∀s`. L1 is the default: min–max only sees the single hottest storage

and is indifferent to everything below it, which tends to produce plans that fix the worst storage and ignore a second nearly-as-bad one.

(C7) Capacity-spread linearization. Let $b_s = (\Sigma_d z_d \cdot x_{d,s} + U^{ext}) / C_s$ be the storage's **fill fraction** and $\bar{b} = (\Sigma_d z_d + \Sigma_s U^{ext}) / (\Sigma_s C_s)$ the group's mean fill — a constant, like u^*, because total bytes are invariant under reassignment. Each storage's **fill target** is $\bar{b} \cdot C_s$, its capacity-proportional share of the group's bytes, and d_s measures how far the bytes actually on it, $B_s = b_s \cdot C_s$, are from that target, **in TiB**:

$$B_s - \bar{b} \cdot C_s \le d_s \qquad \text{and} \qquad \bar{b} \cdot C_s - B_s \le d_s \qquad \forall s \in S$$

B_s is linear in x, so no variables beyond d_s are needed. d_s carries the same size unit as z_d: the CBC model holds sizes in MiB (§5.5), so δ enters the objective as $\delta / 2^{20}$ per MiB, exactly as γ does.

Why absolute TiB, and not the fill deviation relative to $\bar{b}$. An earlier revision divided by $\bar{b}$, $\Sigma_s |b_s - \bar{b}| / \bar{b}$, to make the term scale-free. That made its *level* independent of the group's size and its *slope* dependent on it, and the slope is what decides whether a move is made. Moving X TiB from a storage above its target to one below it, with neither crossing its target, lowers $\Sigma_s d_s$ by exactly $2X$, and the objective by $2\delta \cdot X$. Under the relative form it lowered the objective by $2\delta \cdot X / \bar{B}$ on storages of equal size, where $\bar{B}$ is the average number of TiB per storage. A move costs $\beta + \gamma \cdot X$ (§5.4), so under the relative form a move paid for itself only for disks larger than $\beta / (2\delta/\bar{B} - \gamma)$. That threshold grows with the amount of data in the group, and once $\bar{B} \ge 2\delta/\gamma$ (20 TiB per storage at the defaults) no capacity-only move of any size could pay. Disk sizes do not grow with a cluster, so a group with ten times the data needs ten times the moves to correct the same relative imbalance, each of which earned a tenth as much. Found on a replayed production bundle (`cluster-6`): five 20 TiB storages holding 12.3/3.1/15.0/1.3/0 TiB, the capacity gate at 236 %, $\bar{B} = 6.33$ TiB, so only disks above about 2.3 TiB could ever move for data spread, and the group held none that would not overshoot. Every run opened the group and moved nothing.

The relative form also measured the wrong risk. What a storage failure costs is bytes, so a group filled to 5 % that keeps 60 % of its bytes on one storage really does have less data at risk than a full one with the same shape. In absolute TiB its deviation is smaller, and its term is smaller too. The *gain per TiB moved* stays 2δ in both groups, so the smaller group is not neglected: every move toward the target still earns its full price. The concern that first motivated dividing by $\bar{b}$, that a term "stops acting as the group empties", is true of deviations measured in fill *fractions*, whose slope per TiB moved falls as capacity grows. It is not true of TiB, whose slope per TiB moved is constant.

If $\bar{b} = 0$ the group holds no data: the term is inactive, and so is §6's capacity gate. The fill counts managed disks and foreign volumes but **not** the snapshot reserve — the quantity spread is *data at risk*, and free space is not data.

(C8) Small disks follow their VM. Let $D^{small} = \{ d \in D^{mov} : z_d < migration.tiny_disk_bytes \}$ and, for $d \in D^{small}$, let $B(d)$ be the disks of $v(d)$ **in this group** that are not small, pinned ones included. Then, for every storage other than the disk's current one:

$$x_{d,s} \le \Sigma_{b \in B(d) \cap D^{mov}} x_{b,s} + |\{ b \in B(d) \setminus D^{mov} : \sigma_0(b) = s \}| \qquad \forall d \in D^{small}, s \neq \sigma_0(d)$$

A small disk ends the plan where it is, or on a storage where at least one larger disk of its own VM ends it. That allows every move a small disk is actually wanted for — moving together with its VM, rejoining a VM that is already split, following one half of a VM that the plan splits — and forbids the one it is not: moving alone. Moving alone is never worth it. A small disk carries next to no load, so it cannot buy balance; κ alone can only prefer it next to its VM, which the constraint already allows; and as a (C5) repair it frees less than `tiny_disk_bytes` of a shortfall while splitting the VM. That last case is not hypothetical: found

on a real bundle, a storage 231 GiB short, all its large disks pinned, and the lexicographic stage 1 moving a 528 KiB `efidisk0` to another storage because it lowered $\Sigma\ r_s$ by one whole MiB (a MiB boundary crossed). Whole-MiB shortfalls cannot prevent that — any quantisation has boundaries — so the rule is on the disk, not on the arithmetic.

Consequences, all deliberate:

- **Large disks keep their freedom.** A VM's larger disks may still be split across storages when they do not fit together, or when only a split balances; κ (§5.4) is the soft preference against it, as before. (C8) constrains the small ones only.
- **Groups are never crossed.** $B(d)$ counts only the VM's disks in *this* group, and every x of the model ranges over this group's storages. A VM whose system disk sits in another group has nothing here for its small disk to follow: that disk is pinned by (C2) above and never moves.
- **Staying is always allowed**, so the current assignment satisfies (C8) and the model never becomes infeasible because of it.
- **`tiny_disk_bytes: 0` switches it off** — no disk is small.

`topology.small_disk_placement_ok()` is the rule's one definition. The MILP transcribes it as the constraint above; the heuristic discards any candidate assignment that breaks it — in its descent, its (C5) repair step and `explain`'s closest-alternative search — and the exhaustive fixture generator filters its enumeration by an independent restatement of it.

5.3.2 Enforcing a disk format on a storage: `enforce_format`

By default a move preserves the disk's format: `move_disk` is called without `format=`, PVE keeps the source format, and (C2) keeps a disk off any storage that cannot hold that format (§3.5). That stays the default. `enforce_format` is the operator's opt-in to the alternative — **a storage that names a format gets every disk the plan moves onto it in that format**, converted during the move if the source is in another one. The motivating case is shared (non-thin) LVM on PVE 9.2, where a `qcow2` LV is what enables volume-chain snapshots (§3.5): an operator who wants every disk that lands on such a LUN to become snapshot-capable sets `enforce_format: qcow2` on it; one who wants the opposite — everything on a LUN raw, for the performance of a plain block device — sets `raw`.

```yaml
groups:
  - name: fc-tier1
    storages:
      - id: /san-.*/
        enforce_format: qcow2   # every disk moved onto any san-* LUN arrives as qcow2
      - id: san-b
        capability_weight: 0.5  # literal beats pattern (§11.4): san-b preserves formats
```

Shape. A per-storage option on `groups[].storages[]` entries, literal or pattern (§11.4), with the same precedence every per-storage option has: a pattern entry's value applies to every storage it matches, a literal entry replaces the pattern's options wholesale. Values: `raw`, `qcow2`, or `null` (the default — preserve the source format, today's behaviour). There is **no global `enforce_format`**: a format is a property of a storage type, and a single cluster-wide value would be invalid on half the storages of any mixed group. `vmdk`, which PVE's `move_disk` also accepts, is deliberately not offered — nothing in this tool's scope needs a balancer to produce VMware images.

The target format. For disk d and storage s let

$$\varphi(d,s) = \begin{cases} \text{enforce_format}_s & \text{if } s \neq \sigma_0(d) \text{ and enforce_format}_s \text{ is set and } d \text{ is not tpmstate0} \\ \text{fmt}_d & \text{otherwise} \end{cases}$$

where fmt_d is the disk's current format (§3.5). φ is what the disk *would be* on s; it only differs from fmt_d

for a storage the disk would have to move to. Three rules follow from it:

1. **Eligibility uses φ, not `fmt_d`.** (C2)'s format bullet becomes "`s` cannot hold `φ(d,s)`". Since §11.1 validates `enforce_format_s` against the storage type, an enforcing storage accepts every disk on format grounds — enforcement *widens* eligibility (a `qcow2` disk on a directory storage becomes eligible for an LVM-thin target enforcing `raw`, which it was not before) and never narrows it. `topology.storage_accepts_format()` stays the single implementation; it gains `target_format(d, s)` as its one caller-facing companion, and both solver backends and the executor ask that pair, never `fmt_d` directly (AGENTS.md §5: one implementation, two callers).
2. **Enforcement never creates a move.** A disk already on `s` in a format other than `enforce_format_s` is *non-conforming*, and that is **not a violation**: no slack, no objective term, no gate override. The balancer converts a disk only as a side effect of a move it chose for its own reasons (balance, reserve, free space, affinity), and the payback test of §7.3 prices that move exactly as it would without the conversion. A disk is never moved *in order to* convert it, and never moved off and back. Converting a whole storage in place is a storage-policy operation the operator runs with `qm disk move` themselves; a "conform" mode is out of scope for this phase and would need its own section here before it is built. Non-conforming disks are counted per storage by `verify-storages` (§3.5) so the operator can see how far a storage is from its policy.
3. **`tpmstate0` is exempt and keeps its format.** swtpm state is a raw volume, and whether PVE would convert or refuse a `format=qcow2` move of it is **not verified**; the exemption is the safe answer either way and costs nothing — it is a few MiB below `migration.tiny_disk_bytes`. `efidisk0` is *not* exempt: PVE stores EFI vars as `qcow2` on file-backed storages routinely. That an `efidisk0` converts cleanly in both directions on PVE 9.2 is an item for the phase 16 live test (§12), not an assumption.

Size on the target: `z_{d,s}`. A converting move does not produce a copy of the source volume; PVE allocates a new one of the target kind. §9.2 step 2 already charges such a move at the larger of the listed size and the VM config's `size=` (as the operator describes PVE's allocation, not read from its source). Enforcement makes that case a planned one rather than a guard, so the model must count it at the endpoint too, not only in flight:

```
z_{d,s} = max(z_d, z_d^cfg)    if φ(d,s) ≠ fmt_d
        = z_d                  otherwise
```

`z_d^cfg` is the disk line's parsed `size=` (`z_d` when the line carries none). (C4), (C5), (C7) and §8.1's transient predicate use `z_{d,s}` wherever they use `z_d` for a disk placed on `s`; the MILP's capacity rows simply get per-pair coefficients, which keeps the model linear. With no `enforce_format` anywhere `z_{d,s} = z_d` identically, so every existing fixture and every existing plan is unchanged. The §7.1 cost keeps `z_d`: the mirror reads the source's virtual size whatever the target's format. The group mean fill $\bar{b}$ of (C7), a target rather than a constraint, also stays at `z_d`.

The `qcow2`-on-`lvm` allocation (verified against PVE's source). A `qcow2` image on an LV needs room for its own metadata, and PVE allocates for it: `LVMPlugin::alloc_image` → `alloc_lvm_image` creates the LV with `calculate_lvm_size()`, which for `qcow2` is `qemu-img measure`'s `fully-allocated` figure for the requested size — the virtual size *plus* the image's L1/L2 and refcount metadata — not `size=`. A `raw` LV is allocated at exactly the requested size. (The same function checks the VG's free space against the unpadded `size`, not the padded LV size, so PVE's own refusal is not a substitute for ours.) With the default 64 KiB clusters, which the plugin uses for a volume without a backing image, that metadata is an 8-byte L2 entry and a 2-byte refcount entry per cluster, about 0.015 % of the size. The tool does not shell out to `qemu-img`, so `z_{d,s}` takes a third term, an **upper bound** on it:

```
z_{d,s} = max(z_d, z_d^cfg, qcow2_lvm(max(z_d, z_d^cfg)))   if φ(d,s) = qcow2, φ ≠ fmt_d and s is type lvm
qcow2_lvm(n)  = n + ⌈n / 4096⌉ + 8 MiB                      (0.0244 % + header/table/extent rounding)
```

The bound is deliberately above qemu's figure for every size: under-counting would erode the reserve by the metadata of every converted disk, a direction AGENTS.md §6 does not permit. It applies only to a *converting* move onto `lvm`, so the no-enforcement identity `z_{d,s} = z_d` holds; a `qcow2` disk that merely moves between storages without conversion keeps its listed size as before. **Still unverified on a live cluster:** that `efidisk0` converts in both directions (phase 16's live test), and that the target's listed size stays within this bound.

What the operator gives up. Converting `qcow2 → raw` on LVM removes the disk's ability to take volume-chain snapshots on that storage; converting `raw → qcow2` adds a qcow2 layer to every I/O. Both are the point of the option, and both are the operator's policy, stated once in config. Disks with snapshots are pinned by (C2) regardless (§3.7), so no existing snapshot chain is ever converted.

5.3.3 The per-VM snapshot footprint, and splitting large VMs

A snapshot is taken of a VM, not of a disk. Creating a snapshot snapshots *every* disk of the VM at once (confirmed by the operator; §3.7 relies on the same fact when one VM-level snapshot pins all of a VM's disks). On a volume-chain storage each snapshotted disk gets a new full-size volume on the storage the disk lives on, so what a snapshot of VM `v` needs on storage `s` is not the size of `v`'s largest disk but the sum of all of `v`'s disks *on `s`* — its **footprint** there:

```
F_{v,s}  =  Σ_{d∈D : v(d)=v} z_{d,s} · x_{d,s}
Z_s      =  max_v F_{v,s}
```

An earlier revision defined `Z_s` as the largest single disk on `s`. That under-reserves every storage holding two or more disks of the same VM: a VM with five 2 TiB disks on one storage needs 10 TiB for one snapshot, and the per-disk rule reserved for 2 TiB of it. The footprint replaces the largest disk everywhere the reserve is computed — (C4)/(C5), §8.1's transient predicate, §9.2's live re-check, `reserve.compute_reserve_status()` and therefore `show-load`, the gate, the heuristic and §7.3's repair test. `f_s` keeps its meaning (the number of snapshots' worth of room to keep, default 2.0) and its default; what it multiplies is now the right quantity. The per-disk `z_max` survives in one place only, the wipe-time estimate of §7.1/§9.3, because a source wipe is per volume.

Scope of the sum, stated so it is not re-derived differently in two places:

- **Every disk of `v` in `D` on `s` counts**, movable or pinned (§3.6) — a pinned disk is snapshotted with its VM like any other. `efidisk0` and `tpmstate0` count; at under a few MiB they never matter. `unused{N}` volumes count when `exclude.include_unused_disks` brings them into `D`; whether PVE snapshots an `unused` volume has **not been verified** against the source here, and counting it errs toward more reserve, never less.
- **Only `s` itself.** A snapshot of `v` also needs room on every *other* storage holding a disk of `v`, but that room is those storages' own `F_{v,s'}` and is charged there. This is what keeps the per-group decomposition of §5 intact: `F_{v,s}` only ever involves disks on `s`, which all belong to `s`'s group.
- **Foreign volumes contribute bytes, not footprint — by decision.** Disks of stopped or ungrouped VMs count in full toward used space through U^{sext} (§5.1.1), but contribute no footprint, as they never contributed a largest disk. A stopped VM is planned neither for snapshots nor for I/O (operator direction): the snapshot reserve exists for running workloads, and a stopped VM is not one. Its bytes are what matter, and those are counted.

Upgrade consequence: the reserve gets larger, and a cluster can be in violation on the first run. Any storage holding two disks of one VM now reserves more than before, and one that was compliant under the per-disk rule can show an `r_s > 0` it did not have yesterday. That is the rule being right, not the cluster changing — and the consequence is a **repair**: §6's reserve override opens the group and §7.3 exempts the plan from payback, so the first run after the upgrade can plan substantial migrations. Dry-run is

the default (§9.1); `show-load` and `verify-storages` print the new `Z_s` and the VM it comes from before anything moves, and the manual must tell an upgrading operator to read them first.

Splitting large VMs. The footprint rule alone makes a large VM expensive to keep on one storage only when space binds: (C5) then forces a split, as a repair. On a storage with room to spare the reserve is satisfied, nothing moves, and the storage carries `f·10 TiB` of free space for one VM that would need only `f·2 TiB` per storage if its disks were spread. The **split rule** makes spreading a large VM a standing preference rather than a last resort:

- `T = snapshot_reserve.split_vm_footprint` (byte grammar as `free_space`, §5.3.1: integer bytes or a byte-unit string; default `"2TiB"`; `null` disables the rule, and with it (C9), the μ term and the split gate) is the most of one VM that one storage should hold.
- `T_v = max(T, max_{d∈D : v(d)=v} z_d)` is VM v's cap. A disk larger than `T` cannot be split, so its own storage may hold it in full; anything *added* to it there is excess.
- `V^split = { v : Σ_{d∈D : v(d)=v} z_d > T }` are the VMs the rule applies to. A VM no larger than `T` in total can never exceed it on one storage, so restricting the rows below to `V^split` only keeps the model small; it changes no optimum.

(C9) Split excess. One continuous variable per large VM, bounded below by its footprint on every storage:

```
o_v  ≥  Σ_{d∈D : v(d)=v} z_{d,s} · x_{d,s}  −  T_v          ∀ v ∈ V^split, s ∈ S
o_v  ≥  0
```

The objective (§5.4) charges `μ · Σ_{v∈V^split} o_v`, so the bound is exact at the optimum by the same one-sided argument as (C4). `o_v` is the *peak*, not a sum over storages, on purpose: the reserve a storage needs is driven by the largest footprint on it, so the quantity to drive down is the largest share any one storage holds. A sum of per-storage excesses would score five 2 TiB disks on three storages as 6+2+2 and 4+4+2 alike (excess 4 TiB either way, at `T = 2 TiB`), and the first needs half again as much reserve on its fullest storage.

Fewer storages than disks is the normal case, and the rule copes with it. Five 2 TiB disks, `T = 2 TiB`:

Storages available	Best split	Footprints (TiB)	o_v	Snapshot reserve on the fullest, `f = 2`
1	—	10	8	20 TiB
3	2 + 2 + 1 disks	4, 4, 2	2	8 TiB
4	2 + 1 + 1 + 1 or 2 + 2 + 1	4, 2, 2, 2 or 4, 4, 2	2	8 TiB
5	1 each	2, 2, 2, 2, 2	0	4 TiB

The cap is unattainable with fewer than five storages, and that is not a failure: `o_v` is minimised, not required to reach zero, so the solver lands on the smallest peak the storages allow. Where two splits reach the same peak (the four-storage row), κ (§5.4) breaks the tie toward fewer storages — the affinity preference survives inside the split rule, so a large VM is spread only as far as the cap requires. The reserve, not the split rule, is what prefers the wider of two equal-peak splits when space is tight, through (C5). §14.9 works the three-storage row as a fixture.

Weight, and how it meets affinity. `μ = objective.mu_vm_split_per_tib` (default 1.0, per TiB of `o_v`, in the same TiB scaling as γ; `0` disables the term, the split gate **and** rule 1 below — with the policy off there is nothing for the weight to defer to, so `V^split` is empty for every purpose, the weights included). Three rules make it the *strong* preference §1 asks for rather than one κ can veto:

1. **A large VM's affinity is not I/O-weighted.** `w_v = 1` for every `v ∈ V^split` (§5.4 otherwise gives `w_v = max(1, ℓ_v/ℓ⌋)`). The weight exists to keep a busy VM together; for a VM above `T` the operator

has said it should not be together, and its I/O must not veto that policy. κ still counts its extra storages at weight 1 — that is the tie-break above.

2. **Break-even, at the defaults.** Moving disk d of a large VM off its peak storage lowers o_v by up to z_d, and costs β + γ·z_d, plus κ if it opens a storage the VM did not use yet. So the move pays on the objective when μ·z_d > 0.25 + 0.05·z_d — any disk above about 0.26 TiB — onto a storage the VM already uses, and when μ·z_d > 0.75 + 0.05·z_d — above about 0.79 TiB — onto a new one (α/δ effects aside: a split that wrecks I/O balance still has to pay for that). A VM made of many small disks therefore splits more reluctantly than one made of a few large ones; raising μ is the knob, and §11.1 warns when it is below κ, where a single TiB of excess no longer buys a new storage.
3. **The split is a preference, not a mandate.** o_v is in stage 2 of the lexicographic solve (§5.3), never stage 1: a split never displaces a reserve or free-space repair, and a split that would itself create a shortfall is never chosen. Cooldowns, (C8), the transient invariant (§8.1) and the hard per-move rules of §7.3 all apply.

The split gate. A group with I/O in balance and data evenly spread would never be planned (§6), so a large VM sitting on one storage would never be split. §6 gains a gate that opens the group when the split rule can make progress — see §6 for its exact test, which is chosen so that a VM already at the best split its storages allow does not hold the group open on every run.

Payback. o_v persists for as long as the placement does, like E, F and A, so §7.2's benefit gains μ·(O_before − O_after)·H with O = Σ_{v∈V^split} o_v, and a move that piles a large VM back onto one storage pays for it out of its other gains. At H = 365d the term dwarfs any mirror cost — one TiB of excess removed is worth 3.15×10^{7} load·s against ~1.0×10^{4} to mirror a TiB — so, as for κ, the aggregate test defers to the solver's own μ-versus-β/γ pricing. A split is *not* a repair: it gets no §7.3 exemption of its own, and a plan that both splits and repairs is exempt because it repairs.

Transient states. While disk d of VM v is mirroring onto b, it is counted on both storages (§8.1), and so is its share of v's footprint: v's footprint on b rises by z_{d,b} the moment the move starts, and its footprint on the source does not fall until the source volume is observed gone. §8.1 states the predicate.

Heuristic. The heuristic (§5.5) evaluates the same objective, μ term included, through the shared `evaluate_assignment()`, so its single-disk moves already split a large VM. Two of its steps need saying: the repair step's "most reduces the violation per byte moved" now counts the drop in f_s·Z_s a move causes as well as the bytes it frees, and the polish step never reunites a V^split VM's disks past T_v — a reunion that raises o_v is not polish.

5.4 Objective

```
min   α · Σ_{s∈S} e_s                              (imbalance)
    + β · Σ_{d∈D^big} (1 − x_{d,σ₀(d)})            (number of migrations)
    + γ · Σ_{d∈D^big} z_d · (1 − x_{d,σ₀(d)})      (bytes migrated)
    + κ · Σ_{v∈V} w_v · ( Σ_{s∈S} y_{v,s} − 1 )    (VM disk fragmentation)
    + δ · Σ_{s∈S} d_s                              (data spread / failure risk)
    + μ · Σ_{v∈V^split} o_v                        (large-VM split excess, §5.3.3)
    + P · Σ_{s∈S} r_s                              (reserve violation)
```

with D^big = { d ∈ D : z_d ≥ migration.tiny_disk_bytes } (default 64 MiB), w_v = max(1, ℓ_v / ℓ⌋ defined below — except w_v = 1 for v ∈ V^split (§5.3.3) — and μ, V^split, o_v from §5.3.3.

(1 − x_{d,σ₀(d)}) is exactly 1 when disk d moves and 0 when it stays, so β directly implements "minimize the number of migrations" and γ biases against moving *large* disks specifically. Both terms range over D^big, not over D: a disk below tiny_disk_bytes transfers in under a second and holds its VM's lock for seconds, so charging it a migration count would price exactly the affinity-repair moves the κ term exists to enable out of the plan — a tiny disk rejoins its VM whenever that reduces spread, at zero objective cost.

κ counts the number of **extra** storages a VM is spread across, so it is 0 for a VM whose disks are all together and grows by 1 per additional storage — a soft preference that free space (C5) or a strong imbalance can legitimately override, as required. Whether a VM *can* fit on one storage is (C5)'s business; where it cannot, κ still rewards keeping the spread as narrow as possible. Two strengthenings of the affinity term, both motivated by the same live finding (a plan of four pure affinity repairs rejected by the payback rule, §7.2):

- **The preference is weighted by the VM's own I/O** — for every VM not in `V^split`; a VM the split rule says should be spread keeps `w_v = 1`, so its I/O cannot veto that rule (§5.3.3). `w_v = max(1, ℓ_v / ℓ̄)`, where `ℓ_v = Σ_{d ∈ D : v(d)=v} ℓ_d` is the VM's total load within the group (pinned disks included — their I/O is the VM's I/O) and `ℓ̄ = T_g / |V_all|` the group's mean per-VM load, with `V_all` **every distinct VM owning a disk in the group, pinned-only VMs included** — deliberately wider than (C3)'s `V`, which by default ranges over movable-disk VMs only. A pinned-only VM's I/O is still real traffic on the group's storages, and diluting it out of the mean would make `ℓ̄` depend on which disks happen to be movable this run rather than on the group's actual load; the §14.7 fixture is worked with VM 309 pinned entirely out of `V` and still divides by all three of the group's VMs (`ℓ̄ = 6.0/3`, not `6.0/2`). A VM doing several times the average I/O is worth correspondingly more to keep together: its fragmentation is measured in the same in-flight-I/O unit as everything else (§4), and comparing the full VM's I/O against the single disks the balance term shuffles is exactly the comparison that decides whether the VM should move as a unit or be scattered. The floor of 1 keeps a quiet VM's fragmentation worth the same as before — the observed failures were quiet VMs whose reunions the solver *did* want, killed later by payback — so only the ceiling is new; the default κ itself is deliberately unchanged and remains the knob to turn if quiet-VM cohesion should tighten further. `w_v` is data, not a variable, so this stays a plain per-VM coefficient, and the §14 fixture's optima survive it (there the heaviest VM's cohesion rises 2.7× and the split still wins — §14.3 works the new arithmetic).
- **Disks too small to matter are free to place.** With `D^big` excluding them from β and γ, and §7.1/§7.3 exempting them from cost and from the aggregate acceptance test, a tiny disk's placement is decided by κ and the capacity constraints alone. Churn stays bounded: κ only rewards *reducing* spread, and per-disk cooldowns (§6) still apply.

κ measures **within-group** fragmentation only. Because the problem decomposes per group and a disk can never leave its group, a VM with disks in two different groups is not counted as fragmented — that spread is structural and no migration could ever repair it. This is a consequence of the decomposition, not an oversight.

δ spreads **bytes**, not load. The failure of a storage takes with it everything on it, so an even fill fraction bounds the share of the group's data that any single failure costs, and it keeps peak fill — hence reserve headroom — uniform across the group. `Σ_s d_s` is in TiB (§5.3 (C7)), so δ is a price per TiB of misplaced data, in the same unit as γ's price per TiB copied.

Which disks the term moves on its own. Moving a disk of `X` TiB from a storage above its fill target to one below it, without either crossing its target, earns `2δ·X` and costs `β + γ·X`, so a capacity-only move pays when

```
X  >  X*  =  β / (2δ − γ)              (no move ever pays when 2δ ≤ γ)
```

At the defaults (`β = 0.25`, `γ = 0.05`, `δ = 0.5`), `X* = 0.25 / 0.95 ≈ 0.26 TiB ≈ 270 GiB`. Disks smaller than that do not move for data spread alone. They still move when a larger disk of the same VM moves and κ makes following it cheaper than splitting the VM, and they still move for I/O balance. `X*` does not depend on how much data the group holds, so the same weights mean the same thing on a 5 TiB group and on a 500 TiB one. A move that overshoots, leaving the source below its target or the target above it, earns less, and the solver accounts for that exactly. A move that splits a VM also pays `κ·w_v`. On the

`cluster-6` bundle that motivated this form (§5.3 (C7)), whose storages are all the same size, so the new term equals the old relative one at $\delta_rel = \delta \cdot \bar{B} \approx 3.2$, a replay at that weight plans seventeen data disks in the five-storage group (thirteen between 290 GiB and 1.95 TiB, and four of 100–200 GiB that follow a larger disk of their VM) and three in the three-storage one (0.7–1.95 TiB). Both plans clear payback by a ratio of several thousand. At the old default the same bundle planned nothing on either group. This is live-verified on the operator's cluster and **not replayable from the repository**: the `cluster-6` bundle is not committed. The committed corpus sweeps only `beta_move_count: 0.0` (where $X^* = 0$), and at the default β its 24h bundle correctly plans no capacity move (its total fill deviation is 2.28 TiB, so $\delta \cdot \Delta F$ tops out at 1.14 against a cheapest plan costing about 1.56), so it demonstrates the mechanism and not this criterion.

I/O balance stays the first priority, as an exchange rate. At $\delta = 0.5$, one TiB of data moved from above its target to below it is worth one in-flight request of summed load deviation ($2\delta = 1.0$, against $\alpha = 1.0$). A disk the solver moves for data spread is therefore one whose load it can place without giving back more I/O balance than the move earns. In practice those are the group's cold disks, and on an imbalanced group the α term still dominates the comparison (in §14.2: $\alpha \cdot \Sigma e = 8.07$ against $\delta \cdot \Sigma d = 2.33$). δ decides among plans the load objective is nearly indifferent between, and acts on its own when a group's I/O is balanced but its data is concentrated — the case §6's capacity gate exists to reach.

Like β and κ, δ is an exchange rate, not a lexicographic order — a strict "I/O first, bytes only among exactly equal load optima" rule would never act, for the same reason §1 gives for refusing a strict migration minimum. It says how much summed utilization deviation one TiB of summed fill deviation is worth (`objective.delta_capacity_spread`, default 0.5 per TiB; `0` disables the term). Raise it to spread data more aggressively: X^* falls, and smaller disks qualify. Lower it, and X^* rises until, at $2\delta \le \gamma$, the term can never pay for a move at all (§11.1 warns). How much data spread is worth moving terabytes for is operator policy. What *is* required is that an empty plan on a capacity-gated group explains itself (§9.5's "no moves made" line, which prints X^*) and that the manual works the break-even through. The term is L1 and stays L1 whatever `objective.spread_metric` is set to: the risk argument cares about every storage's share, not only the fullest one.

Scaling matters: express z_d in TiB and ℓ_d in average in-flight I/O requests (§4) before applying the weights, so the defaults in the example config are meaningful.

5.5 Solver backends

CBC via PuLP — the one MILP backend. Direct transcription of §5.3's constraint set and §5.4's objective; continuous e_s, Z_s, r_s are fine, so there is no scaling discipline at all. Two unit rules still apply, both for conditioning rather than correctness: size-valued quantities are expressed in whole MiB (Z_s, R_s, r_s, z_d, C_s, U^{sext}, $soft_s$, T_v, o_v) and loads in average in-flight I/O requests (§4), so raw bytes (~10^{12}-10^{14}) never sit next to load values (~1-10) in the LP matrix — CBC's simplex does not error on a badly conditioned matrix, it silently returns a numerically poor "optimal" (confirmed on a real corpus bundle, where an unscaled model made CBC's own post-plan spread almost 100× worse than the heuristic's on the same weights).

`solver.mip_gap` must be relative to the objective as written, constant included. PuLP drops an objective's constant term when it passes the model to CBC, and CBC's relative gap is then measured against the shifted value. §5.4's move terms, linearized as $1 - x_{d,\sigma_0(d)}$, make that constant large and negative, so the configured gap silently became a far larger absolute tolerance (as built, found on a committed corpus bundle: a 2 % gap stopped at 0.389 against an optimum of 0.300). Carry the constant on a variable fixed to 1 instead.

Assert after solving that the unscaled objective recomputed in floating point from the returned assignment — `heuristic.evaluate_assignment()`, the one objective implementation every backend reports through

— agrees with what the solve was told it achieved; a cheap guard against a modeling mistake silently producing wrong plans.

A **CP-SAT (`ortools`) backend was specified here once and removed** (REVIEW.md AL-02): not in Debian, excluded from vendoring by this section's own dependency rules, and not something operators install by hand on PVE hosts, so `solver.backend: auto` resolved to CBC on every deployment — the second model builder, its integral-coefficient discipline (per-coefficient constant folding, the `K/W` scales, the γ-quantization trap of REVIEW.md F-14, the int64 magnitude assertions) and its CI/packaging exceptions existed only for a backend production could never run. Those rules are CP-SAT's own and were removed with it; a future integer backend must re-earn F-14's analysis rather than inherit it. The §14 fixture agreement that once read "CP-SAT and CBC agree on every β case" now binds CBC to the exhaustively-enumerated optimum alone.

Heuristic fallback (no dependency, and the path for very large groups).

1. **Seed** with the current assignment (not from scratch — we are minimizing *change*).
2. **Repair**: while any `s` violates (C5), move the disk from `s` that most reduces the violation per byte moved, to the feasible storage with the lowest `u_s`.
3. **Descend**: repeatedly evaluate every single-disk move, every pairwise swap, **and every whole-VM co-relocation** (every movable disk of one multi-disk VM moved to the same target storage together, in one trial); apply the one that most improves the full objective (including β, γ, κ, δ); stop when no move improves it or `heuristic_iterations` is reached. The third candidate family was added after the first two (confirmed live on a real production cluster, not found by review): whenever κ is large enough to make moving one disk of an N-disk VM a net loss on its own (it pays κ's fragmentation penalty before a later move could reunite it), neither a single move nor a swap can ever reach the state where relocating the whole VM together is a clear win — descend found *zero* improving moves at all on a real, 196%-imbalanced cluster without this candidate, which a "the path for very large groups" fallback must not do.
4. **Polish**: attempt to reunite fragmented VMs where doing so does not worsen imbalance beyond `imbalance_threshold`. With step 3's whole-VM co-relocation above, this step's remaining scope is narrower than originally written: only an *N-way rotation* across three or more storages (no disk's own move improving alone, and no two disks sharing a VM) is still outside descend's own reach.

Swaps matter and must not be omitted: when both storages are near their capacity limit, no single move is feasible, and only an exchange of two disks can improve the balance.

The heuristic must use the **same** feasibility and objective functions as the MILP path so the two backends are directly comparable; make them shared pure functions.

---

6. Gating — deciding whether to act at all

Applied in order, before the solver runs. Any gate that fails ends the run with "no action".

Drift gate — the "minimum % of changed average traffic" requirement. Compare the current load vector against the one recorded at the last *executed* balance:

```
‖ℓ_now − ℓ_last‖₁ / ‖ℓ_last‖₁   ≥   gates.drift_threshold      (default 0.10)
```

Using the L1 norm over the whole vector, rather than a per-disk test, means many small correlated changes can legitimately trigger a re-plan while one noisy disk cannot.

Vector alignment. The two vectors are indexed over the **union** of disk keys present in either, with a missing disk contributing load `0` to the vector it is absent from. A disk created since the last balance

therefore contributes its full current load to the numerator, and a deleted disk contributes its full former load — both are genuine changes to the group's I/O profile and should be able to trigger a re-plan on their own.

Degenerate cases, which must be handled explicitly rather than left to produce a division by zero:

Condition	Behaviour
No ℓ_last recorded (first run ever, or state file reset)	**Skip the drift gate**, proceed to the imbalance gate
$\|\ell_last\|_1 = 0$ (group was entirely idle) and $\|\ell_now\|_1 > 0$	Treat as fully drifted, proceed
$\|\ell_last\|_1 = 0$ and $\|\ell_now\|_1 = 0$	No load, no imbalance — exit "no action"

ℓ_last is recorded **only on a run that actually executed at least one migration**, not on every run. Recording it on planning runs would let load creep past the threshold in sub-threshold steps without ever triggering.

Which ℓ. Both ℓ_now and ℓ_last are the *effective* load — the one every consumer reads (§10.1), so forecast-scaled under `forecast.model: holt_winters` — and ℓ_last records whatever produced that run's ℓ. A change of forecast state between two runs (the model switched, the backtest verdict flipped, the factors moved) therefore changes every disk's basis and reads as drift, by design (REVIEW.md AM-05): the picture the balancer places against changed, so reconsidering the placement is right. The drift gate passing only opens the imbalance gate; a plan still has to clear payback (§7.3), so the worst case is a re-plan that finds nothing worth doing. Recording the observed vector instead would compare forecast-scaled ℓ_now against observed ℓ_last — a permanent mismatch — unless the gate were moved onto observed loads too, which would have it ignore exactly the change the forecast exists to anticipate.

Imbalance gate — the "% of IOPS difference over all storages in the group" requirement:

```
(max_s u_s − min_s u_s) / u*   ≥   gates.imbalance_threshold   (default 0.20)
```

Evaluated per group; a group that passes it — or the capacity gate or split gate below — is planned, others are skipped.

Capacity gate — the data-spread counterpart of the imbalance gate, on fill fractions rather than loads:

```
(max_s b_s − min_s b_s) / b̄   ≥   gates.capacity_spread_threshold      (default 0.25)
```

with b_s and $\bar{b}$ as in §5.3 (C7). A group that passes it is planned even when its I/O is perfectly balanced — in that case the δ term of §5.4 is what does the work — and it bypasses the drift and imbalance gates for the same reason the reserve override does: a stable workload is not a reason to keep data concentrated. Unlike a reserve violation this is a preference rather than a safety property: cooldowns, payback (§7) and the transient invariant (§8.1) all still apply, and `gates.capacity_spread_threshold: null` disables the gate — with `objective.delta_capacity_spread: 0`, the whole feature. The gate cannot fire when $\bar{b} = 0$: an empty group has nothing to spread. Passing it opens the group for planning and nothing more. The plan is still empty when the only disks that would even the data out are smaller than §5.4's break-even X^*, which §9.5's "no moves made" line explains.

The gate stays **relative** while the objective's δ term is absolute (§5.3 (C7)), on purpose: they answer different questions. The gate asks whether the shape of the group's data is lopsided enough to plan for at all, which is a question about proportions, and a threshold in TiB would mean something different on every group. The objective asks whether a particular move is worth its cost, which is a question about bytes, priced in the same unit as γ.

Split gate — the counterpart of the capacity gate for §5.3.3's split rule. It fires when some VM $v \in V^{split}$

has a movable disk `d` and an eligible storage `s ≠ σ₀(d)` ((C2), not in cooldown) such that moving `d` alone to `s`

1. strictly lowers `v`'s peak footprint `max_s F_{v,s}`, and
2. leaves `s` without a (C5) shortfall, by the one shared reserve function.

It bypasses the drift and imbalance gates, like the capacity gate and for the same reason: a stable workload is no reason to keep a large VM's snapshot reserve on one storage. It is a preference gate, not a safety one — cooldowns, payback and the transient invariant still apply — and it is off when `snapshot_reserve.split_vm_footprint` is `null` or `objective.mu_vm_split_per_tib` is 0. The single-move test is what keeps it quiet once the job is done: a VM at the best split its storages allow (§5.3.3's `4, 4, 2` on three storages) has no single move that lowers its peak, so the gate does not hold the group open on every run, and a VM whose every disk is pinned never opens it. The test costs one footprint pass per large VM plus `O(|S|)` per (movable disk, target) pair to find the peak after the move, and a reserve evaluation only for the pairs that lower it (REVIEW.md AO-06) — cheap enough to run every time, and a group with no VM above the cap pays nothing. It is a sufficient test, not an exact one: a split that only a *pair* of moves can improve does not open the group by itself — the next run that passes any other gate gets it — and that is accepted for a preference. Passing it opens the group for planning and nothing more; at the default weights the plan may still be empty (§5.3.3's break-even), which §9.5's "no moves made" line explains.

Cooldowns — a disk moved within `cooldown_per_disk` (default 24h) is pinned in place; a storage involved in a migration within `cooldown_per_storage` accepts no new incoming moves.

Reserve override — a storage in violation of (C5) — snapshot reserve or configured free space (§5.3.1), at plan time — bypasses the drift and imbalance gates entirely. Safety is not subject to hysteresis, and neither is the free-space mandate: an admin's new VM on an overfull storage must be reacted to on the very next run, not once the workload has drifted 10%.

---

7. Migration cost and the payback rule

This section implements the requirement that *migrating a very large disk may generate more traffic than it saves*. It is expressed as a **hard acceptance test on the finished plan**, not merely as a soft γ penalty, because a penalty can always be outweighed by a large enough imbalance term.

7.1 Cost

A `move_disk` on a running VM performs a QEMU `drive-mirror`: it reads the whole source volume and writes it to the target, then switches over. For thick provisioning the full provisioned size is transferred. But the mirror is only the first half of the operation — `delete=1` then removes the source volume, and on a storage with `saferemove` enabled that removal is a full-size **zeroing pass**, throttled and often far slower than the mirror it follows:

```
duration_mirror_d = z_d / min(bwlimit, headroom_src, headroom_dst)

duration_wipe_d   = z_d / |saferemove_throughput(σ₀(d))|   if saferemove is enabled there
                  = 0                                       otherwise

duration_d        = duration_mirror_d + duration_wipe_d

cost_d            = duration_mirror_d · (ω_src + ω_dst)  +  duration_wipe_d · ω_wipe
```

`ω_src` and `ω_dst` (default 1.0 each) are the added in-flight I/O on source and target — a mirror is one sequential reader plus one sequential writer, so 1.0 each is the natural unit and is directly comparable to ℓ, which is measured in the same units (§4). `cost_d` is therefore in **load-seconds**. The wipe is charged

to the source only, because it is a sequential write over the old volume with nothing happening on the target; `w_wipe` (`migration.wipe_load_weight`, default 1.0) is its own weight so an operator who knows their array shrugs off a throttled zeroing pass can lower it without touching the mirror weights. **The mirror duration is `z_d / bwlimit`, full stop: `migration.bwlimit_bytes_per_sec` is the only throttle a migration needs, and this tool does not model storage saturation while one runs (§12.1; REVIEW.md AL-04).** An earlier draft's `headroom_src`/ `headroom_dst` terms were never defined and never built.

A disk below `migration.tiny_disk_bytes` costs nothing. `cost_d = 0` when `z_d < tiny_disk_bytes` (default 64 MiB — comfortably above an EFI var store or TPM state, and far below any disk the payback rule was written for), and §7.3's aggregate test counts neither its cost nor needs its benefit. A 528 KiB `efidisk0` mirrors and wipes in seconds; asking it to justify itself against `λ` is a category error — this rule exists for the migration whose own traffic can outweigh its benefit, and no disk this small can generate such traffic. Together with §5.4's `D^big` exemption this is what makes affinity repair of tiny disks possible at all.

Why the wipe term is not a rounding detail. PVE's LVM `saferemove` ("Wipe Removed Volumes" in the UI) defaults to a throughput of **10 MiB/s**. At that rate the 1.5 TiB disk of the §14 example takes about **44 hours** to wipe, against roughly 2.2 hours to mirror it at 200 MiB/s — the cleanup is twenty times the move. A cost model that stops at the mirror understates such a migration by that factor and will happily schedule a plan that occupies the source array for two days.

`saferemove_throughput` is signed, and the sign is not part of the rate. PVE passes the configured value straight through to `cstream -t`, where the sign selects *how* the limit is enforced and the magnitude is the rate in bytes/second (`cstream(1)`, confirmed by the operator against a production cluster):

- **positive** — a session average. cstream accumulates its own error and may exceed the rate for a while to make good on earlier underutilization, so the whole session converges on it.
- **negative** — an upper limit on each individual read/write syscall pair, never exceeded.

So `-1073741824` means 1 GiB/s, not minus anything, and negative values are ordinary in PVE configurations — they are what an operator writes when they want a rate the wipe can never burst above. Hence the `| … |` in the formula. `z_d / |throughput|` is the right estimate under either sign; under the negative one it is additionally a hard floor, since the wipe cannot finish ahead of a rate that is never exceeded.

Dividing by the signed value is not a cosmetic error. `duration_wipe_d` would come out **negative**, `duration_d = duration_mirror_d + duration_wipe_d` would collapse towards zero — to *exactly* zero on the common configuration where the operator sets `|saferemove_throughput|` equal to `migration.bwlimit_bytes_per_sec` — and §7.3's `max_single_move_duration` rejection, along with `verify-storages`' cooldown-versus-wipe warning, would then be unable to fire for a disk of any size. This is not hypothetical: it was found by replaying a real bundle from a cluster whose three LVM storages all carry `saferemove_throughput -1073741824` against a `bwlimit_bytes_per_sec` of `1073741824`, where a hypothetical 100 TiB move reported a total duration of 0 s and passed a 6 h limit.

Read `saferemove` and `saferemove_throughput` from `GET /storage` (§3.5 — the list form, not `GET /storage/{id}`) per storage; never assume, and never assume a sign either. `migration.account_saferemove_wipe: false` disables the term for an operator who has verified their storages do not wipe, but the default is to account for it. Note the knock-on effects, all covered in §9.3: the wipe also determines when the source's space is actually released, and it holds a storage-level lock while it runs.

7.2 Benefit

The plan improves the four *persistent* parts of the §5.4 objective — load balance, data spread, VM affinity and large-VM splitting — from `(E_before, F_before, A_before, O_before)` to `(E_after, F_after, A_after, O_after)`, where `E = Σ_s e_s`, `F = Σ_s d_s` (§5.3 (C7)), `A = Σ_{v∈V} w_v · ( Σ_{s∈S} y_{v,s}`

– 1) is the objective's affinity debt, weights included — the same sum the κ term charges, from the one shared implementation (AGENTS.md §5) — and O = Σ_{v∈V^split} o_v the summed split excess of §5.3.3, in TiB. All four improvements persist for as long as the workloads keep running on the new placement, and we account them over the payback horizon H:

```
benefit  =  ( α·(E_before − E_after)  +  δ·(F_before − F_after)  +  κ·(A_before − A_after)
                                      +  μ·(O_before − O_after) )  ·  H
```

also in load-seconds — δ converts a TiB of fill deviation, κ·w_v a busy VM's fragmentation and μ a TiB of split excess into load-deviation equivalents (§5.4), so both sides of §7.3's comparison stay in the unit that is the whole point of using I/O time as the load metric. β and γ charge the move itself and belong on the cost side.

H defaults to 365d (migration.payback_horizon), with λ = 10. The reasoning is the asymmetry of the two sides: a migration's cost is paid once and early — days of mirror I/O (§7.1), plus a source wipe that can outlast the mirror by an order of magnitude — while its benefit accrues for as long as the placement lasts, which for infrastructure is months to years. Assume, by default, that a VM balanced today keeps running where it is for at least another year. A short horizon asks "does this pay back before the bruise heals?", and at the previous 7d default the answer was *no* for every move worth less than λ·cost/H — 0.17 of summed deviation per TiB moved, 0.69 for the 4 TiB disk of §14.5 — rejecting, at the same λ, precisely the slow-accruing but real benefits a balancer exists to capture: a week of degraded I/O on the storages looks bad, and still pays for itself many times over across a year of better balance.

Three things H is **not**, to keep the semantics honest:

- **Not a prediction that nothing changes for a year.** Drift and operator action will re-plan long before that (§6), and a re-plan does not claw back benefit already accrued. H is the accounting period over which a recurring benefit pays for a one-time cost — the convention capacity planning has always used for exactly this shape of decision.
- **Not the brake on doing too much.** That role belongs to β/γ and the gates. With a one-year horizon the aggregate test's remaining job is to reject plans whose persistent-objective improvement is genuinely negligible — ΔObj < λ·cost/H, about 0.008 for the two-move plan of §14.5 — which is the right shape for a safety test: a guard against absurdity, not the main cost control.
- **Not universal.** It is an assumption about VM lifetime. Short-lived fleets (CI, render farms, lab clusters) should lower it toward the actual lifetime of their VMs; §11.1 warns below 30d, where the test starts rejecting real benefits again.

κ belongs in this sum, and its earlier exclusion was a defect. The original wording here read "κ is a preference, not a physical benefit, and appears in neither" — and dogfooding broke on exactly that sentence. A plan whose four moves were *all* affinity repairs (two of them 528 KiB EFI disks) scored benefit = 9 load·s against cost = 232 load·s — ratio 0.0388, reject — and the tool refused the one action the operator most wanted. No oversized balance move was involved; the rule rejected a plan it had never been given a way to value. Co-location persists exactly as long as balance does, and its worth is measurable — the fragmented VM's own I/O, which is what w_v weighs. Two properties of the corrected term, stated so they are not quietly reverted:

- **ΔA may be negative**, and then it *reduces* the benefit: a balance move that splits a VM must pay for the fragmentation out of its α gain. The §14 fixture's two-move plan carries both signs at once (§14.5).
- **At H = 365d any nonzero κ·ΔA dwarfs any achievable cost** (a 4 TiB mirror costs ~4×10^4 load·s; one reunited average-load VM is worth 0.5 × 31 536 000). For affinity-motivated plans the aggregate test therefore defers to the solver's own κ-versus-β/γ pricing — a genuine cost/benefit test in

objective units, not a rubber stamp — and the test's remaining job stays the one the second bullet above gives it: rejecting plans whose *physical* benefit is negligible.

$\mu \cdot \Delta 0$ behaves exactly like $\kappa \cdot \Delta A$ in both respects: it may be negative (a balance move that gathers a large VM onto one storage pays for it), and at `H = 365d` it outweighs any mirror cost, so a split the solver chose is not vetoed here (§5.3.3).

7.3 Acceptance

```
accept plan   ⟺   benefit  ≥  migration.payback_ratio · Σ_d cost_d
```

with `H = 365d` and `λ = 10` by default. The sums — and the test itself — cover only disks with `z_d ≥ migration.tiny_disk_bytes`: a smaller disk's move carries `cost_d = 0` (§7.1) and needs no verdict, while every hard rule below applies to it exactly as to any other move. Additional **hard** rules, applied per move, that reject individual migrations regardless of the aggregate test:

- `duration_d > migration.max_single_move_duration` (default 6h) → reject the move;
- the move violates the transient reserve invariant of section 8 → reject.

A plan that repairs is exempt from the aggregate test. The trigger is the plan's *outcome*: its final assignment's `Σ r_s` is strictly below the current assignment's — the plan leaves the group with less reserve/free-space shortfall than it found, and no mirror cost may veto that (the Storage-DRS mandate). The "final assignment" is the **scheduled** one — the state the ordered moves actually reach, `schedule_result.final_assignment`, the same R-02 distinction the payback benefit already draws (`cli.py` evaluates its `final_breakdown` against it, not against the solver's aspirational target): a partially deadlocked plan is scored on what it will really run, and a plan whose *target* repairs but whose schedule never gets there is not exempt. **"What it will really run" means after the hard per-move rules below have taken their moves out, not merely after ordering.** The one rule that fires at this gate rather than during scheduling — the `max_single_move_duration` rejection — drops moves from what is executed but not from `schedule_result.order`, so a repair move it refuses would otherwise sit in the order, satisfy the trigger, and buy a **plan-level exemption for the balance moves that survive it** — the economic test skipped on a plan that no longer repairs, and whose shortfall §9.5 then reports as unmet. Take both sums over the move set the gate will actually execute (the order minus the refused moves), which is well-defined because the refusal does not depend on the exemption: it is a per-move verdict on `duration_d` alone, computed before the aggregate test is consulted. §8.1's transient invariant needs no such treatment — `order_moves()` enforces it while building the order, so a breaching move never reaches it in the first place. The outcome trigger, not a per-move flag, is what makes the mandate total: a plan of **redundant repairs** — two moves from a violating storage where *either one alone* repairs it — contains no move that passes the revert test below, yet the plan still repairs and is still exempt; a per-move trigger would let payback veto exactly that plan and leave the violation standing. Which moves carried the repair is reported per move by the **revert test**: a move is marked `repair: true` iff holding that one disk on its current storage would strictly raise the plan's final `Σ r_s` — its source ends the plan below its snapshot reserve or its configured free-space requirement (§5.3.1), or the move empties the destination another repair needs (§5.3, §14.8). The test is evaluated by re-scoring `Σ r_s` on the plan's final assignment — the same scheduled assignment the trigger scores — with that one $x_{d,\sigma_0(d)}$ held — no re-solve. Repairs are not priced: the requirement is the operator's configured policy. The exemption is **plan-level**: one repairing plan skips the economic test for the whole plan, and every move in it carries its `repair: true/false` marker in the plan output so the operator can see which move carried the repair. (The marker and the trigger are allowed to disagree, in either direction — the redundant-repair plan above repairs with no marked move, and the converse exists too: a move can be load-bearing on a plan that does not repair — holding it back raises the plan's *final* `Σ r_s` above zero while the current assignment was already at zero, e.g. one half of a swap whose other half lands a new largest VM footprint and needs the room this move frees. Such a move is marked `repair:`

`true` — and, once §8.2's exception 2 is implemented (it is spec-only today, as §14.8 records), scheduled first, because it frees space a later move needs — but its plan is *not* exempt — the plan leaves the group no better than it found it, and a plan that does not repair has nothing the mandate protects.) This is deliberately narrower in its trigger than the built `payback.py` already implements (`has_reserve_override` fires when *any* move's source was violating at scheduling time — a plan that moves a disk off a violating storage but ends no less short is exempt today and will not be under the outcome trigger), and the free-space work replaces that detection with the outcome test plus the revert-test markers. The breadth cuts both ways, and the wide side is a deliberate cost too: the trigger is one byte of shortfall reduction, so a plan that repairs by a byte while carrying expensive balance moves is exempt **in full** — the economic test is skipped for the whole plan, not prorated, and on a permanently-short cluster (§5.3.1's thrash paragraph) every shortfall-reducing plan is fully payback-exempt for as long as the shortfall stands. That is accepted for the same reason the narrower side is: the mandate does not grade repairs by size — a partial repair is still the operator's configured requirement being met — and there is no principled way to price only part of a repairing plan, because the benefit is plan-level (§7.2's before→after on the whole assignment), so "price the balance moves but not the repair" would mean inventing a per-move benefit that does not exist. What bounds it: the three hard rules above still apply to every move in an exempt plan, the plan output carries every move's cost and its `repair` marker so the operator sees exactly what the exemption bought, and on a healthy cluster the trigger is rare — rarer than the condition §6's override already singles out, in fact: the outcome trigger is a **strict subset** of both. `Σ r_s` can only fall if some storage's `used_s` or `Z_s` falls, which requires a disk to *leave* that storage — and a storage whose shortfall a move reduces was in violation when the move left it, so every outcome-exempt plan is also exempt under today's `has_reserve_override` and §6's override was already open for it. The change can only *remove* exemptions, never create one, which is what makes "deliberately narrower" a provable claim rather than a comparison. A repair that cannot finish inside `max_single_move_duration`, or that breaches the transient invariant, is rejected like any other move and the shortfall reported as unfixable.

The shipped `fc-tier1` fixture was the redundant-repair case under the per-disk reserve, and is not under the per-VM footprint (§5.3.3). Under the per-disk rule san-a started `r = 0.5` (4.5 used + 2 × 2.0 largest disk > 8.0 TiB), and either move alone repaired it, so both were marked `repair: false`. Under the footprint rule san-a holds both of VM 101's disks, `Z_a = 2.0 + 1.0 = 3.0`, and starts `r = 2.5` (`4.5 + 2 × 3.0 = 10.5 > 8.0`). The §14 two-move plan still ends at `Σ r_s = 0` and is still exempt by the outcome trigger — as the built `has_reserve_override` is (`102:scsi0`'s source is violating when it is scheduled). The revert test now separates the two moves: holding `102:scsi0` back leaves san-a with `101:scsi0` and `102:scsi0`, `3.5 + 2 × 2.0 = 7.5 ≤ 8`, still compliant, so it stays `repair: false`; holding `101:scsi1` back leaves VM 101 whole on san-a, `3.0 + 2 × 3.0 = 9.0 > 8`, short by 1.0, so **`101:scsi1` is `repair: true`** — it is the move that splits VM 101, and splitting VM 101 is what the repair needs. The recorded payback shortfall pair becomes `reserve_shortfall_tib_before/_after` 2.5 → 0.0; every recorded payback *number* is unchanged (the fixture keeps the split rule off, §14.1), and the expected file is regenerated, never hand-edited.

The redundant-repair case therefore lost its fixture, and needs a new one. Nothing else shipped exercises a plan that repairs with no marked move. Phase 17 (§12) adds it as a unit case: two single-disk VMs on one violating storage, each of whose moves alone repairs it — both moves `repair: false`, the plan `repair_exempt: true`.

As built (AH-02): `fc-tier1.expected.json` never recorded the old flag, but it does record the new surface — per-move `disk_key`, the plan's `aggregate_ok`, the `reserve_shortfall_tib_before/ _after` pair, `repair_exempt: true` (the outcome trigger fires; the plan would have cleared the aggregate test regardless) and `repair_markers` — recorded under the per-disk reserve as 0.5 → 0.0 with both markers `false`, and to be regenerated by phase 17 as 2.5 → 0.0 with `101:scsi1: true`; the corpus expected files

record the same fields.

A plan of nothing but tiny disks has no economic gate at all, and that is deliberate — but it means the objective is the only thing holding it. Σ_d `cost_d` is then 0, so — outside the repair exemption above, which skips the test anyway — $\text{benefit} \geq \lambda \cdot 0$ reduces to $\text{benefit} \geq 0$ and any non-negative benefit passes; the implementation reports the ratio as `+inf`. That is the intended reading of "needs no verdict" above, and it is the whole point of §7.1's exemption: a 528 KiB `efidisk0` rejoining its VM must not have to out-earn a rule written for multi-terabyte migrations.

State the consequence plainly, because it was not obvious and it cost a real dogfooding cycle to find. For such a plan, nothing downstream of the §5.4 objective asks whether the moves are worth making. If the objective scores a move at $+\varepsilon$ for any $\varepsilon > 0$, the move happens — and ε can be arbitrarily small, because no term in the objective has a materiality floor either. So the *correctness of the objective's affinity term is load-bearing for tiny moves in a way it is not for any other kind of move*, and a defect in it surfaces directly as migrations that buy nothing.

Exactly that happened. With §5.3 (C3)'s affinity term excluding pinned disks — the pre-fix default, see §3.6 — two 528 KiB `efidisk0` moves on a real cluster scored a κ gain of precisely zero and were emitted anyway on a 3.6×10^{-7} capacity-spread difference, a relative improvement of about 6×10^{-8}, on which the three then-existing solver backends did not even agree (CP-SAT emitted two moves, CBC none; the CP-SAT backend has since been removed — AL-02). Each was a live migration with a VM lock, a PVE task and a `saferemove` wipe, and each burned `gates.cooldown_per_storage` on its target. Correcting (C3)'s default turned the same two moves into genuine reunifications worth a discrete `1.0` of objective, and the three backends into agreement.

No materiality floor is specified, and none should be added speculatively: any threshold would be a magic number standing in for a decision this model does not otherwise need to make, and the observed failure was a defect in the objective rather than a missing gate. But the branch is a real one, and a future bundle showing tiny moves emitted with $A_{before} = A_{after}$ is evidence to revisit it — the narrowest available fix being to require $A_{before} > A_{after}$ for a zero-cost plan, which needs no threshold because the affinity debt moves in discrete steps.

Migrations are throttled by `bwlimit`, and by nothing else. A migration may run at any time. The tool does not model storage saturation — no per-storage queue depth, no forecast of the load a mirror would meet, no deferral. `migration.bwlimit_bytes_per_sec` is passed to every `move_disk` call and is the whole throttle; `max_single_move_duration`, the transient reserve invariant of §8.1 and `execution.cooldown_per_storage` (sized against the wipe time, §9.3) are the only per-move and per-storage limits. Earlier revisions carried a best-effort `saturation_load`/`saturation_ceiling` guard here; it was inactive unless an operator set a number that has no safe default, could only defer moves, and was removed by phase 14a (§12.1, REVIEW.md AL-04). Both config keys were deleted outright: the schema is closed, so a config that still sets them fails validation.

The `draining` state below still matters, to capacity (§8.1) and ordering (§8.2): a move stays in `M` until its source volume is gone.

If the plan fails the aggregate test, re-solve with β and γ doubled and retry, up to three times. This naturally converges on the smaller subset of high-value moves rather than abandoning the run — usually the one or two disks with the highest ℓ_d / z_d ratio, which is exactly the right thing to move.

As built: the three changes this section and §7.2 specify — the w_v weighting of κ, the `tiny_disk_bytes` exemptions, κ's place in the benefit — are implemented: `payback.py`'s `compute_benefit_load_seconds()` takes the three-term formula, both solver backends fold a per-vmid $\kappa \cdot w_v$ coefficient (`heuristic.compute_vm_weights()`, one implementation shared by both MILP backends and the heur-

istic), and the four-affinity-repair plan that previously scored `benefit 9 load·s vs cost 232 load·s → ratio 0.0388 ✗` and refused to act now prices the reunions correctly (see §14.7's `affinity-repair.yaml`, the fixture built to prove exactly this). The β/γ-doubling re-solve above remains unimplemented — a separate, still-open scope decision (`payback.py`'s own module docstring lists it under "deliberately not implemented in this pass"), not part of this fix.

`ℓ_d / z_d` — load per byte — is worth surfacing in `pve-storage-drs explain` output. It is the single best indicator of a good migration candidate: high I/O concentrated in a small disk.

---

8. Migration ordering

The solver produces a *target assignment*. It says nothing about the order of moves, and order matters: the requirement that the snapshot reserve holds **even during storage migrations** is a constraint on every intermediate state, not just the endpoints.

8.1 The transient invariant

During a `move_disk` of disk `d` from `a` to `b`, the volume exists on **both** storages — the mirror target is fully allocated before the switchover, and the source is only removed afterwards by `delete=1`. So while a single move is in flight, `b` must satisfy:

```
used_b + z_d + f_b · max(Z_b, F_{v(d),b} + z_d)   ≤   C_b
```

`Z_b` is the largest per-VM footprint on `b` and `F_{v(d),b}` the footprint `d`'s own VM already has there (§5.3.3), both in the current state. Note the `max(Z_b, F_{v(d),b} + z_d)`: if the incoming disk makes its VM's footprint the new largest on `b`, the required reserve grows at the same moment the disk arrives — and with the per-VM footprint that happens not only when the disk is large but whenever it joins enough of its own VM's other disks. (For a move that converts the disk's format onto an `enforce_format` storage, `z_d` here and below reads `z_{d,b}`, the converted size — §5.3.2.) This is the case most likely to be missed, and the one most likely to fill a SAN LUN.

The disk counts toward its VM's footprint on **both** storages while it is in flight: on `b` from the start of the mirror, on the source until its volume is observed gone (§8.2's `draining`). Counting it twice is conservative, which is the direction a transient check must err in.

The transient free-space floor. The snapshot term above is not the only thing `b` must keep: a storage with a configured free-space requirement (§5.3.1) must not dip below its **hard** floor while the move is in flight. The full single-move predicate is therefore:

```
used_b + z_d + max( f_b · max(Z_b, F_{v(d),b} + z_d) , hard_b )   ≤   C_b
```

`hard_b ≤ soft_b` always (§5.3.1 validation), so the *floor component* of this predicate is a relaxation of the endpoint constraint (C5) — the endpoint demands `max(f_b·Z_b, soft_b)` free, the transient state demands only `max(f_b·max(Z_b, F_{v(d),b}+z_d), hard_b)` of floor. The rest of the predicate is **stronger** than (C5), not weaker, and must not be read as implied by it: the snapshot term grows to `f_b·max(Z_b, F_{v(d),b}+z_d)` the moment the incoming disk makes the new largest footprint, and the source is still charged in full — the two facts that make §8.1 a separate invariant rather than a corollary. With the default `hard: null` (i.e. `hard_b = soft_b`) the floor component does not relax at all and the transient check is exactly as strong as before §5.3.1; an operator who sets `hard` strictly below `soft` buys the scheduler room to land a disk on a storage that is *heading toward* its soft requirement — the target's fill rises transiently past `soft_b` and the plan's final state pulls it back down — without ever crossing the hard floor. The dip is bounded and planned, not discovered: the solver only ever emits moves whose *endpoints* satisfy (C5), so a transient dip below `soft` can occur solely on a storage the finished plan leaves compliant — or, when the

plan ends with `r_s > 0` (§5.3 permits it: physically unachievable), on a storage whose shortfall the plan already minimizes and reports.

The source `a` gets no relief until the move completes, so a plan that depends on freeing space on `a` to make room on `a` is simply infeasible and must be ordered around.

Generalized to concurrent moves. `execution.max_concurrent_migrations` may exceed `1`, and then the single-move form above is **not sufficient**: several disks can be landing on `b` at once, and none of their sources release space until each completes. For an in-flight set `M`, every storage `b` must satisfy:

```
used_b  +  Σ_{m∈M : dst(m)=b} z_{disk(m)}
       +  max( f_b · max( Z_b , max_v ( F_{v,b} + Σ_{m∈M : dst(m)=b, v(disk(m))=v} z_{disk(m)} ) ) ,
              hard_b )   ≤   C_b
```

Both sums and the inner `max` are over the same in-flight set — two disks of one VM landing on `b` together raise that VM's footprint by both — and the `max(…, hard_b)` floor of the single-move form carries over unchanged. Implement this as the single feasibility predicate and call it with `M = {m}` for the sequential case, so there is only one version of this rule in the codebase.

`concurrency_ok(state, m)` is then defined as: adding `m` to the current in-flight set

1. keeps the generalized invariant above satisfied on **every** storage;
2. keeps `|M| ≤ max_concurrent_migrations`;
3. keeps the count of in-flight moves touching any single storage — **as either source or target** — at or below `max_concurrent_per_storage`;
4. violates no per-disk or per-storage cooldown.

With the default `max_concurrent_per_storage: 1`, two moves targeting the same storage serialize automatically and the generalized form collapses to the single-move form. That is the recommended configuration, and raising it should be a deliberate act on a storage with ample headroom.

8.2 Scheduling algorithm

```
pending ← { moves implied by the target assignment }
state   ← current allocation
order   ← []

while pending:
    feasible ← { m ∈ pending : transient_invariant_ok(state, m)
                               and concurrency_ok(state, m)
                               and not in cooldown }
    if feasible is empty:
        if a staging move exists:  order.append(staging_move); continue
        else: report deadlock with the blocking storages; break

    m ← argmax over feasible of  (persistent-objective reduction) / cost_m
                                 # the α, δ, κ·w and μ terms of §5.4 — the parts whose
                                 # improvement persists; β/γ are one-time costs.
                                 # cost_m = 0 (a tiny disk, §7.1) ranks first:
                                 # free value, delivered before anything pays
    order.append(m)
    state ← apply(state, m)          # target charged immediately; the source is charged
                                     # until its volume is observed gone (see below)
```

Ordering by **objective reduction per unit cost** means the plan front-loads its value: if the operator aborts halfway, or a maintenance window closes, the moves that mattered most have already run. Two exceptions take priority and are scheduled first regardless of ratio:

1. moves that resolve a storage currently violating (C5);
2. moves that *free* space on a storage which some later move needs.

Small disks go first, ahead of the larger disks of their own VM, on purpose. A disk below `tiny_disk_bytes` is mirrored and — with `saferemove` — wiped in seconds, so scheduling it first never makes the next move wait: a large disk's source may spend many minutes in *draining* (below) while its zeroing pass runs, holding that storage's concurrency slot, and a small disk queued behind it would wait for that wipe too. The price is an intermediate state in which a small disk already sits on the target while its VM's larger disks have not moved yet; if the run stops there, the VM is left split until the next run. That is accepted: (C8) constrains the plan's *endpoint*, not the order, and the next run moves the larger disks or — if the plan changed — lets κ bring the small one back for free. Do not reorder small disks after their VM's larger disks.

The same acceptance covers the one case that is not an ordering intermediate: when §7.3's hard per-move duration rule refuses a larger disk's move, the small disk's own move (seconds of mirror, never the one a duration limit refuses) still runs, and the executed endpoint leaves the small disk on a storage its VM's larger disk never reaches. The executed endpoint is what payback scores (`κ·ΔA` prices the new split, so a plan whose benefit does not cover it is vetoed), and the next run's κ brings the small disk back for free; no filter re-checks (C8) on the executed assignment, by choice — it constrains the solver's endpoint, and a split of a sub-`tiny_disk_bytes` disk is a balance-quality cost, never a reserve or safety one.

The source is not freed when the task succeeds. `apply(state, m)` must not optimistically credit the source with `z_d` bytes back. With `saferemove` on the source storage the old volume still exists — fully allocated — for the whole duration of the zeroing pass (§7.1), which can be far longer than the move itself. A move therefore leaves the in-flight set `M` in two stages:

```
mirroring  →  draining  →  done

mirroring:  target charged z_d, source still charged z_d   (the drive-mirror window of §8.1)
draining:   target charged z_d, source still charged z_d   (the move_disk task has reported OK,
                                                            but the source volume is still there)
done:       source volume absent from /storage/{a}/content
```

For the generalized transient invariant of §8.1 the *source-side* accounting of `mirroring` and `draining` is identical, so keep a move in `M` until it reaches `done` and the invariant needs no change at all. What does change is that ordering rule 2 — "moves that free space a later move needs" — cannot be satisfied within a run when the source wipes slowly. The scheduler must therefore treat a predicted free-space release as **unrealised until observed**, and a plan whose feasibility depends on one is split rather than executed on faith (§8.3, option 2).

8.3 Deadlock and staging

Three storages each near capacity can produce a cycle in which no move is individually feasible though the target assignment is perfectly valid — the classic pebble-motion deadlock. Resolution, in order of preference:

1. **Staging move** — find any storage in the group with enough slack to temporarily accept the smallest disk in the cycle, move it there, complete the cycle, then move it to its target. Costs one extra migration; permit it only if the *whole* plan still passes the payback test with that extra cost included.
2. **Split the plan** — execute the feasible subset now, re-plan on the next run. Often the workload has changed anyway.
3. **Report** — emit the blocking set and stop. Never force a move that breaches the reserve.

Guard staging moves against loops: a disk may be staged at most once per plan, and a staged disk is exempt from the per-disk cooldown for the duration of that plan only.

9. Execution

9.1 Modes

Mode	Behaviour
`dry-run` *(default)*	Print the plan, the before/after balance, the payback arithmetic, and the exact API calls that *would* be issued. Change nothing.
`confirm`	Execute the plan one move at a time, prompting before each. Re-validate the transient invariant immediately before each prompt, since the cluster may have changed. Offer `[y]es / [n]o skip / [a]ll remaining / [q]uit`.
`auto`	Execute unattended, subject to concurrency caps and `execution.time_windows`.

`auto` must additionally: refuse to start a move that cannot finish inside the remaining time window; stop cleanly at window close rather than aborting an in-flight move; and honour `max_migrations_per_run`.

9.2 Performing one move

```
POST /nodes/{node}/qemu/{vmid}/move_disk
     disk={device} storage={target} delete=1 bwlimit={KiB/s} [format={φ(d,target)}]
  → UPID
poll GET /nodes/{node}/tasks/{upid}/status every execution.poll_interval_seconds
  until status == "stopped"; task success ⟺ exitstatus == "OK"
```

Task success is **not** the completion criterion — see §9.3. The move is done only once the source volume has actually disappeared and the VM's lock has cleared.

`bwlimit` is **KiB/s** in the API, while `migration.bwlimit_bytes_per_sec` is bytes/s; convert at the call site and nowhere else.

With `max_concurrent_migrations > 1` this loop launches up to the cap (subject to `concurrency_ok`, §8.1) and polls **all** in-flight UPIDs each cycle, starting the next queued move as each slot frees. With the default cap of 1 it degenerates to the sequential form above.

Before **every** move, re-read the live state rather than trusting the plan:

1. re-fetch `/nodes/{node}/qemu/{vmid}/config` and confirm the disk is still on the expected source — the VM may have been touched by an operator, or live-migrated to another node by the PVE 9.2 Dynamic Load Balancer (§1), changing `{node}`;
2. re-read the target storage live and re-check the transient invariant (§8.1) against *actual* current usage — in **provisioned** terms, like every other check (§5.1). `used` from `/nodes/{node}/storage/{target}/status` is the pool's *allocated* figure and is never an input: on a thin pool (Ceph RBD, LVM-thin, ZFS) it is far lower than what is provisioned, and a re-check built on it could only ever confirm the plan, never catch a pool that other provisioning has filled *in provisioned terms* since planning. The live `used_b` is instead `Σ size(vol) : vol ∈ GET /nodes/{node}/storage/{target}/content` — the same quantity `schedule.transient_invariant_ok()` sums from the model ($\Sigma\ z_d + U^{ext}$), read fresh — while `C_b` still comes from `/status`'s `total`, since the LUN may have been resized. The third model input, `Z_b`, is **not** re-read: it stays the planning-time largest per-VM footprint of managed disks on the target (§5.3.3), raised only by this run's own completed moves, because a listing entry cannot say whether a volume belongs to a managed disk (§5.3 (C4) defines `Z_b` over managed disks). The footprint the predicate

charges for the moving disk's own VM, `F_{v(d),b}`, is likewise the model's. A managed disk that something else lands on the target after planning is therefore counted in full in `used_b` but does not raise `Z_b`, so only the reserve multiplier's growth on the new largest footprint, `f_b·(Z_live − Z_model)`, goes uncounted — never the disk's own bytes (AJ-03). A listing entry without `size` counts at its `approximate-size`; one with neither makes the figure unknowable, and the move is refused. Any failure to read either endpoint refuses the move too: the check never passes on a partial figure. Those two refusals are *errors*, not mismatches — see "Errors are not mismatches" below: they fail the run rather than re-plan. A target that is simply too full for the move is the ordinary mismatch, and is `replan_needed`.

No double counting under concurrency. With `max_concurrent_migrations > 1` the invariant sums a charge `z_m` for every move of this run already in flight onto the same target (§8.1), so those moves' own mirror targets must not also be counted from the listing — a new RBD image is listed at its full provisioned size the moment `move_disk` allocates it. The executor therefore records, when it launches a move, which volumes the target's listing held at that instant, and leaves out of the sum at most one volume per in-flight move: one that is *not* in that launch-time listing, belongs to the *same VM*, and has one of the two sizes `move_disk` allocates it at. As the operator describes PVE's behaviour (not read from PVE's source): between storages of the same thin kind the target is the same size as the source image (the size in its content listing); between different storage types, or from thin to thick, the target is allocated at the disk line's `size=` in the VM config. Either figure can differ from the other (a volume resized outside PVE, a storage that rounds sizes), so the pre-flight's parsed `size=` is kept alongside the listed size and a volume of either size matches. If PVE ever allocates a target at some third size, nothing matches, the target stays counted as well as charged, and the check is merely stricter than it needs to be. The same two sizes decide what the move itself is charged as `z_m`: the disk's listed size, except when the move changes the kind of volume made — between different storage types (`storage.type` differs), or a qcow2 disk landing on a target that cannot hold qcow2 so PVE writes it raw — where the target is allocated at the config's `size=` and the larger of the two is charged. A same-kind move keeps the listed size — and a move onto a storage whose `enforce_format` differs from the source format (§5.3.2), the one case in which the tool passes `format=`. Without enforcement (C2) keeps a qcow2 disk off a storage that cannot hold it, so the implicit-conversion branch stays a guard; the plan's own §8.1 check uses `z_{d,b}`, which is the listed size for every non-converting move. Nothing else is ever excluded — a foreign volume that appeared since, or a leftover of the same VM that was already there, still counts — and where nothing matches (the window between `move_disk` returning and the allocation) the move is charged by its `z_m` alone. Wrongly keeping a volume only makes the check stricter; wrongly dropping one would weaken it, so the match is deliberately narrow. The sequential executor has no in-flight set and excludes nothing;

3. confirm the VM is still running and untagged for exclusion;
4. confirm `config.lock` is empty — if not, wait per §9.3 rather than failing;
5. confirm no snapshot has appeared for the VM since planning (§3.7); if one has, drop the move and re-plan — `delete=1` would be rejected by PVE anyway;
6. re-fetch `/nodes/{node}/qemu/{vmid}/pending` and confirm this disk still carries no unapplied pending change (§3.8); if one has appeared since planning — an operator can queue one in the PVE UI at any time — drop the move and re-plan rather than risk `move_disk` leaving that entry referring to pre-move state.

`format=` is sent **iff** `φ(d, target) ≠ fmt_d` (§5.3.2), with `fmt_d` taken from the live volume step 1 re-read, not from the plan: a disk whose format no longer matches what the plan assumed is a mismatch, dropped

and re-planned like any other step-1 discrepancy, because its `z_{d,s}` and its eligibility were computed for the old format. A move that does not convert sends exactly the request it sends today.

These re-reads bypass the per-run topology cache (§3.5) for this VM and this storage only. Steps 4 and 5 are cheap: both come from the same `/qemu/{vmid}/config` response as step 1. Step 6 is its own call, since only `/pending` (not `/config`) distinguishes a key's pending value from the one in effect (§3.8).

Re-plan protocol. A mismatch is a *normal* outcome in a live cluster, not an error, and must not be allowed to loop:

1. Abandon the remaining moves in the current plan. Do not attempt to patch it — the state it was computed against no longer holds.
2. Record the moves already completed in `state.json` (including their cooldown timestamps) so the next pass sees them as history rather than re-deriving them.
3. Re-invoke the **whole** pipeline from the new observed state: gates, load model, solver, payback, ordering. The gates may well conclude no further action is needed, which is a correct outcome.
4. Cap re-plans at `execution.max_replans_per_run` (default 3). On exceeding it, stop and report — a cluster churning faster than the engine can plan is a condition for a human to look at, not to iterate against. This is a *bail-out*, not a failure: the run exits `0`, the report and the `run_summary` say why it stopped, and the next run starts from freshly observed state. External changes — a VM moved, a snapshot or a new disk appearing, another tool filling the target — are exactly what this loop is for, so a run is only given up on when they keep breaking every plan it makes.
5. In `auto` mode, a re-plan inherits the remaining time window; if too little remains for the cheapest queued move, stop cleanly rather than starting one that cannot finish.

Errors are not mismatches. A mismatch is the cluster differing from the plan, and a re-plan can cure it. An *error* is the tool failing to observe the cluster at all, and re-planning would only run into the same wall — a plan built from data that could not be read is not a better plan. Errors therefore **fail the whole run**: it stops at once (no later group is planned or executed), exits `1`, and reports what it could not read; moves already completed stay recorded in `state.json` and in the report. The errors are:

- the PVE API failing while the executor re-reads the VM's config or the target storage's status or content before a move, or the target listing a volume whose size cannot be established (steps 1 and 2 of the pre-move re-reads above) — the move outcome is `failed`, marked `abort_run`, never `replan_needed`;
- a metrics (Prometheus) error while computing a group's load, whether in the first plan or in a re-plan (re-plan protocol step 3) — a re-plan whose load model is unavailable is not "the gates concluded no action is needed"; it is a failed run.

`plan` and `explain` are read-only and continue through the remaining groups so the report is as complete as possible, but they too exit `1` when any group's load could not be computed. Every other PVE API error raised during a run already propagates and exits `1`.

9.3 Locks, and why a completed move is not a finished move

Two distinct mechanisms can make a VM untouchable, and the engine must handle both. Neither is an error condition — both are ordinary states in a working cluster — so the response to both is to **wait**, not to fail.

1. The VM config lock. PVE writes a `lock:` line into the VM config for any operation that must not be interrupted. Its documented values are `backup`, `clone`, `create`, `migrate`, `rollback`, `snapshot`, `snapshot-delete`, `suspending` and `suspended`. Any of them makes `move_disk` fail with *"VM is locked"*.

```
before issuing move_disk for VM v:
    lock ← config(v).lock                    # /qemu/{vmid}/config, or /status/current
    while lock is set:
        if waited > execution.locks.wait_timeout:  → apply execution.locks.on_timeout
        sleep execution.locks.poll_interval
        re-read lock
```

Treat the value set as **open-ended**. Never whitelist "harmless" locks and proceed anyway: a future PVE version may add a value, and the failure mode of guessing wrong is a half-completed operation on someone else's backup. Any non-empty `lock` means wait. Log which lock was seen and for how long — a VM stuck in `backup` for six hours is something the operator wants to know about, and it is the main reason the wait timeout is measured in hours rather than minutes.

The pre-flight check is also a *planning*-time input: a VM locked when the run starts is pinned by (C2) for that run, so the solver does not build a plan around a disk it cannot touch. The check here is the second line of defence, because a lock can appear between planning and execution.

2. The post-move wipe — the one that is easy to miss. When `move_disk delete=1` completes and the task reports `exitstatus: OK`, the migration is *not* over on a storage with `saferemove` enabled. PVE then zeroes the old volume at `saferemove_throughput` (default **10 MiB/s**, §7.1), which for a large disk runs for hours or days. During that window:

- the source volume still exists and its space is **not** reclaimed — §8.2's `draining` state;
- the operation holds a storage-level lock, so other volume operations on that storage queue behind it, and a subsequent `move_disk` touching the same VM or storage can fail even though *our* move finished cleanly;
- the task list shows nothing running, so a naive "poll until the UPID stops" loop concludes the move is done and immediately issues the next one — straight into the lock.

This is why the executor's completion criterion is deliberately stronger than task success:

```
move m from a to b is DONE  ⟺  task(upid) exitstatus == OK
                            ∧  volume(m) absent from GET /storage/{a}/content
                            ∧  config(vmid(m)).lock is empty
                            ∧  elapsed(drain) ≥ min_wipe_seconds(m)
```

`min_wipe_seconds(m)` is `size_bytes(m) / |storage.saferemove_throughput|` — PVE's own configured wipe rate, already read live off the storage definition (§3.5), magnitude taken because the sign selects `cstream`'s throttling mode rather than the rate (§7.1) — or, when that throughput is not configured (the storage type has no such concept, e.g. Ceph RBD or ZFS, or `saferemove` is off there), simply absent from the criterion (the first three conditions alone still govern, exactly as before this term existed). **Found dogfooding against a real cluster: the first three conditions alone are not sufficient.** VM 101's `scsi1` move completed, its source volume was gone from the content listing, and its config `lock` read empty — every one of the first three conditions held — yet the very next move for the *same* VM (`efidisk0`) still had its `move_disk` task fail immediately with PVE's own

```
can't lock file '/var/lock/qemu-server/lock-101.conf' - got timeout
```

PVE's own wipe cleanup for the first move was still finishing at the OS level even though the API-visible signals this criterion polls had already cleared. `min_wipe_seconds` closes this in the one case it can be computed exactly, from the same formula §7.1's planning-time cost estimate already uses — one implementation, two callers, not a second copy of it.

Poll all conditions at `execution.poll_interval_seconds`, bounded by `execution.source_`

`release.timeout` (default **48h**, sized for a multi-TiB wipe at 10 MiB/s). On timeout, do not fail the run: mark the storage `draining`, exclude it as both source and target for the remainder of the run, report it, and let the next run re-evaluate from observed reality. Set `execution.source_release.wait: false` only on storages verified not to wipe — with `saferemove` off, the volume disappears immediately and this condition costs one extra API call.

3. The `move_disk` task's own flock — a residual race `min_wipe_seconds` cannot always close. The `lock:` config attribute in **1** and the file `move_disk` itself must flock to start (`/var/lock/qemu-server/lock-<vmid>.conf`) are not the same thing, and `min_wipe_seconds` above is only as good as the configured throughput it is computed from — no throughput configured, an inaccurate one, or the same "`can't lock file ... got timeout`" exitstatus surfacing for some other momentary reason entirely, and the race in **2** can still reach `move_disk` itself. `_wait_for_unlocked()` cannot catch it either way — there is nothing to poll that shows it in advance, since PVE clears the config `lock:` line before releasing the flock. As a narrow, mechanical safety net under **2**'s proactive fix, the executor retries the `move_disk` task itself when its `exitstatus` matches this exact message: up to `execution.locks.task_retry_limit` extra attempts (default **2**), each preceded by `execution.locks.task_retry_backoff` (default **15s**). A retry that still exhausts the limit is reported `"failed"` exactly like any other task failure — this is a fallback for the residual race, not a substitute for **1**'s open-ended wait or **2**'s computed floor.

Sizing the knobs against each other. With saferemove at its default throughput, one move can occupy its source storage for far longer than a whole planning cycle. Two consequences the implementer should not have to rediscover:

- `gates.cooldown_per_storage` must exceed the expected wipe time for that storage's largest disk, or the next run will plan moves onto a storage that is still draining and stall in `9.3`'s wait loop. `pve-storage-drs verify-storages` computes `z_max / saferemove_throughput` per storage and warns when the configured cooldown is shorter, or when it exceeds `migration.max_single_move_duration`.
- Keep `execution.max_concurrent_per_storage: 1`. Two moves off the same source mean two concurrent wipes sharing one throttle, so both take twice as long while both sources stay fully allocated.

9.4 Failure handling

`move_disk` is atomic from the caller's perspective: on failure QEMU cancels the mirror and the source volume remains authoritative, so there is nothing to roll back. The realistic hazards are:

- **Orphaned target volume.** A failed or cancelled mirror can leave `vm-101-disk-0` on the target. After any failure, list `/nodes/{node}/storage/{target}/content` and warn about volumes owned by the VM that are not referenced in its config. Do **not** delete them automatically — report them. They count against the reserve until removed, so surfacing them matters.
- **Partial plan.** With `abort_on_failure: true` (default), stop and report; the cluster is in a valid intermediate state and the next run will re-plan from reality.
- **Task supervision loss.** If the engine dies mid-move, the PVE task continues. On startup, check for running `move_disk` UPIDs owned by the DRS user before planning anything.

9.5 Output

Every mode emits the same machine-readable plan (JSON) plus a human summary. The example below is the section 14 fixture at `beta_move_count: 0.5` (the two-move variant):

```
Group fc-tier1 — imbalance 255% (threshold 20%) → ACT
  san-a  u=6.50  ████████████████████  used 4.5/8.0 TiB  ⚠ reserve short by 2.5 TiB
  san-b  u=0.70  ██                    used 1.5/8.0 TiB
  san-c  u=0.20  ▌                     used 0.5/8.0 TiB
```

```
1. 102:scsi0  san-a → san-c   1.5 TiB   ~2.2h   Δimbalance −4.53   ℓ/z 1.67
2. 101:scsi1  san-a → san-b   1.0 TiB   ~1.5h   Δimbalance −2.00   ℓ/z 1.00   [repair]

after: san-a u=3.00  san-b u=1.70  san-c u=2.70   spread 53% (from 255%)
payback: benefit 1.93e8 load·s vs cost 2.62e4 load·s → ratio 7344 (need 10) ✓
         plan repairs san-a's reserve (Σ r_s 2.5 → 0) — §7.3-exempt

pinned (not movable this run):
  106  snapshots present (2)      1.0 TiB  ℓ 0.9  on san-a  → clear snapshots to unblock
  107  locked: backup             0.5 TiB  ℓ 0.3  on san-b  → waited 0s, re-check next run
  109  excluded by tag no-drs     0.2 TiB  ℓ 0.0  on san-c
pinned load 1.2 of 8.6 (14%, warn at 25%);  best achievable spread given pins: 44.6%
```

The unfixable-shortfall line. Whenever the planned endpoint leaves any `r_s > 0`, the group's output ends with it, whether or not anything moves — a group the §6 override forced to `ACT` and that then moves nothing otherwise shows the breach once, in the gate line, and never explains it:

```
⚠ unfixable shortfall: 231.05 GiB still short of the snapshot reserve / free-space requirement after this
plan (slow-a 138.00 GiB, slow-b 93.05 GiB) -- no assignment of the movable disks can close it
  pinned on the short storages (unpin to let the next run move them):
    144592:scsi0 on slow-a -- pending config change (unapplied)
```

"No assignment … can close it" is claimed only for a CBC solve with status `optimal`, where stage 1 of the lexicographic solve makes it a theorem; the heuristic's line says it is not proven. `--json` carries the same fact as `groups[].unfixable_shortfall`: `null`, or `total_bytes`, `by_storage`, `proven` and `pinned_blockers` (`disk_key`, `storage`, `reason`). It describes the solver and scheduler's endpoint; a move the hard per-move duration rule later refuses is reported by that rule's own line, and the status file's warning (§2.4) tracks the executed endpoint.

The "no moves made" line. A group the gate sent to `ACT` whose plan moves nothing — with no deadlock and no unfixable shortfall to explain it — ends with a line saying the objective preferred the current placement (proven only for CBC `optimal`; the heuristic's line says it is not) and that `explain` shows the closest rejected move. When the capacity gate alone opened the group, a second line names `objective.delta_capacity_spread`, `objective.beta_move_count` and `objective.gamma_move_bytes_per_tib` with their values, and the break-even disk size they imply, $X^* = \beta / (2\delta - \gamma)$ (§5.4): "only disks larger than X^* move for data spread alone". When $2\delta \le \gamma$ it says instead that no disk of any size can. It then gives the two ways out: raise δ (or lower β) to bring X^* down, or raise or disable `gates.capacity_spread_threshold`. `plan` and `apply` print it; `explain` prints its own fuller version instead. The other common cause after an imbalance gate is a hotspot no placement relieves — one disk carrying most of a group's load — which `explain`'s pinned-load line quantifies.

As built (phase 13): the exemption note under the payback line and the per-move `[repair]` markers are printed by `plan`/`apply`, and the `repair` field reaches `--json` per move — computed by §7.3's revert test, not by the old "source presently violating" flag, which now survives only as `order_moves()`'s scheduling signal (§8.2 priority 1).

The **exemption itself needed a field of its own**, and phase 13 added it. `--json`'s payback block carried `benefit_load_seconds`, `total_cost_load_seconds`, `ratio`, `aggregate_ok`, `rejected_moves`, `deferred_moves` and `accepted` — none of which says *why* the aggregate test passed, so a plan accepted on merit and a plan accepted only because it repairs are the same object to a machine. (A consumer can re-derive it as `aggregate_ok and ratio < λ`, but only by supplying a `payback_ratio` the block does not carry, and re-deriving a verdict the tool already reached is not a specification.) The per-move `repair` markers do not close it either: §7.3's redundant-repair plan is exempt with no marked move, and §14.8 turns on exactly the distinction a bare `accepted: true` erases. Phase 13 therefore emits `repair_exempt` (the trigger's own

verdict) alongside the pair it is computed from, `reserve_shortfall_bytes_before/_after` — the two `Σ r_s` sums the trigger already has in hand at the call site. That pair is also the first instalment on §16.6's X-07/Y-04 gap: with it, check 4's "payback arithmetic" per variant records a verdict a regression diff can read, and check 2's `Σ r_s` invariant (final ≤ current) becomes checkable from `plan --json` without the emitted order.

A split says so. Whenever the plan changes the split excess of any `v ∈ V^split` (§5.3.3), the group's output carries one `split:` line per such VM — its largest footprint before and after, its cap `T_v`, and `o_v` before and after — and `--json` carries the same as `groups[].vm_splits[]` (`vmid`, `cap_bytes`, `peak_footprint_bytes_before/_after`, `excess_bytes_before/_after`), plus the summed `split_excess_bytes_before/_after` in the payback block beside the reserve pair. A VM still above its cap after the plan is listed too, whether or not anything moved, so an operator can see which large VMs the storages cannot spread further. `show-load` prints each storage's `Z_s` with the VM it comes from. For §14.9's fixture:

```
split: VM 701 largest footprint 10.0 → 4.0 TiB (cap 2.0 TiB, excess 8.0 → 2.0 — the solver found no better
split across 3 storages)
```

As built (phase 17): the line never claims a *proven* best split (REVIEW.md AO-03: `solver_status: "optimal"` means within `solver.mip_gap`, and the objective optimum trades the peak against β/γ/κ, so even a gap of 0 would not prove the peak minimal). When the MILP settled (`solver_status: "optimal"`) a result still above its cap ends "— the solver found no better split across N storages"; any other result (the heuristic, a `feasible` incumbent) ends "— still above the cap". The gate's `GateDecision.split_vmid` reaches the log record and the `gate` object of `show-load`/`plan --json` as `split_vmid`.

A converting move says so. A move onto an `enforce_format` storage whose source is in another format (§5.3.2) carries the conversion on its plan line, after the size — `1.5 TiB raw→qcow2` — and `--json` carries `format_from`/`format_to` on every move: `fmt_d` and `φ(d,target)`, equal for a move that does not convert. The conversion is part of what `confirm` mode asks the operator to approve, so it is printed, not inferred from the config.

The pinned block is not optional decoration — it is the "complain" half of the skip-and-complain policy of §3.7, and it is the only place an operator learns which snapshots to clear.

As built (REVIEW.md V-02): `plan`/`apply` print none of the above beyond the move list and payback verdict — neither renderer carries a pinned field, a fragmentation line, or a pinned-load line. That narration lives in `pve-storage-drs explain` instead (`docs/manual/29-explain.md`'s `pinned (not movable this run):`/`cannot fully consolidate:`/`pinned load ...` sections, including the per-pin → action hints shown above); `show-load` also names each pin inline, per disk, as `[pinned: <reason>]`. Read every "pinned block"/"plan output" reference above as `explain`'s output, not `plan`'s or `apply`'s. There is likewise no dedicated `data:` fill-deviation line in any renderer's human output (an earlier revision of this example showed one, REVIEW.md AA-06): the fill deviation reaches a human only via `explain`'s own `objective:` line (its `spread` term, shown further down this section) and `plan`/`apply`/`explain --json`'s `before_capacity_spread`/ `after_capacity_spread` scalars — the same asymmetry `show-load`'s `spread`-only human line already has relative to its own `--json`.

As built, added later: none of the above covers the case where the gate decides to ACT and every disk is eligible (nothing pinned), yet the solver's own optimum is still to move nothing – found dogfooding against a real cluster, where a two-disk VM sitting entirely on the busier storage was the only lever, and `objective.kappa_vm_affinity`'s fragmentation penalty for splitting it legitimately outweighed the imbalance it would fix. `plan`'s one-line verdict gives no way to tell that apart from "the solver didn't try" from the outside. `explain` now reports it: when `decision.act` is true and the final assignment moves nothing, it evaluates every single-disk move (`heuristic.best_single_disk_alternative()` —

every movable disk against every other group storage, via the same `evaluate_assignment()` the solver itself uses, section 5.3's one-move neighbourhood) and names the one closest to being worth it, with the section 5.4 term-by-term arithmetic that rejected it:

```
objective: imbalance 0.576 + moves 0 + bytes 0 + fragmentation 0 + spread 0 + split 0 + reserve 0 = 0.576
no moves made: the objective is lowest at the current assignment
closest alternative: 110:scsi1 VM-krbd → VM
  imbalance 0.576→0.0426, moves 0→0.25, bytes 0→0.00732, fragmentation 0→0.5, spread 0→0, split 0→0,
  reserve 0→0
  total 0.576 → 0.8  (worse by 0.223 -- rejected)
```

`--json` carries the same information as `rejected_alternative` (`null` unless this case applies), alongside `disk_key`/`vmid`/`device`/`from_storage`/`to_storage`, `baseline` and `objective` (each the seven-term breakdown `objective` above already serializes), and `worse_by`. `plan`/`apply` still print none of it, for the same reason as everything else in this section (V-02, above) — it is narration for a human, not a machine-checked verdict.

---

10. Forecasting

The decision statistic behind every placement is `window.quantile` (p95) of a disk's load, which the default `quantile` model takes over the **last** `W = window.lookback`. It is deliberately conservative and needs no fitting. `forecast.model: holt_winters` instead predicts that same statistic over the **next** `W`, so a disk whose load is rising, or has a daily peak the last window missed, is placed for what it will do. The mechanism is §12.1's; this section states what it is and what it needs.

10.1 What is forecast, and what history it needs

`window.lookback` is the **decision** window — the period whose load we are balancing. It is *not* the amount of history the model needs, and conflating the two makes Holt-Winters unreachable: `seasonal_periods` = 288 (24 h at a 5 m step) needs `2 × 288 = 576` samples, i.e. **48 h**, which a 24 h window can never supply. Config validation (§11.1) therefore **rejects** a `window.lookback` below `2 · seasonal_periods · metrics.step` under `holt_winters`, rather than silently degrading; the backtest of §10.2 additionally needs `2W` of Prometheus history, and `metrics.py` exposes `query_range` over an arbitrary range for it.

`forecast.model`	History it needs	The per-disk statistic
`quantile` *(default)*	`window.lookback`	`window.quantile` over the last `W` (no fitting, no extra query)
`holt_winters`	`max(lookback, 2 · seasonal_periods · step)` (48 h), and `2W` for the backtest	`window.quantile` of the fitted forecast path over the next `W`

The forecast is **not** the last forecast point (one sample at one hour of day says nothing about tomorrow's peak) and carries **no** `z · σ` band: a forecast p95 is compared with its `quantile`-model peers' observed p95, and a residual band would inflate exactly the disks that were forecast. There is no upper bound and nothing consumes one (REVIEW.md T-03, AL-01; the §7.3 saturation guard that did was removed by phase 14a). `seasonal_naive` was removed for the same reason — its statistic (the median of the same hour of day) is not a forecast over `W`. The removed config keys `window.upper_quantile`, `forecast.seasonal_lookback_days` and `forecast.holt_winters.residual_z` were deleted with it; a config that sets one fails validation.

The forecast never replaces a load, it **scales** it: `ℓ_d ← ℓ_d · f_d / h_d`, with `f_d` the forecast p95 and `h_d` the observed p95 of the same per-timestamp series (`loadmodel.apply_forecast()`). A disk keeps its observed `ℓ_d` when it is flagged for low coverage, has `h_d = 0`, or has no trustworthy fit. Forecasts are

produced **per disk**, at the one point every consumer (gates, solver, payback, ordering, `show-load`) reads ℓ from — and because a forecast-scaled ℓ is not a measured one, every consumer that *prints* loads also prints where they came from (`forecast:` line, `forecast` JSON object; §12.1 point 6, REVIEW.md AM-01).

As built (phase 14b): `forecast.py` is pure — `holt_winters_quantile()`, `backtest()`, `disk_factors()`, `forecast_group()` and `ForecastReport`; `cli._compute_group_load()` is the single caller-side wiring, used by both `show-load` and `plan`/`apply`.

10.2 Implementation warnings

- **Do not compute Holt-Winters in PromQL.** Prometheus's `holt_winters` was renamed `double_exponential_smoothing` in Prometheus 3.x and requires `--enable-feature=promql-experimental-functions`. More importantly, despite the historical name it is **double** exponential smoothing — level and trend only, with **no seasonal component** — so it cannot learn a daily cycle. Seasonal forecasting must happen engine-side on data pulled via `query_range`.
- Require at least `2 × seasonal_periods` samples before trusting a Holt-Winters fit, and fall back to `quantile` otherwise — with a **logged warning**, since a silent fallback hides a misconfiguration.
- Validate by backtesting: fit on `[t−2T, t−T)`, predict the p95 of `[t−T, t]`, compare against actual — and against the trivial baseline (persist the fit half's p95). Refuse to let a model that does not beat the baseline drive placement. **As built (phase 14b, §12.1 point 3):** no threshold; this replaced a comparison against `gates.imbalance_threshold`, an unrelated knob.

As built (bug fix): the live per-disk forecast-history fetch (formerly `cli.py`'s `_saturation_forecast_inputs()`, via `loadmodel.compute_disk_load_series()`) used to issue one **unchunked** `query_range` over its whole computed range — `2 · window.lookback` from §10.2's own backtest-gate floor above, combined with a fine `metrics.step`, confirmed live to exceed a VictoriaMetrics/gigapipe backend's own max-points-per-timeseries limit (11,000 by default) with a 500 "exceeded maximum resolution" error, something §16.2's `collect-testdata` capture path was already immune to. The live fetch (`loadmodel._fetch_raw_quantity_series()`, via the new `_issue_chunked_range_query()`) is now chunked exactly like §16.2's capture path — day-sized sub-queries (`metrics.RANGE_QUERY_CHUNK_SECONDS`), boundaries falling on the range's own start, never on wall-clock "now", stitched back into one series (`metrics.stitch_range_results()`, factored out of `collect.py`'s own `_stitch_range_captures` so both paths share one merge implementation) — every `plan`/`apply` run with a backtested forecaster active, not just `collect-testdata`, so neither path depends on Prometheus retention or `metrics.step` staying small enough to fit one request.

---

11. Configuration

Where it lives: `/etc/pve/drs.yaml`. That path is on pmxcfs, the cluster filesystem, so the file is replicated to every node automatically: one edit, one config, no per-node drift, and no question about which copy is authoritative. It is also an ordinary path, so a management host that is not a cluster member can simply provide it at the same location and every command, example and manpage stays true on both kinds of host.

Resolution order, first match wins:

#	Source	On failure
1	`--config PATH` (`-c`)	Error and exit non-zero
2	`$PVE_STORAGE_DRS_CONFIG`	Error and exit non-zero
3	`/etc/pve/drs.yaml`	Error naming the path and pointing at the shipped example

An **explicitly requested** config that is missing, unreadable or invalid is a hard failure. Never fall through

to the default: a run that silently balanced a production cluster from a different file than the operator named is the worst outcome in this document. Every run logs the resolved path and the SHA-256 of the file it actually read, in the first line of output and in the JSON report.

Three consequences of living on pmxcfs, all of which the implementation must respect:

1. **An edit is cluster-wide and immediate.** There is no staging step and no per-node rollout. A typo is live everywhere the moment it is saved, which is why §11.1's validation is a hard failure rather than a warning and why `dry-run` is the default mode.
2. **Do not put the state file there.** `state.json` is written on every run; pmxcfs is a small, quorum-gated, cluster-replicated store meant for configuration. Reads do not need quorum, so a node that has lost quorum can still read its config and plan, but writes do — keep `state.path` on local disk (§11.2).
3. **Treat the file as readable by the web server.** Files under `/etc/pve` carry group ownership `www-data` by default, which puts a plaintext `password:` within reach of anything running as the PVE web server. Prefer an API token restricted to the privileges of §3.5, and keep the secret out of the file entirely using `PVE_PASSWORD` / `PVE_TOKEN_SECRET` from the environment or a systemd credential. **Verify the ownership and mode on the target cluster** (`ls -l /etc/pve/drs.yaml`) before storing any secret in it — this document does not assume it.

A shared config does not make the tool cluster-aware. The obvious next step after putting the config on pmxcfs is to enable the systemd timer on every node, and that is wrong: `state.json` is node-local, so each node keeps its own lock, its own cooldowns and its own drift baseline, and the `fcntl` lock of §11.2 cannot see the other nodes at all. What does cross the cluster is the startup scan for in-flight `move_disk` UPIDs owned by the DRS user (§13) — it is the only reason two concurrent instances degrade to "slow and redundant" instead of "conflicting". **Run the timer on exactly one host.**

See `config/drs.example.yaml` for the fully annotated reference. The requirement-to-setting mapping:

Requirement	Setting
Storage groups VMs may not leave	`groups[].storages[]` — literal ids or `/regex/` patterns (§11.4)
2× the largest per-VM footprint free for snapshots	`snapshot_reserve.factor` (default `2.0`), per-storage override
Split a large VM over several storages	`snapshot_reserve.split_vm_footprint` (default `2TiB`, `null` = off), `objective.mu_vm_split_per_tib` (§5.3.3)
Keep N bytes / N% of each storage free	`free_space.soft` — global, per-storage or per-pattern (§5.3.1)
Convert disks to a storage's format when moving them onto it	`groups[].storages[].enforce_format` (default `null` = preserve, §5.3.2)
Min % changed traffic before migrating	`gates.drift_threshold` (default `0.10`)
% I/O difference across the group	`gates.imbalance_threshold`
Timeframe considered	`window.lookback` (default `24h`)
Minimal number of migrations	`objective.beta_move_count`
Keep a VM's disks together	`objective.kappa_vm_affinity`
Spread data evenly across storages (failure risk)	`objective.delta_capacity_spread`, `gates.capacity_spread_threshold`
Migration load accounted for	`migration.*`, `objective.gamma_move_bytes_per_tib`
Manual vs automatic	`execution.mode`
Forecasting	`forecast.model`

The config carries `schema_version: 1` at the top level. `config.py` rejects an unknown **major** version with an explicit message naming the version it understands, so a future incompatible change has a migration path instead of misinterpreting fields.

11.1 Validation rules

Validate with `jsonschema` for structure, then apply these semantic rules. Every one of them is a failure that produces a clear error and a non-zero exit, never a warning-and-continue — a misconfigured balancer moving production disks is worse than one that refuses to start.

Rule	Rationale
`schema_version` major matches	Forward compatibility
Every group non-empty, ≥ 2 storages **after pattern expansion** (§11.4)	A one-storage group has nothing to balance; with patterns the count that matters is storages matched, not entries written
No storage appears in two groups, after pattern expansion	A disk's group would be ambiguous; a pattern matching a storage another group also names is the same defect
Storage ids exist in the cluster; every `/…/` pattern matches at least one storage (§11.4)	Catches typos before they silently exclude disks or silently shrink a group
Every `/…/` pattern compiles as a Python regular expression, checked at load time	A malformed pattern must fail with the compiler's own message, not crash at match time (§11.4)
Within a group, no storage is matched by two pattern entries	Which entry's options apply would be arbitrary; a literal entry overriding a pattern is allowed and is not this error (§11.4)
`capability_weight > 0`	Appears in a denominator
`reserve_factor ≥ 0`	Negative reserve is meaningless
`snapshot_reserve.split_vm_footprint` is `null` or a parseable byte value `> 0`	§5.3.3. `0` would make every multi-disk VM's cap its largest disk and spread every one of them — a typo, not a policy
`mu_vm_split_per_tib ≥ 0` (`0` switches off the term, the split gate and the `w_v = 1` rule together); warn when `> 0` and `< kappa_vm_affinity`	A negative weight would reward gathering large VMs; below κ, a TiB of split excess no longer pays for opening a new storage and the rule rarely acts (§5.3.3's break-even)
`free_space.soft/hard`: absolute values `≥ 0` and parseable (bytes or byte-unit string); percentages `"N%"` with `0 ≤ N < 100`; `hard ≤ soft` after per-storage resolution and percent-to-bytes conversion	§5.3.1. A `hard` above `soft` makes every plan for a compliant storage infeasible; a percentage of 100 or more is a typo, not a policy
`enforce_format ∈ {raw, qcow2, null}`, and for every storage the entry resolves to (after pattern expansion) the value is in the formats that storage's type can hold (§5.3 (C2))	§5.3.2. `qcow2` on an RBD, ZFS or LVM-thin storage could never be honoured; a pattern entry that matches one such storage is the same error, named with the storage, because the cluster changed under the pattern rather than the file
`free_space.soft < C_s` for every storage, after resolution	A requirement no disk could leave room for is a typo; caught only once the inventory is loaded, like the pattern rules of §11.4
`0 ≤ drift_threshold ≤ 1`, `0 ≤ imbalance_threshold ≤ 1`	They are ratios
`quantile ∈ (0,1)`	A fraction; the decision statistic
`min_coverage ∈ (0,1]`	A ratio; `0` would accept a disk with no data
Metric names non-empty; label names non-empty and pairwise distinct	A duplicated label name silently collapses series
`rate_window ≥ 4 × metrics.pvestatd_push_interval`	Below this, `rate()` sees too few points. The interval is a PVE-side setting the tool cannot read, so it is declared in config (default `10s`, PVE's own default) and `verify-metrics` cross-checks it against the observed sample spacing of a live series, erroring if the two disagree by more than 20%
`window.lookback ≥ forecaster.required_range()`	See §10.1 — otherwise the model can never run
`payback_ratio > 0`, `payback_horizon > 0`	Zero disables the safety test
`payback_horizon ≥ 30d` (warn, not error)	A horizon of days rejects slow-accruing but real benefits; it should approximate VM lifetime, not operator patience (§7.2)
`tiny_disk_bytes ≥ 0`	The size below which a disk moves free of β, γ and the payback test (§5.4, §7), and only together with a larger disk of its VM ((C8), §5.3); `0` restores the old accounting and switches (C8) off
`delta_capacity_spread ≥ 0`; warn when `0 < 2·delta_capacity_spread ≤ gamma_move_bytes_per_tib`	A negative weight would reward concentration. Below the warning line the term is enabled but can never pay for a move of any size (§5.4's X^* is unbounded), which is `0` in effect without saying so. The earlier warning at `δ > α` is gone: δ is per TiB and α per in-flight request, so comparing them was comparing unlike units

Rule	Rationale
`capacity_spread_threshold > 0` where set, `null` disables	A ratio of fill fractions to the mean fill; it can legitimately exceed 1 (§14.2 measures 1.85)
`max_concurrent_* ≥ 1`	Zero would deadlock the scheduler
`execution.locks.wait_timeout > 0`, `on_timeout ∈ {skip, abort}`	A zero timeout turns every ordinary backup window into a failed run
`execution.source_release.timeout ≥ z_max / saferemove_throughput` for every storage where saferemove is on	Otherwise every large move times out into `draining` (§9.3)
`gates.cooldown_per_storage ≥ z_max / saferemove_throughput` (warn, not error)	The next run would plan onto a still-draining storage
`report.warn_pinned_load_fraction ∈ (0,1]`	A ratio
Time windows: `start ≠ end`; crossing midnight allowed and explicit	Ambiguity here silently disables `auto`
`execution.mode ∈ {dry-run, confirm, auto}`	Typo must not silently fall back to acting

11.2 `state.json`

The only persistent state, at `state.path`, default `/var/lib/pve-storage-drs/state.json`. Local disk, one copy per host, deliberately **not** on `/etc/pve` for the reasons in §11. Small, versioned, and written atomically (temp file + `os.replace`):

```
{
  "schema_version": 1,
  "lock": { "pid": 12345, "host": "mgmt01", "acquired_at": "2026-09-04T02:00:00Z" },
  "last_balance": {
    "at": "2026-09-03T22:14:03Z",
    "load_vector": { "fc-tier1:101:scsi0": 3.0, "fc-tier1:102:scsi0": 2.5 }
  },
  "cooldowns": {
    "disk":    { "fc-tier1:101:scsi1": "2026-09-03T22:41:55Z" },
    "storage": { "fc-tier1:san-b":     "2026-09-03T22:41:55Z" }
  },
  "inflight_upids": [],
  "staged_disks": []
}
```

- Keys are `"<group>:<vmid>:<device>"` and `"<group>:<storage>"`, so a vmid reused after a VM is destroyed and recreated in a different group cannot collide.
- `last_balance` is updated **only after a run that executed at least one migration** (§6).
- **Locking**: `fcntl.LOCK_EX` on the file for the duration of a run. If the lock is held but `lock.pid` is not alive on `lock.host`, reclaim it and log the reclamation — a killed run must not block the timer forever. A live PID means another instance is running: exit 0 quietly.
- `inflight_upids` is written *before* issuing each `move_disk` and cleared on completion, so a crashed run can be reconciled on the next start (§13).
- Losing this file is safe but not free: cooldowns and drift history reset, so the next run may migrate sooner than intended. Treat it as state to back up, not as a cache.

11.3 Global command-line options

Accepted before the subcommand, and shown by `pve-storage-drs --help` with their defaults:

Option	Default	Effect
`-c, --config PATH`	`/etc/pve/drs.yaml`	Read the configuration from `PATH`. Missing or invalid is a hard failure (§11)

Option	Default	Effect
`--group NAME`	all groups	Restrict the run to one group; repeatable. Groups are independent (§5), so this changes nothing about the result for the groups selected
`--mode {dry-run,confirm,auto}`	`execution.mode`	Override the execution mode for this run only
`--json`	off	Emit the machine-readable report of §9.5 instead of the human one
`-v, --verbose`	warnings only	`-v` adds this run's decision trail (`INFO`), `-vv` adds per-query detail and third-party library logs (`DEBUG`) — §2.3
`--quiet`	warnings only	Errors only. Note that on an `auto` run this discards the audit trail §2.3 otherwise logs unconditionally
`--log-level {error,warning,info,debug}`	—	Set the level explicitly; wins over `-v`/`--quiet` (§2.3)
`--log-format {auto,text,json}`	`auto`	`auto` = human text when stderr is a TTY, JSON otherwise (§2.3)
`--version`	—	Version, then exit
`--manual`	—	Show `pve-storage-drs(1)` (§8.5 of `AGENTS.md`)
`--replay PATH`	live cluster	Run against a `collect-testdata` bundle at `PATH` instead of the live cluster, with no network access at all — §16.5

Two rules the implementation must honour.

Every `--mode` override is logged, and an override toward less safety is a warning. Order the modes `dry-run` < `confirm` < `auto`. Moving *down* that order — `auto` to `confirm`, `confirm` to `dry-run` — is logged at info: an operator being more careful than their config needs no ceremony. Moving *up* it is the operator deliberately removing a barrier they themselves configured, whether that barrier is the dry run or the per-step confirmation, so `dry-run → confirm`, `dry-run → auto` and `confirm → auto` all log at **warning** level, naming both values. In `auto` mode the log is the only record a human will see (§2.1), and "why did it move disks when the config said confirm" must be answerable from it.

No option may set a value that `config.py` would have rejected in the file. The command line goes through the same validation (§11.1), because a knob that is only checked on one of its two paths is a knob that is not checked.

11.4 Storage name patterns in `groups[].storages[]`

A `storages[]` entry may name storages by regular expression rather than by literal id: an `id` value that both begins and ends with `/` is a pattern, and the text between the slashes must compile as a Python `re` expression. `/san-.*/` is a pattern; `san-a` stays a literal id and behaves exactly as before. The form is additive — no `schema_version` bump — and it cannot collide with a literal one: PVE storage IDs never contain `/`, so a value wrapped in slashes can only have been meant as a pattern.

```yaml
groups:
  - name: fc-tier1
    storages:
      - id: /san-.*/            # pattern: every san-* LUN, present and future
        capability_weight: 1.0   # the entry's options apply to EVERY storage it matches
      - id: san-b               # literal: takes precedence over the pattern above
        capability_weight: 0.5  # the documented exception for one slower array
```

Four rules, each shaped so that the one new failure mode — a pattern that matches too much or nothing

at all — stays checkable instead of silent:

- **Matching is `re.fullmatch`, case-sensitive.** The pattern must match the *entire* storage id. `/prod/` names exactly the storage `prod`; under substring semantics it would also capture `preprod`, which is the classic too-greedy pattern and the reason whole-id matching is the rule. Python `re` syntax throughout: inline flags such as `(?i)` work, and there is no flag suffix after the closing slash.
- **The entry's options apply to every storage it matches.** A pattern entry accepts the same per-storage options as a literal one (`capability_weight`, `reserve_factor`, `free_space`, `enforce_format`), and every matched storage inherits them. This is the point of the feature: one entry weights or reserves a whole LUN family. A pattern-level `free_space` is resolved per matched storage — a `"10%"` demands a tenth of *each* LUN's own capacity (§5.3.1).
- **A literal entry beats a pattern.** Within one group, a storage named by a literal entry uses the literal's options even when a pattern also matches it — pattern as the default, literal as the exception, and deliberately not an error, because without this rule a catch-all pattern would make it impossible to single out one member of its own group. Two *patterns* matching the same storage in one group are a hard error (§11.1): which entry's options should apply would be arbitrary, and the config refuses to guess. Precedence is semantic, not positional — entry order never matters.
- **Across groups, disjointness is checked on the expanded set.** A storage matched — literally or by pattern — by entries of two different groups is a hard error naming the storage and both entries. A disk's group must stay unambiguous, exactly as §11.1 already demands for literals.

Expansion happens at run start, against the live cluster. Every pattern is matched once per run against the storage definitions (`GET /storage`, §3.5 — the same call that already validates a literal id's existence and content type, so pattern and literal ids are checked against one consistent source), in the same step that checks literal ids for existence, and the result is part of the per-run topology cache. A storage a pattern matches that has a definition but is reported active by no node (disabled) is also a hard error, naming the pattern and the storage — the same fail-safe outcome a literal reference to it would hit, but with a message that does not require the operator to already know a pattern was involved. Which storages a pattern matches is a property of the cluster, not of the file: a LUN added to the cluster joins its group on the next run with no config edit — half the reason to use a pattern — and a pattern that matches nothing is handled exactly like a literal id that does not exist, a hard error before anything is planned, because a typo is the likelier cause and a silently shrunken group the likelier consequence. Nothing downstream ever sees a pattern: `S` (§5.1), the (C2) eligibility pass, cooldown keys and the `state.json` keys of §11.2 all carry real, expanded storage ids. Those keys embed the group name, so a later config change that moves a storage into a different group leaves its old cooldown keys as stale entries that simply stop matching — harmless. Group names themselves, `--group` (§11.3) and the `exclude.*` lists stay literal; only `storages[]` entries accept the `/…/` form.

Validation and visibility. `config.py` compiles every pattern at load time and reports a malformed one with the compiler's own message; the zero-match and overlap rules run every run once the inventory is loaded, because they compare the file against the cluster. Because an over-broad pattern is the one new way to misconfigure a group, the expansion is always visible: every run logs `entry → matched ids` at INFO, and `pve-storage-drs verify-storages` (§3.5) prints the expansion next to each storage's properties and lists cluster storages matched by no group — the same "ungrouped, not managed" visibility (C2) gives `show-load`. One consequence to keep in mind when writing a pattern: a matched storage becomes a full member of the group, exactly as a literal entry, and `u*` (§5.3 C6) divides by every member's `c_s` — so a pattern that captures an ISO-only or otherwise unusable LUN skews the balance target even though (C2) keeps any disk from ever landing there. Eligibility is unchanged and still per storage; safety never depends on the pattern being precise, but the quality of the balance does.

12. Implementation phases

Each phase is independently testable and useful on its own.

#	Phase	Done when
1	`config.py`, `metrics.py`, `pve-storage-drs verify-metrics`	Real metric/label names confirmed against the live Prometheus; per-disk load printed
2	`pve.py`, `topology.py`	`pve-storage-drs show-load` prints every storage with its disks (all buses), sizes, loads and reserve status; pinned disks flagged with their reason; `pve-storage-drs verify-storages` reports saferemove, implied wipe times and the expansion of every `/…/` storage pattern (§11.4)
3	`loadmodel.py` + gates	Correct act/no-act decision per group, with the reasoning shown
4	`heuristic.py` + `schedule.py`	End-to-end plan in `dry-run`, ordered and transient-feasible; reproduces `expected_order` and `expected_final_reserve` in the §14 fixture
5	`payback.py`	Plans rejected/trimmed on cost grounds, arithmetic shown
6	`optimize.py` (MILP)	Matches or beats the heuristic on the §14 fixture; CP-SAT and CBC agree on every β case, and the §5.5 coefficient assertions pass
7	`execute.py`	`confirm` mode against a lab cluster, including a VM locked mid-run and a source storage with `saferemove` on — the run must wait, not fail
8	`auto` mode + time windows	Unattended operation
9	`forecast.py` beyond p95	Seasonal-naive validated by backtest
10	`anonymize.py`, `collect.py`, `replay.py`, `tests/corpus/` (§16)	A bundle collected from a live cluster replays to the same plan the live run produced; the scrub audit and the determinism test pass on it
11	Logging policy (§2.3)	**Done.** A clean read-only run prints nothing on stderr; `apply --mode auto` logs the full §2.3 audit trail (gate, load, plan, payback, every UPID) without being asked; `--log-format`/`--log-level` behave as specified; the verification tests of §2.3 pass
12	Capacity-spread objective and gate, one-year payback horizon (§5.3 (C7), §5.4 δ, §6, §7.2)	**Done.** Fixtures regenerated with the `delta_values` sweep and the 365d horizon; a replayed bundle shows the capacity gate deciding; `explain` reports the fill deviation; the manual documents `objective.delta_capacity_spread`, `gates.capacity_spread_threshold` and the new `payback_horizon` default (the manpage documents no individual knob, by §11's own established convention)

#	Phase	Done when
13	Free-space requirements (§5.3.1, §5.3 (C5), §6 override, §7.3 repair exemption, §8.1 hard floor) **and the (C2) format-compatibility eligibility it needs**	`config_schema.json` gains the block **first** — the schema is closed (`additionalProperties: false` throughout, deliberately: it is where a typo'd key is caught, §11.1's structural pass), so a `free_space:` key is rejected before `config.py` ever sees it: a top-level `free_space` object and a `free_space` property on `groups[].storages[]`, each with `soft`/`hard` typed `["string", "number", "null"]` for §5.3.1's grammar (integer bytes, byte-unit string, `"N%"`, and `null` with its two by-level meanings); `config.py` then resolves `free_space.soft/hard` per storage (bytes, byte-unit strings, percentages; global, per-storage, per-pattern; the global `snapshot_reserve.min_free_bytes` scalar deprecated, folded in per storage after percent conversion as `soft_s = max(soft_s_resolved, min_free_bytes)` — §5.3.1), validates `hard ≤ soft` and `soft < C_s` **on the written values, before that fold** (§5.3.1, "validate as written, then fold"); the per-storage `soft_s`/`hard_s` pair replaces the `min_free_bytes` scalar parameter across `reserve.compute_reserve_status()`/`transient_charge_ok()`, `heuristic.run_heuristic()` and its helpers, `schedule.transient_invariant_ok()`/`order_moves()`, `optimize.py`, `execute.py`'s live execution-time re-check and every `cli.py` call site that threads the scalar today, and `collect.py`'s bundle manifest (which serialises the scalar, so a replayed bundle carries the pair instead — §16); `topology.Storage` gains the type/format fields (C2) needs and both solver backends fix `x_{d,s}=0` for format-incompatible targets; `payback.py`'s repair detection (`ScheduledMove.resolves_reserve_violation`, set by `schedule.py`'s "source presently violating" test) is replaced by §7.3's outcome trigger (exempt iff the plan's final `Σ r_s` is strictly below the current assignment's) plus a per-move `repair` marker computed by the revert test — re-scoring `Σ r_s` on the final assignment with one `x` held — while `order_moves()`'s internal priority-1 test keeps §8.2's "source currently violating" form (a current-state rule, not a plan-outcome one); the outcome trigger is a **signature and data-flow change**, not a flag swap: `evaluate_plan_payback(move_costs, benefit_load_seconds, payback_ratio)` has no access to `Σ r_s`, so the current and final

#	Phase	Done when
14	Holt-Winters-driven placement; §7.3 saturation guard removed (§12.1; REVIEW.md T-03, AL-01, AL-04)	Two commits, in order. **14a** deletes the saturation guard — `migration.bwlimit_bytes_per_sec` is the only throttle a migration needs — deleting `saturation_load`/`saturation_ceiling` from the schema outright (no compatibility shim: a config that sets them fails validation, and the committed bundles' `config.yaml` were edited). **14b** scales each disk's ℓ_d by a backtest-validated Holt-Winters forecast of its p95 over the next `window.lookback`, at the one point gates, solver, payback and ordering all read it. The default `forecast.model: quantile` is unchanged: §14 fixtures and quantile corpus variants byte-identical apart from the removed saturation fields. Done when §12.1's checklist holds
15	Single-source configuration defaults (§11.1; REVIEW.md AL-03)	Every default exists **exactly once, on the dataclass field**; the loader constructs each config class from the raw mapping by passing **only the keys the operator actually wrote**, through field-level converters (duration/byte/percent strings, list→tuple), so no `.get(key, default)` ever restates a default — today's twin copies in `config.py` (e.g. `model: str = "quantile"` on `ForecastConfig` beside `fc_raw.get("model", "quantile")` in the loader, and the same shape for every other knob) are gone; `config_schema.json`, the third copy of the shape, is generated from the same field/type/enum source — or, if generation proves heavier than checking, a check target fails on drift between schema and dataclasses — so it cannot rot either; **zero operator-visible behaviour change**: `--help`, the manual's option tables and `config/drs.example.yaml` values are byte-identical before and after, proven by the fixture and corpus checks running green untouched

#	Phase	Done when
16	Per-storage target format, `enforce_format` (§5.3.2)	**Implemented, not yet live-tested.** The `qcow2-on-lvm` allocation rule was read from `LVMPlugin.pm` (`alloc_lvm_image`/`calculate_lvm_size`) and recorded in §5.3.2, and `z_{d,s}` has its third term (`topology.qcow2_lvm_allocation_bytes()`, an upper bound). Built as follows: `config_schema.json` gains `enforce_format` (`enum [raw, qcow2, null]`) on `groups[].storages[]` — no top-level key; `config.py` resolves it per expanded storage with §11.4's literal-beats-pattern precedence and runs §11.1's type check once the inventory is loaded; `topology.target_format()` and a per-pair `disk_size_on()` are the single implementations of `φ(d,s)` and `z_{d,s}`, and `storage_accepts_format()`'s callers in `optimize.py`, `heuristic.py` and `execute.py` pass `φ(d,s)` instead of `fmt_d`; the MILP's (C4)/(C5)/(C7) rows and `schedule.transient_invariant_ok()` use `z_{d,s}`; `execute.py` sends `format=` iff `φ ≠ fmt_d` (§9.2), and its existing conversion charge becomes the planned case; `verify-storages`, the plan line and `--json` show it (§3.5, §9.5). The cases of that fixture — one enforcing target makes an otherwise ineligible move feasible, and the converted `z_{d,s}` changes which disk fits — are covered by `tests/unit/test_enforce_format.py` against both solver backends (an exhaustive-enumeration fixture from `generate_expected.py` is not added); every existing fixture and corpus bundle must reproduce the same decisions (no `enforce_format` ⇒ `z_{d,s} = z_d`); the corpus `expected.json` files change only by the two new per-move keys `format_from`/`format_to` (§9.5), which is the whole diff. The manual, `config/drs.example.yaml` and `docs/internals/` move in the same commit (no CLI option is added, so the manpage's `OPTIONS` is unchanged). **Done when** additionally a live `confirm` run on the dev cluster converts one data disk and one `efidisk0` in each direction onto shared LVM, the source volume is observed gone (§9.3), and the target's listed size matches `z_{d,s}`

#	Phase	Done when
17	Per-VM snapshot footprint and large-VM splitting (§5.3.3, (C4), (C9), §5.4 μ, §6 split gate, §7.2, §8.1, §9.2, §9.5)	**Implemented.** `Z_s` is the largest per-VM footprint in one shared function that `reserve.compute_reserve_status()`, `schedule.transient_invariant_ok()`, `execute.py`'s live re-check, `optimize.py`'s (C4) rows and the heuristic all call — a second copy of the footprint sum is the same bug as a second reserve check (AGENTS.md §5); `config_schema.json`, `config.py`, `config/drs.example.yaml` and the manual gain `snapshot_reserve.split_vm_footprint` (default `2TiB`, `null` off) and `objective.mu_vm_split_per_tib` (default 1.0) with §11.1's two rules, and `drs.example.yaml`'s `snapshot_reserve.factor` comment stops saying "largest disk"; `optimize.py` adds `o_v` and (C9) and the μ term, `heuristic.compute_vm_weights()` returns `w_v = 1` for `V^split`, the heuristic's repair and polish steps follow §5.3.3; `gates.py` gains the split gate (`split_gate()`, handed the objective weights and the storage cooldowns by `evaluate_group_gates()`); `payback.py`'s benefit gains μ·ΔO·H; `schedule.py`'s ordering ratio gains the μ term; `plan`/`apply`/`explain` print the `split:` line and `--json` carries `vm_splits[]` and the split-excess pair (§9.5); `show-load` and `verify-storages` print `Z_s` with its VM (§3.5). Fixtures: the generator's independent restatement of the reserve uses the footprint; the four existing fixtures state `split_vm_footprint_tib: null` explicitly (§14.1 says why), and the new `large-vm-split.yaml` of §14.9 states `2.0` and `mu_vm_split_per_tib: 1.0`; `generate_expected.py` regenerates every expected file — `fc-tier1` changes only as §14 records (initial `Z_a` 3.0 and `r_a` 2.5, shortfall pair 2.5 → 0.0, `101:scsi1` `repair: true`; optimum, objectives, order, transient targets and every payback number unchanged), `affinity-repair`'s stor-a reserve basis rises by its two tiny disks' bytes, `reserve-tradeoff` and `free-space-repair` (single-disk VMs only) do not change — and the per-storage `largest_tib` field is renamed `largest_footprint_tib`; a unit case covers §7.3's redundant-repair plan, which `fc-tier1` no longer exercises; the committed corpus bundles are replayed and their expected files regenerated, with any plan change explained in the commit. Every file matching `git grep -il 'largest disk'` is swept, the manual and internals included

#	Phase	Done when
18	Capacity-spread term in absolute TiB (§5.3 (C7), §5.3 `U_obj`, §5.4 δ and X*, §6, §7.2, §9.5, §11.1)	**Implemented**, except the `debian/changelog` note, which belongs to the next release entry. One implementation of $d_s = \lvert B_s - \bar{b} \cdot C_s \rvert$ in TiB, shared by `heuristic.py`'s objective evaluator and `optimize.py`'s (C7) rows (sizes in MiB there, so δ enters as $\delta / 2^{20}$ per MiB, like γ); `payback.py`'s $\delta \cdot \Delta F$ and §8.2's ranking follow from it unchanged; the `U_obj` bound's δ term becomes $\delta \cdot 2 \cdot (\Sigma z_d + \Sigma U^{sext})$ (today only the fixture generator's `computed_p_min` evaluates the bound — `P` is not yet computed in `src/`, so there is nothing else to change, and ΣU^{sext} must be among its logged inputs whenever it is); `config.py` drops the $\delta > \alpha$ warning and warns at $0 < 2\delta \le \gamma$; the §9.5 "no moves made" line prints X*; the §6 gate is unchanged. `generate_expected.py`'s `F_of` and `computed_p_min` change the same way and every fixture is regenerated: checked against a scratch regeneration while this was specified, **no fixture's optimum, move set or order changes**, only `F_*`, objective, `P_min`, agreement-threshold and benefit values, all of which §14 already quotes. Every corpus `expected.json` is regenerated and its diff read by hand, since a real bundle's decisions may change, which is the point. The manual's `delta_capacity_spread` entry (unit, X*, worked break-even), `config/drs.example.yaml`'s comment, `docs/internals/`' objective page move in the same commit; `explain`'s objective line keeps printing the weighted `spread` term, which is now δ times TiB. **Done when** additionally (live-verified on the operator's `cluster-6`, not replayable here: no committed bundle reaches the regime) `--replay` of a bundle like `cluster-6` (equal storages, balanced I/O, data concentrated, disks of a few hundred GiB) plans capacity moves at the default weights, where the relative form planned none, and `debian/changelog` tells operators who had raised δ under the relative form that the same number now means a price per TiB, to be re-derived from X* rather than carried over

Phase 4 before phase 6 is deliberate: a working heuristic makes the MILP verifiable, and it is the production fallback for large groups. Do not start with the solver.

Phase 6's done-when said "CP-SAT and CBC agree on every β case"; that agreement was proven while both backends existed. The CP-SAT backend has since been removed (REVIEW.md AL-02 — not in Debian, never installable on the deployment target), and the fixture now binds CBC to the exhaustively-enumerated optimum alone. Phases 14 and 15 were added after the phase 13 implementation by the twenty-eighth review pass (REVIEW.md section 56): 14 closes §10.1's own target design and is a behaviour change; 15 is a pure internal refactor with no behaviour to specify beyond §11.1's existing validation rules. Phase 14's scope was then cut down by operator direction (REVIEW.md AL-04, §12.1): no p95 → p99 default change, and the saturation guard goes. Also after phase 14, and by the same direction (no compatibility shims: the only users are the maintainers), `snapshot_reserve.min_free_bytes` and its fold into `free_space.soft` were removed; phase 13's row above describes the fold as it was built.

Phase 18 was added after replaying a production bundle (`cluster-6`, §5.3 (C7)) on which the capacity gate opened two groups every run and the relative-fill (C7) term made every move lose. It changes the term's unit, not its role, and needs no new configuration key.

Phase 11 is last only because it was found last — dogfooding the finished tool, where the noise on a clean run and the silence on an `auto` run are both obvious in a way they never were while the engine underneath was still being built.

12.1 Phase 14 in detail

Operator direction (2026-09-25): use forecasts of the VMs' I/O to place disks better. Do **not** model storage saturation around migrations — a migration may run at any time, capped by `migration.bwlimit_bytes_per_sec`, and that is the whole throttle. Keep it small; every item below that is not needed for that goal is out of scope.

14a — remove the §7.3 saturation guard (one commit) — done

Why: it is the forecaster's only consumer today, it is inactive unless an operator sets `saturation_load` (which has no safe default, §7.3, and is set nowhere we know of), and all it can do is defer moves. `execute.py` already passes `bwlimit` to every `move_disk`; `max_single_move_duration`, the transient reserve invariant (§8.1) and `execution.cooldown_per_storage` stay as they are.

Delete (use `git grep -n saturation` as the checklist; it must come back empty outside REVIEW.md and this plan's history notes):

- `payback.py`: `_saturation_deferred()`, `MoveCost.saturation_deferred`, `PaybackResult.deferred_moves`, `compute_move_cost()`'s `target`/`l_hat_src`/`l_hat_dst` parameters, and the module docstring's `headroom_*` and "mirroring-phase-only reading" bullets.
- `cli.py`: `_saturation_forecast_inputs()`; `_compute_one_move_cost()` collapses to a plain `compute_move_cost()` call; `saturation_deferred` leaves `excluded_disk_keys`; the deferred branch of `_refused_move_outcomes()` and `_apply_payback_gate()`; the `saturation_ceiling` output line (~l. 1150). **Keep** `_backtest_gated_forecaster()` — 14b rewrites and reuses it.
- `forecast.storage_upper_bound()` and its tests.
- `topology.Storage.saturation_load`, `config.StorageConfig.saturation_load`, `MigrationConfig.saturation_ceiling`, `config._check_saturation_load()`.
- `collect.py`: stop writing both keys into a new bundle's config.
- `execute.py`: the docstring notes on the saturation check.
- `config/drs.example.yaml`: both keys and their comments.
- Docs: every manual/internals/`.agents` file `git grep -l saturation` lists. This plan: §7.1's formula becomes `duration_mirror_d = z_d / bwlimit`; §7.3 loses the second hard rule and the "Defining 'during the mirror'" text through the `N_s` paragraphs, and "the two rules that fire at this gate" becomes one (`max_single_move_duration`); §10.1 loses the $\hat{L}$ `_s(Δ)` paragraph; §15.1 loses the knob rows. In their place one sentence in §7.3: migrations are throttled by `bwlimit` only; the tool does

not model storage saturation.

No compatibility shim (operator direction: the only users are the maintainers): `config_schema.json` drops `groups[].storages[].saturation_load` and `migration.saturation_ceiling`, so a config that still sets either fails validation. The three committed corpus bundles' `config.yaml` had `saturation_ceiling` (and, for 14b, `upper_quantile`, `seasonal_lookback_days`, `residual_z`); those lines were removed and each bundle's `SHA256SUMS` entry updated. `plan --json` loses `deferred_moves` and the per-move `saturation_deferred`; the fixture and corpus expected files are regenerated.

14b — Holt-Winters-driven `ℓ_d` (one commit) — done

This replaces REVIEW.md AL-01's original design (decision statistic → the forecaster's upper bound, default p95 → p99). That changed every plan on every cluster for no forecasting gain and is dropped.

1. **What is forecast.** The decision statistic stays `window.quantile` (p95) of a disk's load. The `quantile` model takes it over the *last* `W = window.lookback`; `holt_winters` predicts it over the *next* `W`: fit the disk's series, forecast `ceil(W / metrics.step)` steps, take `window.quantile` of that forecast path, clamp at 0. **Not** the last forecast point — today's `HoltWintersForecaster.predict()` returns the single value at `now + horizon`, i.e. one sample at the current hour of day, which says nothing about tomorrow's peak — and **no** `z·σ` band: a forecast p95 is compared with its quantile-model peers' observed p95, and a residual band would inflate exactly the disks that were forecast.

2. **How it enters `ℓ_d`: a ratio, never a substitution.** `compute_disk_load_series()` normalizes per timestamp while `compute_group_load()` normalizes over the window and also owns coverage rejection and the `last_known_loads` fallback. So scale, do not replace: `ℓ_d ← ℓ_d · f_d / h_d`, with `h_d` the `window.quantile` of the disk's own series over `[now−W, now]` and `f_d` the forecast from point 1. Keep `ℓ_d` unchanged when `h_d = 0`, when the disk carries a `DiskLoad.flagged_reason`, or when its fit fails (fewer than `2 · seasonal_periods` samples, a constant series, any statsmodels exception or `ConvergenceWarning`). One helper — e.g. `loadmodel.apply_forecast(group_load, factors) -> GroupLoad` — called at **both** `compute_group_load()` sites in `cli.py` (show-load/explain ~l. 1010, plan/apply ~l. 2474): a group's gates and its plan must see the same `ℓ`.

3. **Gate: beat the baseline, no threshold.** Once per group, on `group_aggregate_series()`: fit on `[now−2W, now−W)` and have both models predict the p95 of `[now−W, now]` (the quantile model's prediction is the p95 of the fit half — persistence). `holt_winters` is used for this group iff its absolute error is ≤ the quantile model's; otherwise quantile for this run, with the existing `forecast_backtest_failed` warning. Less than `2W` of history → quantile, as today. This replaces §10.2's comparison against `gates.imbalance_threshold` (an unrelated knob) and `backtest_error()`'s point-at-horizon-versus-window-mean mismatch.

4. **Fetch.** Only when `forecast.model == holt_winters`, via the existing (chunked) `compute_disk_load_series()` over `max(required_range_seconds(...), 2·W)`. The default quantile path issues no additional Prometheus query.

5. **Drop `seasonal_naive`.** Its statistic is the same-hour-of-day median at `now` — not a forecast over `W` — and after 14a nothing consumes it; Holt-Winters' seasonal term covers the diurnal case. It leaves the schema enum (loud, like `cpsat`; changelog) and the corpus variant matrix. Likewise delete `Forecast.upper_bound`, `holt_winters.residual_z` and the forecaster-side `upper_quantile` if nothing reads them any more; a removed **config key** is deleted from the schema too — no compatibility shim, as in 14a.

6. **Output.** Holt-Winters' per-call fallback warning drops to DEBUG (one per disk would flood the journal); instead one INFO per group, e.g. `group g: forecast holt_winters used (backtest err 0.08 vs baseline 0.14), 37 disks scaled, 5 kept.` `explain`, `show-load` and `plan --json`'s

group report gain `forecast: {model, used, backtest_error, baseline_error, disks_scaled, disks_kept}` (present only when a report exists; `explain` and `show-load` render it as one `forecast:` line — under the group header in `show-load` — so scaled loads never pass as measured, REVIEW.md AM-01).

7. **Cost.** One statsmodels fit per disk with history, per group per run — order 0.1 s at 2016 samples (7 d at 5 min). Fine for a timer; no caching, no parallelism.

As built and observed (2026-09-25, dev cluster, `window.lookback: 3d`, 1 h step, 35 disks): with 7 d of history a `7d` window has no `2W` to backtest (fit half: 10 samples), and the report says so (`backtest_error: null`). With `3d`, Holt-Winters fitted (error 6.5) but lost to the baseline (3.6), so the group stayed on `quantile` — the gate doing its job on a noisy cluster. Forcing the gate open to look at the factors gave median `f_d/h_d` 1.37 but a range of 0.00 – 39: an additive trend extrapolated over a whole window is undamped, and a disk with a tiny observed p95 can be scaled by a large ratio. No clamp was added (it would be a magic number the plan forbids); if a real cluster's gate opens and the factors look wild, a damped trend (`statsmodels`' `damped_trend`) is the first thing to try.

Follow-up sweep (same day, 5 m step, ~7.5 d of retention). At the default 288 periods `initialization_method="estimated"` never converged (it optimizes all 288 initial seasonals as well), so every fit was rejected; the fit now uses `"heuristic"` (initial states from a classical decomposition of the first cycles, only the smoothing parameters optimized). A rolling-origin backtest (every 6 h back through the retention) over step × lookback × trend × seasonal found: `seasonal: none` always loses; a 1 h or 15 min step is worse than 5 min (each point is one 5-min `rate()` sample, not an average over the step, so a coarse step aliases); 5 min with `3d` beat persistence at 4 of 5 origins with or without trend. `trend: add` produced the runaway factors above (one disk forecast at 95 % of the whole group's load); `trend: none` kept the worst single-disk shift at 23 % for the same score, so `trend` now defaults to `none`. With 5 min / `3d` / `trend: none` the live gate opened (`used: true`, 1.43 against 2.26, 35 of 39 disks scaled) — the first real run on Holt-Winters.

Done when:

- With `forecast.model: quantile`, `plan --json` for every fixture and quantile corpus variant is identical to 14a's output.
- Unit tests cover: the ratio (`h_d = 0`, flagged disk and failed fit keep ℓ_d); the p95-of-path statistic on a synthetic diurnal series whose next-day peak is higher; the gate choosing Holt-Winters on a seasonal-plus-trend series and quantile on white noise; the removed keys and `seasonal_naive` rejected by the schema.
- `bzed-dev-cluster-2d-holt-winters` (replacing the 7d bundle, whose capture held only `W`, not the backtest's `2W`) replays with the forecast block populated; its regenerated expected file is read by hand before committing. As captured, the backtest runs and Holt-Winters loses to the baseline (`used: false`), so `used: true` is covered by unit tests only.
- Manual: when `holt_winters` is worth selecting (diurnal or trending load), what the gate does, and that it needs `python3-statsmodels`.
- `make check` green.

---

13. Failure modes and safety

Hazard	Handling
Metric gap / disk below `min_coverage`	Use last known load from state; flag in output; never treat as zero
Counter reset (VM reboot, node migration)	Handled by `rate()`; `sum by (vmid, device)` collapses node labels

Hazard	Handling
VM live-migrated between nodes mid-plan	Re-fetch node before each move (§9.2); mismatch → abort move, re-plan
Disk has an existing snapshot chain	Pinned, load and bytes still counted, reported at WARN every run with the pinned load/bytes per VM (§3.7). `move_disk delete=1` is rejected by PVE on such volumes and would not carry the snapshots anyway
Snapshot created between planning and execution	Re-checked immediately before every move (§9.2 step 5); the move is dropped and the plan re-planned
Target storage filled, or a disk created or moved onto it, by another user or tool after planning	The live provisioned re-check (§9.2 step 2) refuses the move; the run re-plans from fresh state, up to `execution.max_replans_per_run`, then bails out (exit `0`, next run starts fresh) — §9.2 re-plan protocol
PVE API error while re-reading the VM or the target before a move, or an unsized volume on the target	Not a mismatch: the run fails (exit 1), no further group is visited — §9.2 "Errors are not mismatches"
Prometheus error while computing a group's load, in the first plan or a re-plan	The run fails (exit 1); `plan`/`explain` report every group they can and exit 1 — §9.2 "Errors are not mismatches"
VM has `efidisk0` / `tpmstate0`	Ordinary movable disks on PVE 9.2 (verified on a live cluster). Below `migration.tiny_disk_bytes` they move free of β, γ and the payback test, so κ reunites them with their VM (§3.6, §5.4, §7.3) — and only ever move with a larger disk of their VM ((C8), §5.3); with none in the group they are pinned. Do not assume `drive-mirror` semantics for `tpmstate0`; §8.1's both-storages invariant holds either way (§3.6)
VM has disks on `ide`/`sata`/`virtio`, not just `scsi`	Full bus regex in §3.5; enumerating only `scsi*` silently mis-accounts capacity
`unused{N}` volumes	Movable with $\ell_d = 0$, so the solver relocates them only to repair a reserve violation — the intended policy
VM is `locked` (backup, snapshot, migrate, ...)	Pinned at planning time, waited for at execution time up to `execution.locks.wait_timeout`; the lock value set is treated as open-ended and never whitelisted (§9.3)
Source space not reclaimed after a successful move	`saferemove` zeroes the old volume at ~10 MiB/s; the move stays in the `draining` state and keeps charging the source until the volume is observed gone (§8.2, §9.3)
Next move blocked by the previous move's wipe	Completion requires task OK **and** source volume absent **and** lock clear; `cooldown_per_storage` validated against the wipe time (§9.3)
Thin provisioning	**Never considered** (§5.1): every disk counts at its provisioned size, so a thin pool's "used" is $\Sigma\ z_d + U^{ext}$, which can be several times what the pool reports allocated. `migration.assume_thick_provisioning` survives only as an accepted-`true`, refused-`false` key; there is no allocated-size mode
Foreign volumes on a storage	Counted via `count_foreign_volumes`; otherwise the reserve silently overstates free space
A move converts a disk's format	Only onto a storage with `enforce_format` set, only as a side effect of a move the plan makes anyway, never for `tpmstate0`, never for a disk with snapshots (§5.3.2); the converted size $z_\{d,s\}$ is what the reserve and the transient invariant count; the conversion is printed on the move line
Orphaned target volume after a failure	Detected and reported, never auto-deleted (§9.3)
Storage already violating the reserve	Soft slack r_s keeps the model feasible; violation bypasses gates and is scheduled first
Storage holding several disks of one VM	The reserve is the largest per-VM footprint, all of that VM's disks there summed (§5.3.3) — one snapshot covers every disk at once. After the upgrade from the per-disk rule a storage can show a shortfall it did not have before; it is repaired like any other, so read `show-load` in dry-run first

Hazard	Handling
Large VM on one storage, plenty of space	The split rule (§5.3.3) spreads a VM above `snapshot_reserve.split_vm_footprint` so that no storage holds more of it than the cap, or the smallest peak the storages allow; the split gate (§6) opens the group for it; `null` switches it off
Storage below its configured free-space requirement (`free_space.soft`, §5.3.1)	Same handling as a reserve violation: slack keeps the model feasible, the §6 override bypasses drift/imbalance, a repairing plan is payback-exempt (§7.3) and the *direct* repair — the move off the short storage — is scheduled first (§8.2 exception 1); an *indirect* repair, one that frees the space the direct repair needs, waits for the spec-only exception 2, as §14.8's second move shows; an unrepairable shortfall is reported with the byte amount
Group I/O-balanced but data concentrated on few storages	The capacity gate (§6) triggers planning anyway and the `δ` term (§5.4) does the spreading; it still honours cooldowns, payback and the transient invariant
Plan's entire value is affinity repair (`Δimbalance ≈ 0`)	Affinity improvement counts in the payback benefit and tiny moves are payback-exempt (§7.2, §7.3); acting with near-zero balance benefit is a correct outcome, not a defect
Two DRS instances running	Advisory lock in `state.json` plus a startup scan for in-flight `move_disk` UPIDs owned by the DRS user. The lock is node-local; only the UPID scan crosses the cluster (§11)
Config edited mid-run, cluster-wide	The config is read once at startup and never re-read; the resolved path and its SHA-256 are logged, so a plan can be traced to the exact file that produced it
Storage `/…/` pattern matches nothing in the cluster	Hard error before planning, exactly like a literal id that does not exist: the likelier cause is a typo, and the alternative is a silently shrunken group (§11.4)
Cluster storage set changes after the config is written	Patterns are re-expanded against the live inventory every run, so a new LUN joins its group with no config edit; the expansion is logged and shown by `verify-storages` (§11.4)
Solver infeasible or timing out	Fall back to the heuristic; never emit a partial/unvalidated assignment
`bwlimit` misunderstood	It is **KiB/s** in the API; config is bytes/s and must be converted
PVE Dynamic Load Balancer moves a VM mid-plan	Node re-fetched before every move (§9.2); mismatch triggers a bounded re-plan
Engine crashes mid-move	`inflight_upids` in `state.json` + startup scan; the PVE task continues regardless
Concurrent moves onto one storage	Generalized transient invariant over the in-flight set (§8.1)
Config knob with no effect	§11.1 validation; every knob maps to exactly one formula (§15)
A diagnostic bundle carries an identifier or a secret	Allowlist, never denylist: a field reaches a bundle only if `anonymize.py` names it, and an identifier with no mapping drops its whole record rather than passing through. The corpus scrub audit re-checks every committed bundle against the same allowlist plus IP/email/IQN/PEM/hex patterns, on the assumption that the collector has a bug (§16.3, §16.6)
Two unrelated clusters produce the same pseudonyms	The salt is 32 bytes of `os.urandom()` persisted per host, never `/etc/machine-id` — which is routinely cloned by templates and golden images and would turn the mapping into a cross-bundle correlation key (§16.3)
A bundle replays to a different plan than the live run	The replay clients key on the anonymized query text the engine itself regenerates, and a miss is a loud error naming the query and the range, never an empty result (§16.5). `collect-testdata` captures the superset of every forecaster's `required_range()`, not the configured model's (§16.2)

Hazard	Handling
A bundle looks complete but a permission silently emptied a response	Every captured call records its outcome (`ok`/`http_error`/`empty`/`refused`/`skipped`) and the bundle ships `verify-metrics` findings from capture time (`verify-storages`'s own report replays from the topology/config the bundle already carries, needing no separate capture) — the `Datastore.Allocate` false negative of §3.5 must be visible in the bundle, not inferred from it (§16.2)

Overarching rule: **the reserve constraint is never traded against balance.** Stated precisely: with the lexicographic solve of (C5) this is exact — the reserve shortfall is minimized in a prior stage that the balance objective cannot influence at any weight. With the single-stage big-M alternative it is *effectively* rather than *provably* hard, because `P` is a calibrated constant; §5.3 gives the bound `P` must clear. Either way `Σ r_s > 0` means physically impossible, not merely unattractive, and (C5) is enforced before, during and after every move. The same rule covers the configured free-space requirement (§5.3.1): `soft_s` enters the same `max()`, the same slack, the same lexicographic stage, so a byte of configured free space is exactly as non-negotiable as a byte of snapshot reserve — and `hard_s` is the floor the transient states of §8.1 may not cross.

---

14. Worked example

A complete, self-consistent fixture. Implementations must reproduce these numbers exactly.

The machine-readable form lives in **`tests/fixtures/fc-tier1.yaml`** (input) and **`tests/fixtures/fc-tier1.expected.json`** (expected derivations for every `(β, δ)` pair in the input's `beta_values` × `delta_values` sweeps, the execution order with its transient checks, the post-plan reserve state, and both payback calculations). Assert against those files in CI rather than transcribing the tables below.

The expected file is **generated, not written**: `tests/fixtures/generate_expected.py` enumerates all $3^6 = 729$ assignments per `(β, δ)` pair, so the recorded optimum is proven rather than hand-worked, and derives the order with the §8.2 rule and the §8.1 transient predicate. Run it with `--check` in CI to assert the committed file is current; that check is also the regression test for §5.5's coefficient scaling, since a scaling bug shows up as a different optimum.

This example cannot test everything, and one gap is worth naming. In `fc-tier1` every reserve-violating assignment is *also* worse on balance, so the lexicographic solve and the single-stage big-M solve agree here at **any** `P ≥ 0` — the fixture simply never exercises the distinction the two options of §5.3 exist to make. `tests/fixtures/reserve-tradeoff.yaml` is the companion fixture that does; see §14.6. §14.7's `affinity-repair.yaml` covers a third gap of the same kind: the payback rule's treatment of plans whose value is affinity. §14.8's `free-space-repair.yaml` covers a fourth: the free-space repair mandate of §5.3.1, a plan payback rejects and the requirement executes anyway. §14.9's `large-vm-split.yaml` covers a fifth: the large-VM split rule of §5.3.3, on a group with nothing else to fix.

14.1 Input

Group `fc-tier1`, three storages of 8.0 TiB each, all `capability_weight = 1.0`, `f = 2.0`, no foreign volumes. `config/drs.example.yaml` uses the same group and storage names with the same weights, so the example config and this fixture agree; the differing-weight feature is illustrated on `fc-tier2` there instead. ℓ_d is in average in-flight I/O requests (§4), *not* normalized to sum to one — which is what makes the `ω = 1.0` per mirror endpoint in §14.5 commensurable with it.

The split rule is off here (`split_vm_footprint: null`, stated in the fixture like every other weight). This

is the balancing example, and §14.3 uses VM 101 to demonstrate the I/O weighting `w_v`; at the default 2 TiB cap VM 101 (3.0 TiB in total) would be in `V^split` with $w_{101} = 1$ (§5.3.3), every plan below that splits it would score $\kappa \cdot (2.703 - 1) = 0.851$ lower, and the optimum would be the same two moves. The split rule has its own fixture, §14.9. The per-VM footprint of §5.3.3 is *not* a knob and applies here as everywhere: it is what makes san-a's violation 2.5 TiB rather than the 0.5 TiB the per-disk rule computed.

Disk	VM	`z_d` (TiB)	ℓ_d	On
`101:scsi0`	101	2.0	3.0	san-a
`101:scsi1`	101	1.0	1.0	san-a
`102:scsi0`	102	1.5	2.5	san-a
`103:scsi0`	103	0.5	0.4	san-b
`104:scsi0`	104	1.0	0.3	san-b
`105:scsi0`	105	0.5	0.2	san-c

$\Sigma\ell = 7.4$, so `u*` $= 7.4 / 3 = 2.4667$.

14.2 Initial state

`Z_s` is the largest per-VM footprint (§5.3.3): on san-a, VM 101's two disks together.

Storage	`L_s`	used	`Z_s`	`used + f·Z_s`	vs `C_s`
san-a	6.50	4.5	3.0 (VM 101: 2.0 + 1.0)	**10.5**	8.0 → **violates by 2.5 TiB**
san-b	0.70	1.5	1.0	3.5	8.0 ✓
san-c	0.20	0.5	0.5	1.5	8.0 ✓

- Spread $(6.50 - 0.20)/2.4667 = 2.554$ → **255%**, far above the 20% gate.
- `E_before = |6.50−2.4667| + |0.70−2.4667| + |0.20−2.4667| = 4.0333 + 1.7667 + 2.2667 = 8.0667`
- Data spread (§5.3 (C7)): fills 56%/19%/6% against $\bar{b} = 0.2708$, so each storage's fill target is $\bar{b} \cdot$ `C_s` $= 2.167$ TiB and `F_before = 2.333 + 0.667 + 1.667 = 4.667 TiB` — san-a alone holds 69% of the group's bytes. $(0.5625 - 0.0625)/0.2708 = 1.85$, far above the 25% capacity gate.
- san-a already breaches the snapshot reserve. This is why (C5) carries slack `r_s` rather than being hard — a hard constraint would report *infeasible* here and refuse to help. Under the per-disk rule of earlier revisions the breach was 0.5 TiB ($4.5 + 2 \times 2.0$); summing VM 101's disks makes it 2.5 TiB, and makes VM 101 impossible to keep whole on *any* storage of this group: its 3.0 TiB plus 2×3.0 of reserve is 9.0 TiB, more than a whole 8 TiB storage.

14.3 Solution

With default weights (α=1.0, β=0.25, γ=0.05/TiB, κ=0.5, δ=0.5) the optimum is **two** moves:

1. `102:scsi0` san-a → san-c
2. `101:scsi1` san-a → san-b

Storage	`L_s`	used	`Z_s`	`used + f·Z_s`	fill	✓
san-a	3.00	2.0	2.0	6.0	25%	✓
san-b	1.70	2.5	1.0	4.5	31%	✓
san-c	2.70	2.0	1.5	5.0	25%	✓

`E_after = 0.5333 + 0.7667 + 0.2333 = 1.5333`, spread (3.00–1.70)/2.4667 = **53%**; the fill deviation drops from `F_before = 4.667 TiB` to `F_after = 0.167 + 0.333 + 0.167 = 0.667 TiB`. The reserve violation is repaired. The footprint rule leaves this optimum, and every other optimum the fixture records,

exactly where the per-disk rule had it: every one of them already splits VM 101, so its final `Z_s` values are the same under both rules, and the assignments the footprint rule newly marks short are ones that keep VM 101 whole, none of which was optimal.

The β knob, demonstrated at δ = 0. Switch the capacity term off and a third move becomes worth taking: `105:scsi0 san-c → san-b` improves the imbalance by `1.5333 − 1.1333 = 0.400`, and its objective contribution is

```
α·ΔE + β·1 + γ·0.5 TiB  =  −0.400 + 0.250 + 0.025  =  −0.125   → accepted at β=0.25
                        =  −0.400 + 0.500 + 0.025  =  +0.125   → rejected at β=0.50
```

So with `delta_capacity_spread: 0` the solver returns the **three-move** plan — `(3.00, 1.90, 2.50)`, `E = 1.1333`, spread 44.6%, fills 25%/38%/19% — for every `beta_move_count < 0.375`, and the two-move plan above for everything past it. This is exactly the "minimal number of migrations" trade-off made explicit and tunable; both plans are correct, and β chooses.

The δ knob, demonstrated at the defaults. The third move concentrates data — san-b's fill rises to 38% while san-c's falls to 19%, `F` goes from 0.667 to 1.667 TiB, `δ·ΔF = +0.500` — and that regression outweighs the move's net I/O gain:

```
α·ΔE + β·1 + γ·0.5 TiB + δ·ΔF  =  −0.400 + 0.250 + 0.025 + 0.500  =  +0.375   → rejected
```

The two-move plan is therefore the optimum at the defaults (full objectives: 3.843 against 4.218 — both split VM 101, so both carry the same $\kappa \cdot w_{101} = 1.351$ and this particular comparison is unchanged by the weighting), and it is also ahead on migrations (2 < 3), bytes moved (2.5 < 3.0 TiB) and data spread. That is the intended shape of the term: α decides how much imbalance to remove; δ helps decide when a further move would concentrate data more than it evens load. (At `beta_move_count: 0.5` the two-move plan wins with or without δ.)

The affinity trade-off, demonstrated. The two-move plan splits VM 101 (`scsi0` on san-a, `scsi1` on san-b). VM 101 carries $\ell_v = 4.0$ against a group mean of $\bar{\ell} = 7.4/5 = 1.48$, so $w_{101} = 2.703$ and the split incurs $\kappa \cdot w = 1.351$ where an unweighted term would charge `0.5` — the group's heaviest VM is worth 2.7 average ones to keep together (§5.4). Under the footprint rule the reserve settles the question before κ is consulted: VM 101 whole needs 9.0 TiB on one 8 TiB storage (§14.2), so every assignment that keeps it together carries a shortfall, and stage 1 of the lexicographic solve discards them all. The comparison is still worth making on the persistent terms, because it shows what the weighting does wherever space does *not* decide (it did here under the per-disk rule, with VM 101 whole on san-c within its reserve). Keeping VM 101 together forces san-a to `L = 4.0` and the best reachable `E` becomes `3.133`. Comparing $E + \kappa \cdot w + \delta \cdot F$: $3.133 + 0 + 0.5 \cdot 1.667 = 3.97$ (together) against $1.5333 + 1.351 + 0.5 \cdot 0.667 = 3.22$ (split). Splitting still wins — high I/O legitimately overrides the preference — but by 0.75 rather than the 1.6 an unweighted κ would have given, and a somewhat busier VM 101 would flip the comparison entirely, which is what the weighting is for.

14.4 Ordering

`102:scsi0` is scheduled first. Both moves leave san-a while it is violating (C5), so both rank under §8.2's first exception, and among them `102:scsi0` has the larger persistent-objective reduction per unit cost (`6.03` per `15 729 load·s` against `1.32` per `10 486` — VM 101's fragmentation is charged to the second move). It does **not** repair san-a on its own: with `102:scsi0` gone san-a holds 3.0 TiB, all of it VM 101, so $3.0 + 2 \times 3.0 = 9.0 > 8.0$ — still 1.0 TiB short until `101:scsi1` leaves and san-a's footprint drops to 2.0 ($2.0 + 4.0 = 6.0$). (Under the per-disk rule `102:scsi0` alone repaired it, $3.0 + 4.0 = 7.0$; the order is the same either way.) Transient checks for the two-move plan:

```
move 1 → san-c:  used 0.5 + 1.5 = 2.0,  max(Z_c, 1.5) = 1.5,  2.0 + 3.0 = 5.0 ≤ 8.0  ✓
move 2 → san-b:  used 1.5 + 1.0 = 2.5,  max(Z_b, 1.0) = 1.0,  2.5 + 2.0 = 4.5 ≤ 8.0  ✓
```

The `δ = 0` three-move variant appends `105:scsi0 → san-b — used 2.5 + 0.5 = 3.0, max(Z_b, 0.5) = 1.0, 3.0 + 2.0 = 5.0 ≤ 8.0 ✓` — recorded alongside in the expected file's `delta_values` sweep.

14.5 Payback

At `bwlimit = 200 MiB/s`, `ω_src = ω_dst = 1.0`, `H = 365d = 31 536 000 s`, `λ = 10`, and **`saferemove` off on all three storages** so `duration_wipe_d = 0` (§7.1), for the two-move plan:

That last assumption is not a detail of the prose — the config default is `migration.account_saferemove_wipe: true`, and with a wipe at the PVE default of 10 MiB/s the 1.5 TiB move below would carry ~44 h of zeroing on top of its 2.2 h mirror and the arithmetic would look nothing like this. So the fixture states it in the data rather than leaving it to be inferred: `saferemove: false` on each storage, `account_saferemove_wipe: false` in the `migration` block, and an `assumptions` object echoed into the expected file. The per-move records carry `duration_mirror_seconds` and `duration_wipe_seconds` separately, so a run with wiping enabled is a different number in a named field rather than a silent discrepancy.

Move	Size	Duration	`cost = 2 × duration`
`102:scsi0`	1.5 TiB	7 864 s (2.18 h)	15 729 load·s
`101:scsi1`	1.0 TiB	5 243 s (1.46 h)	10 486 load·s
		Σ	**26 214 load·s**

```
benefit = (α·ΔE + δ·ΔF + κ·ΔA) · H
        = (6.5333 + 0.5 × 4.0000 − 0.5 × 2.7027) × 31 536 000
                            (ΔF = 4.6667 − 0.6667 TiB;  ΔA = 0 − 2.7027: the plan splits
                             VM 101 — the group's heaviest, w₁₀₁ = 2.703 — and §7.2
                             charges that fragmentation against the benefit)
        = 7.1820 × 31 536 000 = 226 490 984 ≈ 2.26×10⁸ load·s
ratio   = 226 490 984 / 26 214 ≈ 8 640   ≥ λ = 10   → ACCEPT
```

A move that fails payback. Consider instead a 4.0 TiB archive disk with `ℓ = 0.1` whose relocation would improve `E` by only 0.01 and leave the data spread essentially unchanged:

```
cost    = 2 × (4.0 TiB / 200 MiB/s) = 2 × 20 972 = 41 943 load·s
benefit = 0.01 × 31 536 000 = 315 360 load·s
ratio   = 7.5   <  λ = 10   → REJECT
```

Even a full year of accumulated benefit does not pay for this migration. This is the requirement that "migrating a very large disk might generate more traffic than we are trying to save", enforced numerically — and it is the horizon that sets the bar: at the previous `7d` default this same test demanded `ΔObj ≥ 0.69` for a disk of this size (`0.17` per TiB moved) and rejected a large share of the moves that were worth making, which is why `H` is configurable and why its default is now stated as an explicit assumption about how long a placement lasts (§7.2).

14.6 Companion fixture: when the reserve and the balance objective disagree

`tests/fixtures/reserve-tradeoff.yaml` is a second, deliberately awkward group, and its only job is to separate the two solve paths of §5.3.

Two storages: `roomy` (20 TiB, empty) and `cramped` (5 TiB, of which 3 TiB is already U^{ext} — volumes DRS does not manage). `f = 2.0`. Two disks, both 1.0 TiB, both `ℓ_d = 5.0`, both on `roomy`, belonging to different VMs. So `u* = 5.0`, and:

Assignment	Σ r_s	E	Non-reserve objective at β = 0.25, δ = 0.5
both on roomy (current)	**0**	10.0	12.00
one moved to cramped	1.0 TiB	**0.0**	3.30
both moved to cramped	2.0 TiB	10.0	14.60

Moving one disk balances the group *perfectly* and costs a 1 TiB reserve breach on cramped (3 + 1 + 2·1 = 6 > 5). That is the trade the reserve rule exists to forbid, and the three answers are:

- **Lexicographic (the default).** Stage 1 finds min Σ r_s = 0, stage 2 optimises within that set — so the plan is *no moves at all*, and the group stays at E = 10. Correct, and it needed no calibration to be correct.
- **Big-M at the configured P = 1000.** Same answer: 3.30 + 1000 > 12.00.
- **Big-M at P = 5.** 3.30 + 5 = 8.30 < 12.00, so it moves the disk and breaches the reserve for balance. The exact flip point, recorded in the expected file as big_m_agreement_threshold_p, is **P = 8.70**: below it big-M is wrong, above it big-M is right. Note how small that number is — nothing about P = 5 looks obviously wrong to an operator, which is the whole argument for computing P rather than configuring it. (The δ term raises the moved plan's objective — the group's fill targets are 4 TiB on roomy and 1 TiB on cramped, so moving a disk onto cramped, which already holds 3 TiB of foreign data, takes F from 4 to 6 TiB — and the gap between the two plans narrows from 9.70 to 8.70, but not enough to matter at any sane P.)

The build-time bound of §5.3 gives P_min = 26.6 · 2^{20} ≈ 2.79×10^{7} for this group, four orders above the 8.70 actually needed. That is the bound doing its job: it is deliberately worst-case (it refuses to trade even one mebibyte), and being conservative in the safe direction costs nothing.

One further check the fixture records: the P = 5 plan is not merely undesirable, it is **unschedulable**. Its single move fails the §8.1 transient predicate on cramped, so §8.2 reports a deadlock rather than emitting it. The two safety mechanisms are independent, and the fixture asserts that both fire.

Disk 201:scsi0 and 202:scsi0 are interchangeable here; the recorded move names 202:scsi0 because that is how the enumerator breaks the tie. An implementation may pick either — assert on the move *count* and the resulting slack, not on the disk identity.

14.7 Companion fixture: affinity repair under the payback rule

tests/fixtures/affinity-repair.yaml isolates what §5.4's strengthenings and §7.2's corrected benefit exist for: a group whose I/O is perfectly balanced and whose only improving moves are a VM's tiny disks reuniting with it. Three 8 TiB storages stor-a/stor-b/stor-c, f = 2.0, equal capabilities, saferemove off:

Disk	VM	z_d	ℓ_d	On
301:scsi0	301	1.0 TiB	2.0	stor-a
301:efidisk0	301	528 KiB	0.0	stor-c
301:tpmstate0	301	1 MiB	0.0	stor-b
302:scsi0	302	1.0 TiB	2.0	stor-b
309:scsi0	309	1.0 TiB	2.0	stor-c (pinned: exclude.vmids)

plus 4.0 TiB of foreign volumes on stor-c. Loads read 2.0/2.0/2.0, so the imbalance gate stays shut; fills read 0.125/0.125/0.625 against $\bar{b}$ = (3.0 + 4.0)/24 = 0.2917, so the capacity gate fires on a spread of (0.625 − 0.125)/0.2917 = 1.714. VM 301 sits on three storages; $\bar{\ell}$ = 6.0/3 = 2.0, so w_{301} = 1.

The optimum is exactly two moves — 301:efidisk0 stor-c → stor-a and 301:tpmstate0 stor-b →

stor-a — worth $\kappa \cdot w_{301} \cdot 2 = 1.0$ at zero β/γ cost. The alternatives all improve less: joining scsi0 to its tiny disks costs $\beta + \gamma \cdot 1.0$ TiB $= 0.3$ for half the affinity gain and a large fill-deviation regression, and every other byte-moving candidate worsens δ more than it helps anything. Payback accepts with benefit $\approx \kappa \cdot \Delta A \cdot H = 3.15 \times 10^7$ load·s against cost = 0.

This fixture pins §5.4/§7.2's *value*, not a verdict flip: the affinity repair is worth 31 536 000 of the plan's benefit_load_seconds: 31 536 015.51 ($\kappa \cdot \Delta A \cdot H$, 99.99995 % of it) at exactly zero cost, and test_affinity_repair_fixture.py asserts that by value rather than by pass/fail. It is **not** the fixture that flips reject-to-accept — replayed through the pre-§7.2 formula (no $\kappa \cdot \Delta A$ term, tiny_disk_bytes = 0 so both tiny disks are charged their real, nonzero mirror cost), this same two-move plan still *accepts*: $\Delta E = 0$ exactly, ΔF is positive (the tiny disks leave above-mean storages for a below-mean one), giving benefit $\approx \delta \cdot \Delta F \cdot H = +15.51$ load·s against cost ≈ 0.0152 load·s — ratio ≈ 1 020, the same ✓ the fix produces. The verdict flip the live cluster actually suffered belongs to the *pre-§12* formula alone (benefit = $\Delta E \cdot H$ at the old 7d horizon, §12's phase table): that formula scores this plan benefit $= 0 < \lambda \cdot$ cost and rejects it. What a silent revert of κ/w_v does catch on this fixture is the test's exact-value assertions (benefit ≈ 31 536 015.51, cost_load_seconds == 0.0), not the verdict.

14.8 Companion fixture: the free-space repair mandate

tests/fixtures/free-space-repair.yaml isolates what §5.3.1's requirement and §7.3's repair exemption exist for: a group whose I/O is **perfectly balanced**, whose snapshot reserve is satisfied everywhere, and where a single storage sits below its configured free-space requirement because an admin placed foreign volumes on it. The engine must repair it anyway — the Storage-DRS cluster function — and the repair is exactly the kind of plan the payback rule would reject on its own.

Three 10 TiB storages packed/roomy/swapme, f = 2.0, equal capabilities, saferemove off, free_space.soft: "30%" (3.0 TiB) on all three, free_space.hard swept by the fixture: null (= 3.0) and "10%" (1.0 TiB). swapme accepts only raw volumes — (C2) format eligibility — which is what makes the repair a *two*-move plan rather than a one-move one.

Two prerequisites this fixture states openly rather than assumes. First, the (C2) format-compatibility rule it leans on. *It was not implemented when the fixture was specified* — topology.Storage did not expose storage type/format, so both solver backends treated every group storage as an eligible target for every disk — and under that behaviour the fixture's optimum would have been the *one*-move repair 601:scsi0 packed → swapme (objective 1.081 under the relative-fill (C7) of that time — and then **not** a plan that paid for itself: it relocates the quiet 0.05 disk off a perfectly balanced group, so E: 0 → 0.10 against a relative F: 1.608 → 1.412, benefit $(-0.100 + 0.5 \times 0.196) \times 31\,536\,000 \approx -6.2 \times 10^4$ load·s against a 5 243 load·s cost, ratio -11.8, clearing the gate only through the same repair exemption the two-move plan needs — its own Σ r_s is 0.5 → 0, so both the built has_reserve_override and §7.3's outcome trigger fire on it. Under the absolute-TiB (C7) it would pay on its own: objective 3.975, F: 8.2 → 7.2 TiB, benefit $(-0.100 + 0.5 \times 1.0) \times 31\,536\,000 \approx 1.26 \times 10^7$ load·s, ratio ≈ 2 406). What depends on the format rule is the plan's *shape*, not its economics: the two-move repair, the indirect repair the revert test marks, and both hard-sweep orders below. Capacity alone cannot substitute for the format rule: §8.1's predicate is monotone in z_d, so any storage that accepts the 1.0 TiB 603 accepts the 0.5 TiB 601, and pinning 603 (the affinity-repair.yaml mechanism) does not help either — it leaves 601 → swapme legal and optimal. **As built (AH-03):** phase 13 landed the rule (Storage.storage_type/allowed_formats, one shared storage_accepts_format(), both backends fixing x_{d,s}=0) *in the same commit* as the fixture, so the one-move counterfactual above is history rather than a state the fixture can observe — which is why the expected file carries no requires_format_eligibility marker: its only job was to fail loudly on a run *before* the rule existed, and no such run can exist. The one-move figures survive as the hand derivation in REVIEW.md §48 (AG-01), not as a recorded case. Second, the group's fills (0.75/0.68/0.10) put the capacity gate at (0.75 − 0.10)/0.51 = 127%, far above the 25% default — the gate is what lets the engine *plan* here, and

the fixture sets `gates.capacity_spread_threshold: null` to keep that planning decision from being the fixture's own doing: with the gate open, the soft-less counterfactual below is exactly §9.5's "gate says ACT, solver's optimum is to move nothing" case, pinned deliberately.

Disk	VM	z_d	ℓ_d	format	On
`601:scsi0`	601	0.5 TiB	0.05	qcow2	packed
`602:scsi0`	602	1.0 TiB	2.00	qcow2	packed
`603:scsi0`	603	1.0 TiB	2.05	raw	roomy
`604:scsi0`	604	1.0 TiB	2.05	raw	swapme

plus foreign volumes — the admin's new VMs, which DRS does not manage — of 6.0 TiB on `packed` and 5.8 TiB on `roomy`. Initial state:

Storage	used	Z_s	$f \cdot Z_s$	$soft_s$	R_s	free	r_s
packed	7.5	1.0	2.0	3.0	3.0	2.5	**0.5**
roomy	6.8	1.0	2.0	3.0	3.0	3.2	0
swapme	1.0	1.0	2.0	3.0	3.0	9.0	0

Loads read 2.05/2.05/2.05 — $E = 0$, the drift and imbalance gates shut — and the snapshot reserve alone is satisfied everywhere (`packed`: 7.5 + 2.0 = 9.5 ≤ 10). Only the configured free space is violated, by 0.5 TiB, on `packed` alone. This is the case the removed `min_free_bytes` could not express as a *per-storage policy* and could not *repair*: nothing distinguishes it from a healthy cluster except the requirement. (The gate's ACT is what lets the engine plan — see the capacity-gate prerequisite above — and the requirement is what makes the solver *move*. The isolation claim is about the solver's objective, not the gate stack.)

The optimum is two moves, and it is a repair, not a balance. Exhaustive enumeration over all format-feasible assignments confirms the lexicographic optimum is unique:

1. `601:scsi0 packed → roomy` — the quiet 0.5 TiB disk, chosen over the 1.0 TiB `602` by γ (both repairs cost $\beta \cdot 1$; the smaller disk moves fewer bytes);
2. `603:scsi0 roomy → swapme` — **not because `roomy` violates anything**: `roomy` ends at 6.3 used, 3.7 free, compliant. It moves because `601` cannot land on `swapme` (qcow2 on a raw-only storage, (C2)), and `roomy` cannot end below its own `soft` with `601` added on top of `603` (6.8 + 0.5 + 3.0 = 10.3 > 10). The move is an *indirect repair* — it empties the destination the direct repair needs — which is why §7.3's per-move marker is a revert test rather than "source was in violation": holding `603` back raises the plan's final $\Sigma\ r_s$ from 0 to 0.3 (`roomy` at 7.3 used, 2.7 free, short by 0.3), so it is marked `repair: true` even though `roomy` never violated anything.

Storage	used	Z_s	R_s	free	r_s
packed	7.0	1.0	3.0	3.0	0
roomy	6.3	0.5	3.0	3.7	0
swapme	2.0	1.0	3.0	8.0	0

$\Sigma\ r_s$: 0.5 → 0. The counterfactual is recorded in the expected file: with `free_space.soft: 0` on the same cluster, the optimum is **no moves at all** (objective 4.100 against the repair's 7.775) — the requirement, and nothing else, is what makes the solver move. (With the capacity gate nulled, that counterfactual is the engine declining to act at all; with the gate left at its default it is §9.5's ACT-but-no-moves case, reported by `explain`'s closest-alternative block.)

Payback rejects this plan, and the exemption is what executes it. The repair wrecks the balance the

cluster started with: loads become 2.00/0.05/4.10, `E: 0 → 4.1`, and the fill spread improves only from `F = 8.2 TiB` to `F = 6.2 TiB` (targets 5.1 TiB on each storage):

```
benefit = (α·ΔE + δ·ΔF) · H = (−4.100 + 0.5 × 2.0) × 31 536 000 = −9.78×10⁷ load·s
cost    = 2 × (0.5 TiB/200 MiB/s) + 2 × (1.0 TiB/200 MiB/s) = 5 243 + 10 486 = 15 729 load·s
ratio   = −6 216   <  λ = 10   → REJECT — overridden: the plan repairs (Σ r_s 0.5 → 0), §7.3-exempt
```

A negative benefit is the point: this plan moves a 0.05-load disk off a perfectly balanced group and concentrates 4.10 of load on `swapme`. Only a δ above 2.05 per TiB — four times the default, pricing data spread above I/O balance — would even make its benefit positive, and no payback horizon can make it pay — the mandate is the only thing that produces it, which is precisely why the exemption must be structural (§5.3) rather than a weight.

The `hard` sweep, and why the order flips. §8.2 schedules `601 → roomy` first by its exception 1 — it is the move that resolves `packed`'s (C5) violation — with `603 → swapme` behind it: exception 2 applies to that move too (it frees the `roomy` space the first move needs), but rule 1 outranks rule 2. Note the ratio itself would rank both moves *last*: a repair's persistent-objective reduction is negative here (§8.2's α/δ/κ terms all get worse), which is exactly why the exceptions exist as priorities rather than as weights. (As built, only exception 1 is implemented — `schedule.py`'s `_resolves_reserve_violation` — and exception 2 remains spec-only; the flip demonstrated here needs only exception 1 plus the §8.1 predicate, so the fixture does not depend on the gap.) The transient check on `roomy` at that moment: `603` has not moved yet, so `roomy` holds 6.8 + 0.5 incoming = 7.3 used, and the predicate demands `7.3 + max(2·1.0, hard) ≤ 10`:

- `hard: null` (= `soft` = 3.0): `7.3 + 3.0 = 10.3 > 10` — **infeasible**. The scheduler reverses the order: `603 → swapme` first (its own check on `swapme` is `1.0 + 1.0 + max(2·1.0, 3.0) = 5.0 ≤ 10` ✓), which empties `roomy` to 5.8 used, and then `601` lands at `5.8 + 0.5 + max(2·max(0, 0.5), 3.0) = 5.8 + 0.5 + 3.0 = 9.3 ≤ 10` ✓ — note `Z_roomy` has dropped to 0 with `603` gone (the 5.8 TiB left on it is foreign, and `601` has not arrived yet), so the inner term is `max(0, 0.5) = 0.5` and the `hard` floor of 3.0 dominates the snapshot term either way. The plan still executes — the requirement is met either way, at the cost of the less preferred order.
- `hard: "10%"` (= 1.0 TiB): `7.3 + 2.0 = 9.3 ≤ 10` — feasible as preferred. `roomy` dips to **2.7 TiB free**, below its 3.0 soft requirement from `601`'s target allocation until `603`'s source volume is observed gone — the whole repair, both mirrors and the drain between them, not just the first mirror — and never below the 1.0 hard floor; the plan's endpoint pulls it back to 3.7. (The reversed order's second move is looser still: `5.8 + 0.5 + max(1.0, 1.0) = 7.3 ≤ 10`.)

Both orders end compliant; the expected file records `expected_order` per `hard` value, and the `hard: "10%"` entry is the fixture's proof that a transient dip below `soft` is a real, schedulable state rather than a contradiction of (C5) — the dip is on a storage the finished plan leaves compliant, exactly as §8.1 requires.

One further assertion the fixture makes: with `free_space.hard` above `free_space.soft` on any storage, `config.py` rejects the file (§11.1) — the sweep values are chosen so the invalid configuration is a one-token edit away, and the error is part of the test.

14.9 Companion fixture: splitting a large VM

`tests/fixtures/large-vm-split.yaml` isolates §5.3.3's split rule: a group with balanced I/O, every reserve satisfied, and one large VM sitting whole on one storage. Nothing but the split rule can make the engine move, so the fixture shows the rule, the split gate and the payback term on their own.

Three 40 TiB storages `st-a`/`st-b`/`st-c`, `f = 2.0`, equal capabilities, `saferemove` off, `split_vm_footprint: 2 TiB`, `μ = 1.0`, and — to keep data spread from doing any of the work — `delta_capacity_spread: 0` and `gates.capacity_spread_threshold: null`, stated in the fixture:

Disk	VM	z_d	ℓ_d	On
701:scsi0 … 701:scsi4	701	2.0 TiB each	0.0	st-a
702:scsi0	702	1.0 TiB	1.0	st-a
703:scsi0	703	1.0 TiB	1.0	st-b
704:scsi0	704	1.0 TiB	1.0	st-c

VM 701 is a quiet 10 TiB file server: five disks, no I/O worth balancing. Loads read 1.0/1.0/1.0, `E = 0`, so the drift and imbalance gates stay shut. Every reserve is satisfied — st-a holds 11 TiB and keeps `2 × 10 = 20 TiB` free for VM 701's snapshot, `31 ≤ 40` — so there is nothing to repair either. VM 701 is in `V^split` (`10 > 2`), T_{701} `= max(2, 2) = 2`, and its excess is o_{701} `= 10 − 2 = 8`.

The split gate opens the group: moving one of VM 701's disks to st-b lowers its peak from 10 to 8 TiB and leaves st-b compliant (§6).

The optimum is three moves, two of VM 701's disks to one storage and one to the other, leaving footprints 4, 4, 2 — the best split three storages allow (§5.3.3's table). With α and δ silent, the candidates differ only in $\mu \cdot o_{701} + \beta \cdot \text{moves} + \gamma \cdot \text{bytes} + \kappa \cdot w_{701} \cdot (\text{storages} - 1)$, and w_{701} `= 1`:

Disks of 701 on (a, b, c)	Footprints	o_{701}	Objective
5, 0, 0 (current)	10, 0, 0	8	**8.00**
4, 1, 0	8, 2, 0	6	6 + 0.35 + 0.5 = 6.85
3, 2, 0	6, 4, 0	4	4 + 0.70 + 0.5 = 5.20
3, 1, 1	6, 2, 2	4	4 + 0.70 + 1.0 = 5.70
2, 2, 1	**4, 4, 2**	**2**	2 + 1.05 + 1.0 = **4.05**
1, 2, 2	2, 4, 4	2	2 + 1.40 + 1.0 = 4.40

Each move costs `β + γ·2 TiB = 0.25 + 0.10 = 0.35`. `1, 2, 2` reaches the same peak with one more move, and loses on β/γ — the rule spreads only as far as the cap requires. Moving any of the single-disk VMs only adds imbalance. `2, 2, 1` and `2, 1, 2` are the same plan with st-b and st-c exchanged, and score identically; the expected file records the enumerator's tie-break, and — as for §14.6 — the harness asserts the footprint multiset `{4, 4, 2}` and the move count, not which storage got two disks or which of the five interchangeable disks moved.

Final reserves, all compliant, VM 701's footprint the largest on each storage: st-a and the two-disk storage `5 + 2 × 4 = 13 ≤ 40`, the one-disk storage `3 + 2 × 2 = 7 ≤ 40`. The fullest storage now keeps 8 TiB free for VM 701 instead of 20.

The gate shuts on the result. No single move lowers a peak of 4 on three storages — moving a disk from a 4 to the 2 turns `4, 4, 2` into `2, 4, 4` — so the next run's split gate does not fire, and the group is left alone until some other gate opens it.

Payback accepts on merit — the plan repairs nothing, so §7.3's exemption does not apply:

```
cost    = 3 × 2 × (2.0 TiB / 200 MiB/s) = 3 × 20 971.52 = 62 914.56 load·s
benefit = (μ·ΔO + κ·ΔA) · H = (1.0 × (8 − 2) − 0.5 × (2 − 0)) × 31 536 000
        = 5.0 × 31 536 000 = 157 680 000 load·s
ratio   ≈ 2 506   ≥  λ = 10   → ACCEPT
```

$\kappa \cdot \Delta A$ is negative — the plan spreads VM 701 over two extra storages — and is paid out of the split term, at w_{701} `= 1` because VM 701 is in `V^split`.

Counterfactual, recorded in the expected file: with `split_vm_footprint: null` the same cluster passes no gate at all and the run ends "no action" — the split rule, and nothing else, is what moves VM 701.

15. Requirements traceability

Original requirement	Where addressed
Proxmox API via user/password	§3.5
Per-disk metrics for all running VMs	§3.1, §3.4 (blockstat via Prometheus, not RRD — see §3.2)
Equalize I/O across storages in a group	§4, §5.3 (C6), §5.4
VMs confined to their storage group	§5.1, §5.3 (C1, C2)
Restrict to shared storages	§5.3 (C2)
2× largest disk free for snapshots — as the per-VM footprint, since a snapshot covers all of a VM's disks	§5.3 (C4)/(C5), §5.3.3
Split VMs larger than a set size over several storages	§5.3.3 (C9), §5.4 μ, §6 split gate, §7.2 benefit; demonstrated §14.9
...including *during* migrations	§8.1 transient invariant
Reserve factor configurable	`snapshot_reserve.factor`, per-storage override
Configurable free space per storage (bytes or %), kept free by migration	`free_space.soft` (§5.3.1), §5.3 (C5), §6 override, §7.3 repair exemption; demonstrated §14.8
Minimal number of migrations	§5.4 β term; demonstrated §14.3
Min % changed traffic before acting (10%)	§6 drift gate
% I/O difference across the group	§6 imbalance gate
Keep a VM's disks together, unless space/IO forces otherwise	§5.3 (C3), §5.4 κ (I/O-weighted, tiny disks free), §7.2 payback benefit; demonstrated §14.3, §14.7
Spread data evenly over the storages (failure risk)	§5.3 (C7), §5.4 δ, §6 capacity gate; demonstrated §14.3
Mathematical optimization formulation	§5
Migration order planned	§8
Automatic or manual confirmation	§9.1 (`dry-run` / `confirm` / `auto`)
Forecasting (Holt-Winters or other)	§10
Configurable decision timeframe, default 24h	`window.lookback`; §3.4
Migration load counted against the benefit	§7; demonstrated §14.5

15.1 Config knob → formula

Every knob must appear in exactly one formula. A knob with no formula is dead configuration and a bug waiting to happen; this table is the audit.

Knob	Where it acts
`load_weights.iotime/ops/bytes`	§4, ℓ_d
`load_weights.read_factor/write_factor`	§4, `raw_X(d)`, applied engine-side before normalization
`window.lookback`	§3.4 reduction range
`window.quantile`	§10.1, the decision statistic
`window.min_coverage`	§3.4, disk data rejection
`groups[].storages[].id` in pattern form (`/…/`)	§11.4 expansion into group membership; the entry's options apply to every matched storage
`groups[].storages[].capability_weight`	§4, `u_s = L_s / c_s`
`snapshot_reserve.factor`	§5.3 (C5), `R_s ≥ f_s·Z_s`, `Z_s` the largest per-VM footprint (§5.3.3)
`snapshot_reserve.split_vm_footprint`	§5.3.3, `T` in `T_v` and `V^split`; (C9); §6 split gate
`free_space.soft` (global, per-storage, per-pattern)	§5.3 (C5), `R_s ≥ soft_s`; §6 reserve override; §7.3 repair exemption
`free_space.hard` (global, per-storage, per-pattern)	§8.1 transient invariant, `max(f_b·…, hard_b)` floor
`snapshot_reserve.count_foreign_volumes`	§5.1.1, U^{sext}
`groups[].storages[].enforce_format`	§5.3.2, `φ(d,s)` in (C2) eligibility and `z_{d,s}` in (C4)/(C5)/(C7)/§8.1; `format=` on `move_disk` (§9.2)
`gates.drift_threshold`	§6 drift gate
`gates.imbalance_threshold`	§6 imbalance gate
`gates.capacity_spread_threshold`	§6 capacity gate (fill fractions from §5.3 (C7))
`gates.cooldown_per_disk/storage`	§5.3 (C2) pinning, §8.1 `concurrency_ok`

Knob	Where it acts
migration.bwlimit_bytes_per_sec	§7.1 duration_d; converted to KiB/s at the API call
migration.source/target_load_weight	§7.1 ω_src, ω_dst
migration.payback_horizon / payback_ratio	§7.2, §7.3 acceptance test
migration.max_single_move_duration	§7.3 hard per-move rule; compared against duration_d *including* the wipe
migration.account_saferemove_wipe	§7.1 duration_wipe_d
migration.wipe_load_weight	§7.1 ω_wipe in cost_d; §7.3 ω_role(m,s) while draining
migration.tiny_disk_bytes	§5.4 D^big (β/γ exemption); §5.3 (C8) small disks follow their VM, (C2) lone-small-disk pin; §7.1 cost_d = 0; §7.3 aggregate-test exemption
execution.locks.*	§9.3 lock wait loop; §5.3 (C2) planning-time pin
execution.source_release.*	§9.3 completion criterion; §8.2 draining state
exclude.include_unused_disks	§3.6 membership of D
exclude.skip_vms_with_snapshots	§3.7, §5.3 (C2) pinning
objective.affinity_counts_pinned_disks	§5.3 (C3) range: all of D (default) or D^mov
report.warn_pinned_load_fraction	§3.7 unreachable-goal warning
objective.alpha_spread/beta_move_count/gamma_move_bytes_per_tib/kappa_vm_affinity/delta_capacity_spread/mu_vm_split_per_tib	§5.4 (the δ term, the I/O-weighted κ term and the μ term also enter §7.2's benefit; μ also §6's split gate)
objective.reserve_violation_penalty	§5.3 (C5), *floor* for the single-stage P alternative
metrics.pvestatd_push_interval	§11.1 rate_window validation; §3.3 verify-metrics
objective.spread_metric	§5.3 (C6), L1 vs min–max
solver.*	§5.5
execution.max_concurrent_migrations/per_storage	§8.1 generalized invariant, concurrency_ok
execution.max_replans_per_run	§9.2 re-plan protocol
execution.time_windows	§9.1 auto mode gating
exclude.*	§5.3 (C2) variable fixing
forecast.model + holt_winters.*	§10.1
proxmox.read_workers	§3.5 read-path concurrency
support.salt_path	§16.3, the persisted anonymization salt — per node by default, cluster-wide if pointed at pmxcfs
support.bundle_dir	§16.4, default output location for collect-testdata
support.max_series_points	§16.2, hard refusal ceiling on the series capture
support.capture_range	§16.2, auto for the maximum over every forecaster's required_range()
monitoring.status_file	§2.4, where apply writes its check_statusfile-format status report; null writes none

---

16. Diagnostic bundles: `collect-testdata` and `--replay`

Two facts motivate this section, and neither is a hypothetical.

One cluster is not a test corpus. Everything in §14 is a hand-built fixture, and the only real cluster this tool has ever run against is the author's own. Every interesting failure so far — a metric silently missing for one disk (§3.3), `/content` returning empty under `Datastore.Audit` (§3.5), a coverage query starved by a too-short `window.lookback` — was found by an operator running the tool and then *describing* what they saw. A description is not a regression test: none of those three is in CI as the shape of data that produced it, because that data never left the cluster.

The data cannot leave the cluster as-is. A PVE cluster's inventory is a list of customer names, hostnames, iSCSI IQNs, PBS fingerprints, ticket numbers in snapshot descriptions and storage comments, and credentials in storage definitions. An operator who wants to help cannot simply tar up API responses, and asking them to redact by hand guarantees either a leak or a refusal.

So: one read-only command that captures everything the engine could ever ask for, replaces every

identifier with a keyed pseudonym that is stable on that host and one-way to everybody else, and writes a self-contained bundle the operator can read before they send it. And one global option that runs the engine against such a bundle with no network at all, so a submitted bundle becomes an executable test case.

16.1 The bundle

A bundle is a **directory**, plus a deterministic `.tar.gz` of that directory for sending. The directory is the canonical form: it is what the scrub audit (§16.6) reads, what `git diff` shows when a bundle is committed to the corpus, and what an operator opens to satisfy themselves before attaching anything to an email.

```
drs-testdata-cluster-3f8a91c2-2026-09-11/
  manifest.json            # schema, versions, what was captured, what failed, counts
  config.yaml              # the anonymized, credential-free effective configuration
  findings.json            # verify-metrics output, as captured
  pve/
    cluster-resources-vm.json
    cluster-resources-storage.json
    storage-definitions.json
    nodes.json
    cluster-tasks.json
    vm-config/<vmid>.json
    vm-snapshots/<vmid>.json
    vm-status-current/<vmid>.json
    vm-pending/<vmid>.json
    storage-status/<node>/<storage>.json
    storage-content/<node>/<storage>.json
  prometheus/
    label-values/<label>.json
    instant/<sha256-of-query>.json
    range/<sha256-of-query-and-range>.json
  SHA256SUMS
```

Every file is JSON (or YAML for `config.yaml`), written with sorted keys, two-space indent, `\n` endings and no trailing whitespace, so a bundle diffs line by line. The Prometheus payloads are **not** individually compressed: the directory form is meant to be read and to compress well in git, and the single gzip of the tarball does the compression once.

`prometheus/instant/<sha256>.json` and `range/<sha256>.json` each carry the query text they were recorded for alongside the response, so a bundle is self-describing and a key miss at replay time (§16.5) can name what it was looking for. The hash is over the *anonymized* query text (plus the range parameters), because that is what the engine will generate when it replays against the bundle's own anonymized config — see §16.5 for why that closes rather than opens a gap.

Determinism. Two captures of an unchanged cluster must produce byte-identical bundles apart from the coarsened capture timestamp in `manifest.json` and the metric samples themselves. That means: every collection is sorted before it is written (by vmid, by storage id, by node, by series label tuple); floats are written with `repr()`'s round-trip form and never reformatted; the tar members are sorted, with `mtime=0`, `uid=gid=0`, empty `uname`/`gname` and fixed modes; and the gzip header carries `mtime=0`. `tests/unit/test_collect.py` asserts this by running the collector twice against the same fakes and comparing bytes — a determinism claim nobody checks is a determinism claim that is already false.

16.2 Capture: the superset, not the configured path

The point of a bundle is that the *author* can run configurations the *operator* never ran. A bundle captured for `forecast.model: quantile` and `solver.backend: heuristic` that only contains 24 h of data is worthless for reproducing a Holt-Winters misfit. So the collector deliberately captures the superset of what any supported configuration could ask for.

PVE. Every read endpoint in §3.5's table, exactly once each, for every VM and every (node, storage) pair the topology pass finds — including cluster_tasks, which §13's crash-recovery scan reads and nothing else does. This is the same read path a plan run performs (§3.5's "expected call count per run"), so the cost is one planning run's worth of API calls, not a multiple of it. The write path is never touched: this command has no code path that can issue move_disk, and --replay has none either (§16.5).

Prometheus. For every group, for each of the six raw metrics of §3.3:

- the instant query verify-metrics issues, so a bundle reproduces §3.3's own checks;
- label_values for each of the three configured labels;
- the quantile_over_time reduction of §3.4 that loadmodel.compute_group_load() consumes, for window.quantile;
- the sum by (vmid, device) (rate(...)) **range** query of §3.4 over the **capture range**:

```
capture_range = max(window.lookback,
                    2 · holt_winters.seasonal_periods · metrics.step,
                    2 · window.lookback)
```

at metrics.step resolution — what holt_winters needs, not the model the operator happens to have selected, **plus** 2 · window.lookback: §10.2's backtest fits on [now-2W, now-W) and checks against [now-W, now], so a capture sized only to a model's own minimum (which can equal window.lookback exactly, e.g. holt_winters tuned so 2 · seasonal_periods · metrics.step == window.lookback) would replay the backtest as permanently "not enough history", independent of how much real history Prometheus actually had. With the defaults that is max(24h, 48h, 48h) = 48h.

This is cheap in *queries* and expensive in *bytes*, which is the right way round. Each range query returns every disk in the group as one response, so the query count is 6 · |groups| · (range_chunks + 3 instant) + 3 label_values, where range_chunks = ⌈capture_range / 1 day⌉ is the day-sized chunking above (7 at the 7 d default, not 1 — as built, REVIEW.md X-09: the printed estimate treated a multi-day range as a single range query per group per metric, undercounting the real HTTP request count by roughly the chunk count). Still tens to low hundreds of queries for any cluster — while the payload is 6 · |disks| · capture_range / metrics.step samples, about 12000 samples per disk at the defaults, or roughly 2.4 million samples for a 200-disk cluster. So:

- the collector **computes the estimate and enforces support.max_series_points against it before fetching anything**, derived from the topology pass it has already done — as built (Y-05): --estimate prints the estimate and exits without fetching; a real capture computes it internally, purely to enforce the refusal below, and prints nothing about it before proceeding;
- support.max_series_points (default 5 million) is a hard refusal, not a truncation, naming the flags that would bring the capture under it;
- --range, --step and --no-series override the capture range, the resolution and the range queries respectively, each recorded in the manifest so the author can see what they are missing rather than inferring it from a short file.

A range longer than Prometheus will serve in one request is **chunked into day-sized sub-queries and stitched**, deterministically (chunk boundaries fall on the range's own start, never on wall-clock "now"), because a backend refusing a 7 d × 5 m request with max_samples exceeded is a configuration difference between deployments and not a reason to hand back a short bundle.

Failures are recorded, never rendered as absence. This is the direct lesson of §3.5's Datastore.Allocate finding: a /content call that returns 200 and an empty list looks exactly like an empty storage. Every captured call carries its outcome in the manifest — ok / http_error / empty / refused / skipped — and the bundle additionally ships findings.json, the verbatim output of verify-metrics against the live cluster

at capture time. (verify-storages's own report needs no separate capture: it is a pure function of the topology and config the bundle already carries in pve/ and config.yaml, so --replay <bundle> verify-storages reproduces it byte-for-byte from data already there — storing it a second time in findings.json would be redundant, not additional coverage. verify-metrics is different because its report depends on a live Prometheus response that is otherwise gone the moment capture ends.) An operator whose bundle records "every storage reported zero volumes" has a bundle that says so, and the author reading it sees a permissions finding instead of a cluster with no disks.

A capture in which some calls failed is still written, and the command exits 1 with the failures summarized. A partial bundle is useful; a bundle that pretends to be complete is not.

16.3 Anonymization

The governing rule: allowlist, never denylist

A field reaches the bundle only if anonymize.py names it. Not "every field except the ones we strip" — every field the engine actually reads, and nothing else. The allowlist is derived from pve.py's own accessors and is the single implementation (AGENTS.md §5) shared by the collector and the corpus scrub audit (§16.6), so the audit cannot drift from what the collector permits.

This is the one decision in this section that must not be softened for convenience. A denylist of "fields known to be sensitive" is wrong on the next PVE release, which will add a field nobody listed: GET /storage alone carries fingerprint, password, encryption-key, keyring, server, portal, target, export, monhost, username, options and comment, of which the engine reads exactly storage, type, content, shared, nodes, disable, saferemove and saferemove_throughput. A VM config carries net0 (MAC addresses), ipconfig0, sshkeys, cipassword, smbios1, description and hookscript, of which the engine reads the disk keys of §3.5's bus regex, lock and template. Everything else is not "redacted" — it is never read into the bundle's data model at all.

Within an allowlisted *value* the same rule applies one level down. A disk value san-a:vm-101-disk-0,size=512G,iothread=1,discard=on is parsed into its volume id, size, format and media and re-serialized from those four; iothread and discard do not survive because nothing reads them.

The pseudonym function, and the salt

```
pseudonym(kind, value) = HMAC-SHA256(salt, kind || "\0" || value)
```

truncated and rendered per kind (below). kind is included so that a node and a storage that happen to share a name do not collide into one pseudonym — they are different objects and must stay different.

The salt is 32 bytes from os.urandom(), generated on first use and persisted at support.salt_path (default /var/lib/pve-storage-drs/anonymization-salt, mode 0600). It is never written into a bundle. The manifest instead carries salt_fingerprint = HMAC-SHA256(salt, "drs-salt-fingerprint"), which lets two bundles be proven to share a mapping — so the author can diff a before-and-after pair from the same cluster — without revealing the salt or letting anyone test a guess at an original name against the bundle.

This is what makes the mapping reproducible in the sense that matters: **the same host maps the same object to the same pseudonym forever**, across runs, across releases and across bundles, until the operator rotates the salt with --new-salt. Two consequences worth stating plainly, because the word "reproducible" invites the wrong one:

- what is stable is the *mapping*, not the payload — a bundle taken a week later has the same names for the same VMs and storages, and different metric samples, which is precisely what makes the pair comparable;

- reproducibility is **per salt file**, so it is per node by default. An operator who wants every node in the cluster to produce the same mapping points `support.salt_path` at a pmxcfs path (`/etc/pve/pve-storage-drs-anon-salt`); the manual documents that as the choice it is, since it also replicates the salt to every node.

Not derived from `/etc/machine-id`, and not from the cluster name. Both are tempting because they need no state file. `machine-id` is disqualified by cloning: it is generated at install time and VM templates, golden images and cloned PVE installs routinely carry the same one, so two unrelated clusters could produce the same pseudonyms for the same vmid — turning the mapping into a cross-bundle correlation key, the exact opposite of the goal. The cluster name is disqualified by entropy: a low-entropy, guessable input to a public HMAC construction is a mapping anyone can invert by brute force over a name list.

Per-kind mapping

Kind	Pseudonym	Notes
Cluster name	`cluster-<8 hex>`	
Node name	`node-<8 hex>`, or `node-<8 hex>.<8 hex>.invalid` when the original was an FQDN	Shape preserved *only* for FQDN-ness, so §3.4's `_escape_promql_regex_literal()` dot-escaping path stays exercised by a bundle from a cluster that uses FQDNs. `.invalid` per RFC 2606
Storage id	`stor-<8 hex>`	Lowercase letters, digits and `-` only — deliberately the narrowest shape real ids take, so a pseudonym is accepted anywhere an id is
Group name	`group-<8 hex>`	`groups[].name`, `--group` and every `state.json` key that embeds it move together
vmid	`100 + (HMAC mod 899_900)`, deterministic linear probing on collision	Must stay an integer: it is a PromQL label value, a config value in `exclude.vmids` and a component of a volume id. Probing walks candidates in order of *original* vmid so the result never depends on iteration order
Volume id	rebuilt as `<storage-pseudonym>:<prefix>-<vmid-pseudonym>-disk-<n>[.<ext>]`	The structural prefix (`vm-`, `base-`), the disk index and an optional trailing `.<ext>` (e.g. `.qcow2`, PVE's own format marker on the volume name for qcow2-on-shared-LVM and similar cases, confirmed against a real cluster) survive because §3.5's parser and §3.6's movability rules read them, and because the same string must still match its own storage-content listing entry at replay time. A volume whose name matches neither the pattern nor its extension-bearing form is dropped
Tag, pool	`tag-<8 hex>`, `pool-<8 hex>`	Pseudonymized rather than dropped because `exclude.tags` filters on them; the same mapping rewrites `exclude.tags` in `config.yaml` so the exclusion replays

Kind	Pseudonym	Notes
UPID	rebuilt from anonymized parts	`UPID:{node}:{pid}:{pstart}:{starttime}:{type}:{id` the grammar confirmed against a real cluster in `crashrecovery.py`. Node, id and user are mapped; `pid`/`pstart` are replaced with fixed constants (they identify a process on a named host and nothing the engine reads). A task entry's own `id` field is the same vmid, so it is set to the id the mapped UPID carries — never copied raw — and dropped from an entry with no UPID; the scrub audit checks the two agree
Username / realm	`user-<8 hex>@realm`	Only ever seen inside a UPID
Metric name	**as built (REVIEW.md X-08): not anonymized, carried verbatim**	The operator's own configured names (`config.metrics.read_ops` etc.) reach the bundle in `config.yaml`, the query text and every `label_values` capture, unchanged. This table used to promise canonical `drs_rd_operations`/`drs_wr_bytes`/... names instead; that mapping was never built. The as-built behaviour leaks nothing (the names are already the operator's own config, already in `config.yaml`) and is self-consistent by construction rather than by a second mapping that could drift from it
Prometheus label name	the configured `vmid`/`device`/`node` label names are kept verbatim; every other label is **dropped**	Label *names* are chosen by the operator's Telegraf config and can be identifying (`customer`, `datacenter`), but the three configured ones must survive or the bundle cannot be joined. They are already in `config.yaml`, so they leak nothing the config does not
Device name (`scsi0`, `efidisk0`, `tpmstate0`, `unused3`)	**not anonymized**	A closed enumerated set (§3.5's bus regex) carrying no identity, and §3.6's movability rules and §3.7's pinning read them directly. Mapping them would destroy the behaviour the bundle exists to reproduce
VM name, description, notes, comment, snapshot name	**dropped**	Free text, nothing reads it. `show-load` under replay prints the vmid where it would print a name

Timestamps are rebased. Every absolute timestamp — sample timestamps, `ctime` on a volume, `snaptime`, a UPID's `starttime` — is shifted by a single per-bundle offset so that the capture window starts at a fixed synthetic epoch, and the manifest records the *duration* and the *wall-clock-hour alignment* rather than the date. Load has a daily and a weekly shape that the seasonal forecasters of §10.1 must be able to learn, so hour-of-day and day-of-week are preserved modulo the week; the absolute date is not, because "this cluster's I/O spiked on the afternoon of the 3rd" is a correlation handle for anyone who reads incident reports.

Unmapped means dropped. If the collector meets an identifier of a mapped kind that is not in its map — a volume id naming a storage outside every group, a UPID for a node that has left the cluster — it drops the containing record, logs a warning and counts it in the manifest. It never passes the original through and never invents a mapping on the spot. Fail closed: the failure mode of "dropped a record the author

might have wanted" is a worse bundle, and the failure mode of the alternative is a leak.

The configuration in the bundle

`config.yaml` is the effective configuration as `config.py` resolved it, with four classes of change. It matters that this file is right: it is what `--replay` runs on, so an inconsistency between it and the recorded responses shows up as a key miss (§16.5) rather than as a wrong answer.

- **Credentials and endpoints are dropped, not blanked.** `proxmox.host`, `.user`, `.password`, `.token_id`, `.token_secret`, `.fingerprint`, `prometheus.url` and every `prometheus` auth field are absent from the file, not present-and-empty. Replay needs none of them (§16.5), and a blanked field invites a future reader to wonder what was in it.
- **Every identifier is mapped with the same mapping as the data**: `groups[].name`, `groups[].storages[].id`, `exclude.vmids`, `exclude.storages`, `exclude.disks` and `exclude.tags`. A knob the engine compares against a captured value must move together with it or the exclusion silently stops applying, which is a quiet behaviour change in a file that claims to reproduce a run. `state.path`, `support.salt_path` and `support.bundle_dir` are replaced with fixed placeholders — a filesystem path is free text and routinely carries an organization's name.
- **`/…/` storage patterns are expanded at capture time** (§11.4) and the bundle carries the literal, anonymized ids they matched. A pattern cannot survive anonymization: its text names real storages, and a pattern rewritten over pseudonyms would match nothing on re-expansion. The manifest records each pattern entry's *existence* and the number of storages it matched, so the author can see that a group was pattern-driven; the bundle loses the ability to test §11.4's expansion itself, which is a real and deliberate gap named here rather than discovered later.
- **`metrics.extra_selector` is rewritten**, not carried, for the same reason: it is operator-authored PromQL over real label values. The collector replaces it with the equivalent anonymized node alternation — the selector §3.4's default tier would have built — and the manifest flags that it did. A bundle from a cluster whose selector does something the default tier cannot express is therefore not byte-faithful to its live queries, and says so in the manifest instead of quietly producing a plan from differently-scoped data.

Everything else — `window`, `snapshot_reserve`, `free_space`, `gates`, `migration`, `objective`, `solver`, `execution`, `forecast`, `report`, the per-storage `capability_weight`/`reserve_factor`/ `free_space`/`enforce_format` — is carried **verbatim**. Those knobs are the test case.

What is deliberately preserved

Sizes, capacities, used/free bytes, per-disk load samples, storage `type`, `content`, `shared`, `disable`, `saferemove`, `saferemove_throughput`, disk `format`, the PVE and Prometheus version strings, the count and grouping of everything, and the entire configuration except its credentials. All of it is what makes a bundle a test case rather than a shape, and none of it is an identifier.

Threat model, stated honestly

The bundle contains **no direct identifier and no secret**: no hostname, no IP, no IQN, no fingerprint, no key, no password, no customer-authored free text. That is a property the scrub audit mechanically checks (§16.6).

It is **not** anonymous against someone who already knows the cluster. A cluster with three nodes, two 8 TiB LUNs and 1214 VMs is recognizable to anyone who has seen it, and the load shape of a particular workload is a fingerprint. The tool must not claim otherwise, and the manual must say so in the same paragraph that explains how to run the command: a bundle is safe to hand to the author, it is not safe to publish, and the decision to submit one is the operator's informed decision and not a formality. `tests/corpus/README.md` carries the same statement for the author's side of the exchange.

16.4 The command

```
pve-storage-drs -c /etc/pve/drs.yaml collect-testdata [options]
```

Option	Default	Effect
`-o, --output DIR`	`support.bundle_dir` (`/var/lib/pve-storage-drs/testdata`)	Where the bundle directory and its tarball are written
`--estimate`	off	Print the query count and payload estimate, then exit without fetching
`--range DURATION`	the §16.2 capture range	Override the range of the series capture
`--step DURATION`	`metrics.step`	Override the series resolution
`--no-series`	off	Capture topology, instant queries and findings only. Produces a small bundle that cannot exercise §10's forecasters
`--no-archive`	off	Write the directory only, no tarball
`--salt-file PATH`	`support.salt_path`	Use a different salt file
`--new-salt`	off	Generate a fresh salt, replacing the persisted one. Logged at warning: bundles made before and after no longer share a mapping

It is a **read-only command and a dry-run-only command**: `--mode confirm` or `--mode auto` alongside it is a usage error, not a silently ignored flag, because the one thing an operator collecting diagnostics must never risk is having started a migration. Exit `0` when every capture step succeeded, `1` when a bundle was written with recorded failures, `2` on a usage error.

Human output names the bundle path, its size, the counts (groups, storages, VMs, disks, series, samples), the salt fingerprint, and every recorded failure. `--json` emits the manifest.

16.5 Replay: the `--replay` global option

```
pve-storage-drs --replay ./drs-testdata-cluster-3f8a91c2-2026-09-11 plan --json
```

`--replay PATH` is a **global** option (§11.3), not a subcommand, and that is the whole design: it substitutes both clients at the one place each is constructed, so every read-only command — `verify-metrics`, `verify-storages`, `show-load`, `plan`, `explain` — runs unchanged against the bundle. One option, five commands under test, and no second copy of the pipeline to keep in step with the real one.

- The configuration comes from the bundle's own `config.yaml` unless `-c` is also given, in which case the file's knobs are used against the bundle's data — that is how the author runs the operator's cluster through a solver backend, a `spread_metric` or a forecaster the operator never selected.
- **`apply` refuses.** So does any `--mode` above `dry-run`. Exit `2`.
- **No network is reachable.** The replay clients are constructed *instead of* the real ones and hold no session, and a bundle's `config.yaml` carries no credentials or endpoints, so a bug cannot fall back to a live cluster. A unit test asserts that constructing the replay path never constructs a `requests.Session` or a `ProxmoxAPI`.
- **A key miss is a loud, specific error**, never an empty result: "bundle has no recorded response for `sum by (vmid, instance) (rate(drs_rd_operations[600s]))` over 7 d at 300 s — captured with `--step 60s`?" This is the failure mode that would otherwise make a replay silently diverge from the live run, and §13 gains a row for it. It is also why the cache key is the anonymized query text: the engine under replay reads the anonymized config, generates the anonymized query, and hits or misses deterministically — a key computed over the *original* text could only ever miss.
- `state.json` under replay is read from the bundle if present and written nowhere. A replay never

touches the host's real state.

- **vm-pending/<vmid>.json is the one deliberate exception to the "key miss is loud" rule above.** A bundle captured before section 3.8 added this call has no vm-pending/ directory at all; a missing file there is treated as "nothing pending", not a bundle defect, so an older committed bundle keeps replaying rather than every one of them needing recapture the moment this file was added. A bundle captured after 3.8 always writes the file for every considered VM, even when its own list is empty, so this fallback can only ever trigger on a genuinely older bundle.

--replay ships with the tool rather than living in tests/, for two reasons: the operator needs it to check their own bundle before sending it ("does plan against this bundle show the problem I am reporting?"), and a debugging aid that only exists inside a test harness is one the author cannot hand back to the operator when the answer is "your bundle replays fine, so the difference is in your live cluster".

16.6 The corpus: tests/corpus/

Submitted bundles land in **tests/corpus/**, one directory per bundle, and become part of the test suite.

```
tests/corpus/
  README.md                      # what goes here, how to add one, and the honesty statement
  <bundle-name>/                 # the unpacked bundle directory, exactly as collected
  <bundle-name>.submission.yaml  # provenance and consent (below)
  <bundle-name>.expected.json    # generated, never hand-edited
  validate_corpus.py             # the generator and the --check gate
```

<name>.submission.yaml records what the repository needs to know about data it did not produce: who submitted it and under what terms (redistribution under AGPL-3.0-or-later, as the rest of the repository), the PVE and Prometheus versions, the collector version, whether the submitter reviewed the bundle before sending, and what the bundle is *for* — the behaviour it is expected to reproduce. A bundle with no such file is not a test case, it is an unattributed dump, and the suite fails on it. debian/copyright gains a stanza for tests/corpus/* when the first bundle lands; the corpus is not installed by the package.

What the suite asserts

The central difficulty is that **a real bundle has no known-correct answer**. Nobody can enumerate 3^1214 assignments, so unlike §14's fixtures a corpus bundle cannot assert "the plan is optimal". Four kinds of assertion that do hold:

1. **The scrub audit** — cheap, and it runs on every bundle in the corpus on every make check, before anything else. It walks every pve/ and prometheus/ file, every key (X-03: this used to be true of only the five top-level pve/ files; a prometheus/ series' metric dict is checked against the bundle's own configured label names, read from its config.yaml) and fails on: any key not in anonymize.py's allowlist (the same allowlist the collector uses, so the two cannot drift); any value matching an IPv4 or IPv6 literal, an email address, an iqn./naa./wwn. prefix, a PEM block, a 64-hex-or-longer run, a JWT-shaped string, a public-suffix hostname (.com/.net/.org/.local/.internal/.corp/.lan/.home/.example/.test — X-02's own reproduction found the original four-suffix list missed a transport failure's own .internal/.corp/.example endpoint), or an unredacted transport host='...' literal (X-02's own backstop for the same class); any node or storage name not matching the pseudonym grammar; any absolute timestamp outside the synthetic epoch window. manifest.json/findings.json have no anonymize.py allowlist of their own (hand-authored bundle metadata, not a captured PVE/Prometheus object shape) and get the value-pattern checks only. It is a second line of defence that assumes the collector has a bug, which is the only useful assumption to make about a privacy control.
2. **Invariants, not optima** — reconstructed from what plan --json's own group report already records per variant (X-07): every move in the plan is to a storage (C2) permits for that group; no accepted move carries exceeds_max_duration: true (§7.3's duration rule); a disk payback rejected never

also appears as an accepted move (structurally). These are checkable without knowing the optimum, and they are what check_invariants() actually asserts. Three properties this bullet used to claim as checked and is not: §8.1's per-step transient predicate needs the emitted *order*, which no plan --json field carries (the Σ r_s half of this gap is closed as of phase 13 — the payback block's reserve_shortfall_bytes_before/_after are the current and final Σ r_s, and check_invariants() asserts the part of it that is an invariant: the plan never *raises* the shortfall. Not = 0: a group with no feasible repair legitimately ends above zero); "the objective the scheduler was handed equals the objective recomputed from the final assignment" needs the seven-term breakdown, which today only explain --json emits. A real and deliberate gap, named here rather than discovered later (the same shape as this section's own pattern-expansion gap above) — either sweep explain --json too or add the missing fields to plan --json's group report to close it.

3. **MILP versus heuristic, on real data.** Run the same bundle through CBC and the heuristic and assert the MILP backend's after_spread (plan --json's already-computed post-plan spread fraction) is not worse than the heuristic's. This is the cross-check §14 can only perform on six disks, and it is the single highest-value thing a real bundle buys: a heuristic that beats the MILP means the two have drifted apart on the shared feasibility or objective functions, which AGENTS.md §5 exists to prevent and which no synthetic fixture of this size can detect. **CBC-versus-CP-SAT agreement was never checked, and is now uncheckable** (X-07; the CP-SAT backend was removed, AL-02): a first attempt comparing after_spread against solver.mip_gap as a relative tolerance produced real disagreement on a committed bundle under --full-matrix (cbc 0.0016 vs. cpsat 0.0034-0.0112 across several variants) that was legitimate under mip_gap on the *objective* the solvers actually optimize — mip_gap bounds suboptimality of the seven-term objective, not of any one derived quantity taken in isolation, and a tiny baseline spread turns a small absolute gap into a large relative one. Getting this right needs the objective breakdown itself, the same gap named in check 2 above.
4. **Regression.** <name>.expected.json records, per variant, the gate verdict, the plan (as a sorted list of moves with their per-move costs), the payback arithmetic and the findings — as built: plan --json's group report carries neither an emitted *order* nor the seven-term objective breakdown (Y-04; the exact gaps checks 2 and 3 above name), so no expected file can record those two — it *does* record the payback block's reserve_shortfall_bytes_before/ _after and repair_exempt since phase 13. It is generated by validate_corpus.py and asserted current by validate_corpus.py --check, exactly as tests/fixtures/generate_expected.py is for §14 — and for the same reason: an expected file that a human may edit is an expected file that will be edited to match a bug.

The variant matrix

Per bundle, the matrix the generator sweeps: solver.backend ∈ {cbc, heuristic} × objective.spread_metric ∈ {l1, minmax} × forecast.model ∈ {quantile, holt_winters} × objective.beta_move_count over a small sweep, with everything else from the bundle's own config.yaml. --check (part of make check) runs a narrow sweep instead: both backends, the first spread metric and beta, and forecast.model ∈ {quantile, the bundle's own configured model}. A variant whose forecaster is unavailable in the running environment is **skipped and recorded as skipped**, never silently dropped: statsmodels is an optional dependency (AGENTS.md §9.1) and a corpus result that quietly means "quantile only" is a corpus result that lies.

Size, and bundles too big to commit

A committed bundle is reviewable data, so there is a ceiling. Rule: a bundle is committed only if its directory is under **8 MiB**, which at the §16.2 estimate is a cluster of a few dozen disks or a --no-series capture of a large one. Anything bigger is kept outside the repository and found via DRS_CORPUS_DIR, a colon-separated list of directories the suite also walks; CI sets it where a large corpus is available and the suite runs the committed bundles alone where it is not. An empty tests/corpus/ is a clean pass, not a failure — the suite must be green on a fresh clone with no bundles at all.

make corpus runs the full matrix over every discoverable bundle. make check runs the scrub audit over every bundle plus the invariant and regression assertions for the committed ones, because those are bounded by the 8 MiB rule; the full matrix over a large external corpus is a CI job of its own.

16.7 Modules, phases and the honest status

Three new modules, split along the line that matters — the part that must be right is pure:

Module	Responsibility
anonymize.py	The allowlists, the pseudonym function, the per-kind mappings, the timestamp rebase. Pure, no I/O, and the shared implementation behind both the collector and the scrub audit
collect.py	Capture orchestration, the estimate, chunking, the manifest, the deterministic writer
replay.py	The bundle reader and the two replay clients, satisfying the same interfaces metrics.py and pve.py define

config.py gains a support block: support.salt_path, support.bundle_dir, support.max_series_points, support.capture_range (auto for §16.2's computed maximum, or an explicit duration). All four appear in §15.1, in config/drs.example.yaml and in the manual, in the commit that implements them and not before.

§12 gains **phase 10**: collect-testdata + --replay + tests/corpus/, done when a bundle collected from the author's own cluster replays to the same plan the live run produced, the scrub audit passes on it, and the determinism test is green.

anonymize.py, collect.py, replay.py, cli.py's wiring (collect-testdata, the global --replay) and tests/corpus/validate_corpus.py are implemented, with unit and CLI-level tests covering capture, every anonymization rule, the deterministic writer, and a full capture → write → replay round trip. tests/corpus/ and its README.md existed from the commit that added this section, ahead of the code, so the place to put a bundle was already there and documented; phase 10's own "done when" — a bundle collected from the author's own cluster replays to the same plan the live run produced, the scrub audit passes on it, and the determinism test is green — is met: bzed-dev-cluster-24h, a --range 24h capture from the project's own dev cluster, is in tests/corpus/, replays all five read-only commands to the same result the live run produced, and passes the scrub audit with zero violations.